Neo and SuperNeo: Post-quantum folding with pay-per-bit costs over small fields

Wilson Nguyen
Stanford University, New York University, and Microsoft Research

Srinath Setty
Microsoft Research

Abstract. We construct the first folding scheme that simultaneously achieves six desirable properties: plausible post-quantum security, pay-per-bit commitment costs, field-native arithmetic (the sum-check and norm checks run purely over a small field), support for general (non-SIMD) constraint systems, small-field support (e.g., Goldilocks), and low recursion overheads. No existing scheme satisfies all six: group-based schemes (e.g., HyperNova) lack post-quantum security and are tied to large elliptic-curve fields; lattice-based schemes (e.g., LatticeFold) require expensive ring arithmetic, lose pay-per-bit costs, and impose SIMD constraints; and hash-based schemes (e.g., Arc) incur large verifier circuits.

We present two lattice-based folding schemes for CCS—an NP-complete relation generalizing R1CS, Plonkish, and AIR—called Neo and SuperNeo. Neo satisfies five of the six properties but requires SIMD constraint systems; SuperNeo removes this restriction and satisfies all six. Both run a single invocation of the sum-check protocol over a small field extension and achieve pay-per-bit costs via new folding-friendly instantiations of Ajtai commitments under the Module-SIS assumption. At the core of our constructions are two new norm-preserving embeddings of field vectors into ring vectors that respect an evaluation homomorphism required for folding. We also introduce *interactive reductions*, a framework that generalizes reductions of knowledge and enables modular security proofs for composed lattice-based protocols.

1 Introduction

A folding scheme [57] is a cryptographic primitive that reduces the task of checking that two instance-witness pairs are in some NP relation to the task of checking that a single instance-witness pair is in the same relation. As an example, for a circuit C and two public inputs (i.e., instances) x_1 and x_2, a folding scheme reduces the task of checking that there exist witnesses w_1 and w_2 such that $C(w_1, x_1) = 1$ and $C(w_2, x_2) = 1$ to the task of checking that there exists a single witness w for a specific public input x such that $C(w, x) = 1$. Furthermore, the verifier's work in a folding scheme is limited to roughly taking the weighted sum of the commitments to the underlying witnesses. By using a folding scheme in a recursive manner, one can continually fold many instance-witness pairs into a single instance-witness pair, providing powerful recursive succinct argument primitives such as incrementally verifiable computation (IVC) [82] and proof-carrying data (PCD) [13].

Importance of folding schemes: prover efficiency and efficient recursion. A modern approach to construct SNARKs [47, 66] is to combine a polynomial interactive oracle proof (PIOP) [23,30,78] with a polynomial commitment scheme (PCS) [46]. However, this yields a "monolithic" SNARK whose prover must prove a fixed-sized computation at once. To scale to larger computations, one typically breaks the computation into smaller pieces and uses SNARK recursion (a la IVC or PCD) to produce a succinct argument [11]. Folding schemes provide a more direct and efficient alternative: they allow recursion to operate at the "statement" level (i.e., prior to producing a PIOP or a PCS evaluation argument), yielding two concrete benefits. First, recursion overheads are far lower: with Nova [57], folding a proof takes only ≈10,000 R1CS constraints, whereas traditional SNARK recursion takes millions [29, 31]. Second, the prover incurs far less work by avoiding a full PIOP and PCS evaluation argument: in monolithic SNARKs such as Marlin [30], the prover performs at least 20× higher work over simply committing to a witness, whereas with state-of-the-art folding schemes [20, 35, 55, 56], the prover's work is dominated by the cost to commit to a witness. This results in at least an order of magnitude speedup over monolithic SNARKs, and up to two orders of magnitude when the witness contains values from a small subset of the entire field.[1]

A motivating application: post-quantum signature aggregation. Ethereum's consensus layer relies on BLS signatures [18], which offer efficient aggregation: a single pairing check suffices for hundreds of thousands of validator attestations. However, BLS signatures are not post-quantum secure. Ethereum's planned transition to a post-quantum scheme (e.g., the hash-based XMSS [44]) reintroduces the scalability challenge, since such schemes lack the algebraic structure of BLS and their signatures are large and expensive to verify individually. A natural solution is to use recursive SNARKs to aggregate these signatures [34] in a distributed manner (a la proof-carrying data). Folding schemes are well suited here: each signature can be folded into the aggregate proof, yielding significantly lower latency and prover cost at each recursive step than traditional SNARK recursion—a critical advantage when up to a million signatures must be aggregated within time-sensitive slots of a consensus protocol.

Realizing this approach requires a folding scheme that is both efficient *and* plausibly post-quantum secure. More broadly, what properties should a practical folding scheme satisfy?

1.1 Six desiderata for a practical folding scheme

State-of-the-art folding schemes [20, 35, 53–57] have converged on an efficient "recipe": the prover commits to a witness with a linearly homomorphic scheme

[1] If the witness contains "small" field elements (e.g., from the set $\{0, 1, \ldots, 2^{32} - 1\}$), state-of-the-art folding schemes perform more than two orders of magnitude less work than a monolithic SNARK prover such as Marlin [30]. Proof systems such as Spartan and variants [78, 79] incur lower overheads than Marlin, but they must still produce a PIOP and PCS evaluation argument.

(Pedersen or KZG) and employs sum-check-type techniques [62]. For instance, the prover in HyperNova [55] performs a *single* multi-scalar multiplication (MSM) to commit to its witness, with costs that scale with the bit-width of witness values ("pay-per-bit"). However, these schemes are not post-quantum secure and are tied to ≈256-bit elliptic-curve fields. Below, we distill six properties that a practical folding scheme should satisfy.

D1: Post-quantum security. The scheme should be plausibly secure against quantum adversaries. Group-based folding schemes [20, 35, 55–57] rely on the hardness of the discrete logarithm problem, which Shor's algorithm [80] can efficiently solve on a quantum computer.

D2: Pay-per-bit commitment costs. The cost to commit to a witness should scale with the bit-width of the witness values: for example, committing to a vector of b-bit values should be roughly b times cheaper than committing to values that span the full field. Group-based schemes achieve this via Pedersen or KZG commitments. LatticeFold, however, relies on the NTT embedding to map field vectors into ring vectors. Because the NTT is not a norm-preserving map, the resulting ring vectors have arbitrary norm regardless of the bit-width of the original witness elements. Since the Ajtai commitment scheme requires decomposing these arbitrary-norm ring elements for binding, the commitment cost is the same whether the original values are 1-bit or 64-bit. Hash-based schemes (e.g., Arc [24]) also lack this property, since their commitment costs are determined by the code rate and security parameter, not by witness bit-widths.

D3: Field-native arithmetic. Besides committing to the witness, the prover's and the verifier's work—in particular, the norm check and the sum-check protocol—should operate natively over a prime field (or extension field), without performing expensive polynomial ring arithmetic. Group-based and hash-based schemes satisfy this property. LatticeFold [14], however, runs the sum-check protocol over a cyclotomic polynomial ring rather than over a prime field (or extension field); ring operations are 10–100× more expensive than field operations.[2]

D4: General constraint systems. The scheme should support general NP-complete constraint systems such as CCS [79] over a single witness vector, without requiring the constraint system to be "data parallel" (SIMD). Group-based and hash-based schemes satisfy this. LatticeFold, by contrast, packs a batch of independent constraints defined over a small prime field into a single constraint over a cyclotomic polynomial ring [14, Remark 4.1], imposing a SIMD requirement. Lova [37] avoids this issue but at the cost of only supporting the subset sum relation, not CCS.

D5: Small-field support. The scheme should work natively over small prime fields, including popular SNARK-friendly fields such as Goldilocks. By "small"

[2] Benchmarks report that a polynomial ring multiplication costs ≈213 ns [67], whereas a field multiplication with M61 costs a fraction of a nanosecond.

we mean fields whose modulus q fits within a machine register—for example, M61 ($q = 2^{61} - 1$) or Goldilocks ($q = 2^{64} - 2^{32} + 1$). Small-field support is important for two reasons: such fields offer arithmetic that is an order of magnitude faster than 256-bit arithmetic, and SNARK-friendly fields enable efficient proof compression using existing SNARKs (e.g., Spartan with a FRI-based polynomial commitment scheme). Group-based schemes are tied to the scalar fields of elliptic curves, which are ≈256 bits for security. LatticeFold's cyclotomic rings of the form $X^d + 1$ (with d a power of 2) cause popular fields like Goldilocks to fully split the ring, ruining security [14, §3.3]; furthermore, supporting small fields requires a non-trivial extension field degree $t \mid d$, introducing a $t\times$ multiplicative overhead in the protocols due to the need for t-parallel repetition [14, §3.3, §5].

D6: Low recursion overheads. The recursive verifier circuit should be small enough that the per-step prover cost of IVC remains practical. Group-based schemes achieve this: Nova's and NeutronNova's verifier circuits are constant-sized: ≈10,000 R1CS constraints. Hash-based schemes suffer from large verifier circuits; for example, Arc [24] requires $2 \cdot \lambda / \log(1/\rho)$ Merkle tree openings, translating to ≈1,600,000 R1CS constraints at $\lambda = 128$ and $\rho = 1/2$ with Poseidon [43]. Prior to this, Bünz et al. [25] provide a different hash-based scheme with even worse verifier circuit overheads [24, Table 1], and only provides "bounded depth" IVC. Lova also incurs extreme overheads, reporting a prover time of ≈3,000 seconds for a subset sum instance of length 2^{19} [37, Table 2], compared to 500 ms for Nova on an R1CS instance of the same size. More broadly, folding schemes that rely on ring sum-check techniques—such as LatticeFold [14] and SALSAA [59]—inherit high recursion overheads because the recursive verifier must hash ring elements rather than field elements. For example, a single ring element in LatticeFold occupies $64 \times 64 = 4{,}096$ bytes, compared to 32 bytes for a 256-bit field element in HyperNova, resulting in 128× more data for the verifier circuit to hash. Achieving constant verifier circuit size like Nova or NeutronNova in the lattice setting is difficult and remains an open problem. Our goal is to achieve logarithmic recursion overhead (with similar constants) analogous to HyperNova.

Research question. Can we build a folding scheme that satisfies all six desiderata—in particular, one that is post-quantum secure, works natively over small prime fields, and matches the efficiency of state-of-the-art group-based schemes?

Figure 1 summarizes the landscape. No existing folding scheme satisfies all six desiderata. Group-based schemes meet D2–D4 and D6 but fail D1 and D5. Hash-based schemes achieve D1 but sacrifice D2 and D6. LatticeFold, LatticeFold+, and SALSAA each achieve D1 but fail D2–D4 and D6; LatticeFold and LatticeFold+ additionally fail D5. Neo satisfies D1–D3, D5, and D6, but requires SIMD constraint systems (D4). SuperNeo is the first scheme to satisfy all six.

1.2 Our work: Neo and SuperNeo

We present Neo and SuperNeo, the first folding schemes to satisfy all six desiderata (Figure 1). Our constructions are lattice analogs of HyperNova [55]: the prover

	D1	D2	D3	D4	D5	D6
Group-based						
HyperNova [55]	✗	✓	✓	✓	✗	✓
NeutronNova [56]	✗	✓	✓	✓	✗	✓
Hash-based						
Arc [24]	✓	✗	✓	✓	✓	✗
Lattice-based						
LatticeFold [14]	✓	✗	✗	✗	✗	✗
Lova [37]	✓	✗	✓	✗	✓	✗
Neo (this work)	✓	✓	✓	✗[†]	✓	✓
LatticeFold+ [16]	✓	✗	✗	✗	✗	✗
SALSAA [59]	✓	✗	✗	✗	✓	✗
SuperNeo (this work)	✓	✓	✓	✓	✓	✓

[†]Neo requires a SIMD constraint system (D4) and is subsumed by SuperNeo, which removes this requirement. The table lists schemes in the order they appeared; we present Neo and SuperNeo separately because Neo was independently preprinted and several subsequent works build on it, so separating the two simplifies attributing techniques to the correct scheme.

Fig. 1: Comparison of folding schemes against the six desiderata. ✓ = satisfied, ✗ = not satisfied. See Section 1.1 for definitions.

commits to a CCS [79] witness using a lattice-based commitment scheme with pay-per-bit costs, runs a single invocation of the sum-check protocol over an extension of a small prime field[3], and achieves plausible post-quantum security under a standard structured lattice assumption (Module-SIS). Both schemes also provide multi-folding, which folds multiple CCS instances at once, amortizing the decomposition costs required to manage lattice norm growth. By applying standard compilers from folding schemes to IVC [55, 57] and PCD [85], we obtain a plausibly post-quantum IVC/PCD scheme. Since our constructions natively support SNARK-friendly fields like Goldilocks, they enable efficient proof compression using Spartan [78, 79] with a FRI-based polynomial commitment scheme [10]—without requiring any non-native arithmetic or field emulation.

The key technical challenge is *embedding* field vectors (CCS witnesses) into the ring vectors that Ajtai commitments operate over, while preserving the algebraic structure—norm bounds, evaluation homomorphisms—that a sum-check-based folding scheme like HyperNova requires. We introduce two new norm-preserving embeddings (the **Neo** and **SuperNeo embeddings**) that resolve this challenge, and a new security framework (**interactive reductions**) that enables modular proofs of knowledge soundness for lattice-based protocols. We detail the problems with prior approaches and our solutions in the following sections.

[3] When using a 64-bit field, a degree-2 extension is sufficient for 128 bits of security.

1.2.1 Challenges and prior solutions Achieving desiderata D2–D4 and D6 for a lattice-based folding scheme requires solving a common problem: how to embed field vectors into the ring vectors that Ajtai commitments [2] operate over. Any such embedding must support protocols that check both the norm of the committed ring vectors and CCS constraints on the underlying field vectors. Below, we describe the Ajtai commitment scheme and the challenges that prior solutions leave open.

Ajtai Commitment Scheme [2, 75] (Informal) [4]

- $\mathsf{Setup}(1^\kappa, n \in \mathbb{N}) \to A \in \mathrm{R}_\mathbb{F}^{\kappa \times n}$, where A is a random matrix over the polynomial ring $\mathrm{R}_\mathbb{F} := \mathbb{F}[X]/(\phi(X))$ with modulus $\phi(X)$ being a cyclotomic polynomial of degree d.
- $\mathsf{Commit}(A, \boldsymbol{z} \in \mathrm{R}_\mathbb{F}^n) \to c$, where $c := A \cdot \boldsymbol{z} \in \mathrm{R}_\mathbb{F}^\kappa$ is a binding commitment to message $\boldsymbol{z}$ if the norm $\|\boldsymbol{z}\|_\infty < b$ is small enough.

Problem 1: Inefficient constraint checking & lack of algebraically friendly embeddings Prior work on lattice-based folding schemes [15] relied on the Number Theoretic Transform (NTT) [5, 61, 75, 77] to embed field vectors into ring vectors. The Number Theoretic Transform is a ring isomorphism between the ring $\mathrm{R}_\mathbb{F}$ and a product space of extension fields $\mathbb{F}_{q^t}^{d/t}$, so each ring operation naturally simulates a Single Instruction, Multiple Data (SIMD) [40, 41] operation over the underlying (d/t)-tuple of field elements. For $\cdot \in \{+, \times\}$,

$$\begin{array}{ccc} (a_1, \ldots, a_{d/t}) \in \mathbb{F}_{q^t}^{d/t} & \underset{\mathsf{iNTT}}{\overset{\mathsf{NTT}}{\longleftrightarrow}} & \boldsymbol{a} \in \mathrm{R}_\mathbb{F} \\ \Big\downarrow \cdot(b_1, \ldots, b_{d/t}) & & \Big\downarrow \cdot \boldsymbol{b} \\ (a_1 \cdot b_1, \ldots, a_{d/t} \cdot b_{d/t}) \in \mathbb{F}_{q^t}^{d/t} & \underset{\mathsf{iNTT}}{\overset{\mathsf{NTT}}{\longleftrightarrow}} & \boldsymbol{a} \cdot \boldsymbol{b} \in \mathrm{R}_\mathbb{F} \end{array}$$

Hence, d/t field vectors $z^{(1)}, \ldots, z^{(d/t)} \in \mathbb{F}^n$ can be embedded into a single ring vector $\boldsymbol{z} \in \mathrm{R}_\mathbb{F}^n$. By adapting a technique from [19], prior work Latticefold [15, Sec 3.3] showed how to express the norm constraint on a ring vector $\boldsymbol{z}$ as Hadamard product constraints $\prod_{i<b}(\hat{\boldsymbol{z}}_j - \boldsymbol{i}) = 0$ over related ring vectors $\hat{\boldsymbol{z}}_1, \ldots, \hat{\boldsymbol{z}}_t$ whose underlying embedded field vectors are the coefficient vectors of the committed ring vector $\boldsymbol{z}$. Since the NTT transformation is a ring isomorphism, they also showed that field constraints (like CCS) over the embedded field vectors could be checked as ring constraints over the committed ring vector $\boldsymbol{z}$ itself. To check both the Hadamard and CCS constraints on these ring vectors, Latticefold relies on the celebrated sum-check protocol [62] to reduce checking these constraints to checking random multilinear evaluations of the ring vectors themselves. The main downside of this approach is that the prover and verifier have to execute the sum-check protocol over the ring itself rather than just the underlying field

[4] To streamline our exposition, we directly discuss the more efficient variant of Ajtai commitments based Module Short-Integer-Solution (M-SIS), because they serve as the basis of most lattice-based proof systems [12, 15, 16, 19, 69, 75].

for which the original CCS constraints are defined over. As such both parties must perform ring operations, which are significantly more expensive than field operations as they must either directly compute or simulate (via NTT) the degree-d polynomial arithmetic. Furthermore, the NTT transformation itself (required for these checks) adds a significant overhead to the prover runtime; as such, lattice-based proof systems attempt to minimize the number of NTT transformations required [12, 69].

Achieving D3 (field-native arithmetic) and D6 (low recursion overheads) requires checking the norm and CCS constraints by **only performing a sum-check over the field**, avoiding all ring operations during the sum-check protocol. Moreover, since our goal is to construct a folding scheme, the embedding must respect an **evaluation homomorphism** so that the resulting field multilinear polynomial evaluations can be folded together.

Problem 2: Lack of packing efficiency & Pay-per-bit cost & SIMD Lattice-based proof systems targeting field constraints (such as CCS) must rely on some embedding of field vectors into ring vectors to be able to commit to these field vectors with Ajtai commitments. Hence, packing efficiency (the density of field elements embedded into ring vectors) is a crucial metric for the efficiency of these proof systems. In particular, the highest packing efficiency possible (information-theoretically) for a ring vector of length n is $d \cdot n$ field elements. Unfortunately, the most algebraically friendly embedding, the NTT transform, has only a packing efficiency of $(d/t) \cdot n$ field elements for a ring vector $\boldsymbol{z}$ of length n. For ideal choices of parameterization [15], t is often a non-trivial factor of d such as $t = 4$ for $d = 64$. Moreover, regardless of the norm of the original field vectors, the NTT embedding results in a ring vector $\boldsymbol{z}$ which has arbitrary norm. Hence, to utilize the Ajtai commitment scheme, the vector $\boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}^{n}$ must be expanded into a larger ring vector $\boldsymbol{z}' \in \mathrm{R}_{\mathbb{F}}^{\log_b(|\mathbb{F}|)\cdot n}$; this further reduces the packing efficiency by a factor of $\log_b(|\mathbb{F}|)$, regardless of the original norm of the embedded vectors. Furthermore, when using the NTT embedding, the cost to commitment to these field elements does not scale with their bit size; Pedersen-style commitments [46, 72, 84] for field vectors have this pay-per-bit cost. Finally, the NTT embedding requires that t separate field vectors be embedded into a single ring vector to reach its peak packing efficiency; however, this inherently requires that the underlying proof system to use a SIMD constraint system (such as CCS over multiple field vectors simultaneously). SIMD constraint systems are not well-suited for applications where the constraints cannot be neatly split into independent, smaller systems. In particular, the ideal constraint system would be over a single field vector of length $d \cdot n$; rather than t independent constraint systems over the t separate field vectors of length n (since the prior can immediately simulate the later, and use a larger constraint system).

Achieving D2 (pay-per-bit) and D4 (general constraints) thus requires an *algebraically friendly* embedding with **optimal packing efficiency** of $d \cdot n$ field elements for a ring vector of length n, with a **pay-per-bit** commitment cost, and without the requirement for a SIMD constraint system.

Problem 3: Complex security proofs lacking modular analysis In the literature of lattice-based proof systems [12, 15, 16, 26, 33, 42, 64], there are often protocols Π which have a particular form and whose security proofs are complex and non-modular. In particular, the prover and verifier take in as input some commitments to witnesses along with some property that needs to be checked on the underlying committed witnesses (such as norm or CCS constraints). We identify that these protocols Π can be broken down into two stages Π_{property} and Π_{special}. The first stage Π_{property} is often some sound testing protocol (such as sum-check or random projections) run on the underlying committed witnesses, which produce additional algebraic claims (such as multilinear evaluations or random inner products) on the witnesses. The second stage Π_{special} is often a special-sound protocol [1, 7, 8, 33, 39] which takes (informally) a random linear combination of the original commitments and of the algebraic claims produced by Π_{property}; this results in a new commitment and a new algebraic claim on the underlying witness. These stages seem to mirror the structure of a reduction of knowledge [51], where proving the composed protocol $\Pi := \Pi_{\mathsf{special}} \circ \Pi_{\mathsf{property}}$ is secure requires proving that both Π_{property} and Π_{special} are knowledge-sound. Unfortunately, this is not the case; in particular, the individual stages Π_{property} and Π_{special} do not meet the strict definition of a reduction of knowledge. As such, the security analysis of these protocols Π is often done in an ad-hoc manner, where the security proof directly analyzes the composed protocol Π as a whole (often leading to complex, non-blackbox security arguments).

A separate challenge is **generalizing the framework of reductions of knowledge** to capture these individual stages Π_{property} and Π_{special}—in particular, identifying what core properties each stage must satisfy—such that **their composition Π is provably secure** (knowledge-sound).

1.2.2 Contributions of our work

Since our work subsumes and extends the prior work Neo, we will discuss both the contributions of the original Neo work and our extension SuperNeo together.

Contribution 1: Neo and SuperNeo Embeddings

Can we check the norm and CCS constraints by only performing a sum-check over the field?

To answer this question, we can ask why the prior NTT-based approach required ring operations in the sum-check protocol in the first place. The main reason is that the NTT transform simply does not preserve enough structure between the underlying field vectors and the committed ring vector to meaningfully check both the norm and CCS constraints by only relying on field constraints (which can be checked solely with a sum-check over the field). Since the NTT transform is not a norm-preserving map, the norm of the underlying field vectors does not directly correspond to the norm of the associated ring vector in which they are embedded. Hence, checking the norm of the ring vector cannot be done by simply applying constraints over the underlying field vectors. More broadly, checking any non-trivial constraints purely over the underlying field vector do not correspond

with constraints over the coefficient vectors of the ring vector itself, and vice versa. Hence, the technique from [19] is required to reduce the norm constraints to ring constraints and the isomorphic property of the NTT map is required to reduce the underlying field CCS constraints to ring constraints. Now that both the norm and CCS constraints are reduced to ring constraints, the sum-check protocol has to be executed over the ring itself. We resolve this issue by introducing the **Neo embedding**, a map which directly embeds field vectors $z^{(1)}, \ldots, z^{(d)} \in \mathbb{F}^n$ along the coefficients slots of the committed ring vector $\boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}^n = (\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n)$.

$$\mathsf{Coeff}(\boldsymbol{z}) = \left[\begin{array}{c} z^{(1)} \\ \hline z^{(2)} \\ \hline \vdots \\ \hline z^{(d)} \end{array}\right] = \left[\begin{array}{c|c|c|c} & & & \\ \boldsymbol{z}_1 & \boldsymbol{z}_2 & \cdots & \boldsymbol{z}_n \\ & & & \end{array}\right]$$

Since the coefficient vectors coincide exactly with the underlying field vectors, the Neo embedding is a norm-preserving map; checking the norm of the ring vector can be done purely with field constraints over the underlying vectors $z^{(1)}, \ldots, z^{(d)} \in \mathbb{F}^n$. Moreover, the field CCS constraints can immediately be checked over the underlying field vectors themselves. As a result, the sum-check protocol can be executed purely over the field, and no ring operations are required. The sum-check protocol reduces these norm and CCS constraints down to checking random multilinear evaluations of the underlying field vectors; for example, $y = \widetilde{Mz}(r)$ for some CCS matrix $M \in \mathbb{F}^{m \times n}$. Explicitly, an evaluation claim has the following form: Consider a committed ring vector $\boldsymbol{z}$ (with commitment $c = A\boldsymbol{z}$), do the underlying field vectors $z^{(1)}, \ldots, z^{(d)}$ evaluate to some claimed values $y^{(1)}, \ldots, y^{(d)}$ at a random extension field point r? Since we are constructing a folding scheme, our goal is to fold these evaluation claims together into a single evaluation claim by taking a random linear combination of the commitments and of the evaluation claims.

Does the Neo embedding respect an ***evaluation homomorphism*** *such that we can fold these evaluation claims together?*

We prove that the Neo embedding does indeed exactly respect the type of evaluation homomorphism required to fold these evaluation claims together. Consider two committed ring vectors $\boldsymbol{z}$ and $\boldsymbol{z}'$ (with commitments $c = A\boldsymbol{z}$ and $c' = A\boldsymbol{z}'$) and their underlying field vectors $z^{(1)}, \ldots, z^{(d)}$ and $z'^{(1)}, \ldots, z'^{(d)}$ with evaluations $y^{(1)}, \ldots, y^{(d)}$ and $y'^{(1)}, \ldots, y'^{(d)}$ at the same random extension field point r. We simply embed the evaluations into ring elements $\boldsymbol{y} := \sum_i y^{(i)} \cdot X^{i-1}$ and $\boldsymbol{y}' := \sum_i y^{(i)} \cdot X^{i-1}$ by placing the evaluations in the same coefficient slots as the underlying field vectors. We show that taking the linear combination $z'' := \boldsymbol{z} + \delta \cdot \boldsymbol{z}'$ ($c'' = c + \delta \cdot c'$) for any $\delta \in \mathrm{R}_{\mathbb{F}}$ results in a ring vector $\boldsymbol{z}''$ (c'') whose underlying field vectors $z''^{(1)}, \ldots, z''^{(d)}$ evaluate to the coefficients of the linear combination $\boldsymbol{y}'' := \boldsymbol{y} + \delta \cdot \boldsymbol{y}'$. If the challenge δ is a ring element with low-norm [3, 28], then the resulting vector $\boldsymbol{z}''$ will also have low norm and the commitment c'' will be a valid commitment to $\boldsymbol{z}''$. It is quite surprising that

this type of embedding would respect any sort of evaluation homomorphism for multilinear evaluations over the underlying field vectors, given that the commitment scheme and linear combination are defined over the ring. For the NTT embedding, this type of evaluation homomorphism is trivial since the NTT is a ring isomorphism, but for the Neo embedding over coefficients, this is not at all clear. While this embedding is quite natural, it is quite non-trivial to show that it also respects an **evaluation homomorphism**, which is required to batch multilinear evaluations of the underlying field vectors together (a requirement for folding schemes based on sum-check). Given two ring vectors and multilinear evaluations of their underlying field vectors, why does taking a random linear combination of these ring vectors over the ring result in a ring vector whose underlying field vectors evaluate to the same random linear combination of the original field evaluations? For the NTT embedding, this is trivial since the NTT is a ring isomorphism, but for the Neo embedding over coefficients, this is not at all clear. In a prior version of this work, we proved that the Neo embedding respected this evaluation homomorphism with the fact that a ring multiplication (and hence the random linear combination) is merely a linear map over the field; in particular, a ring multiplication can be simulated by multiplying the coefficient vector by the rotation matrix $\mathsf{rot}(\delta)$ (or circulant matrix) associated with the ring element δ. Because of this linearity, the ring linear combination must respect the corresponding multilinear evaluations of the coefficients. However, we present a much simpler interpretation and proof of the evaluation homomorphism by identifying the base field $\mathbb{F}$ as merely the constant polynomials in the cyclotomic ring $\mathrm{R}_{\mathbb{F}} := \mathbb{F}[X]/(\phi(X))$ and identifying the ring $\mathrm{R}_{\mathbb{F}}$ itself as base field polynomials in a larger ring $\mathrm{R}_{\mathbb{K}} := \mathbb{K}[X]/(\phi(X))$ consisting of polynomials whose coefficients belong to the extension field $\mathbb{K}$.

$$\begin{array}{ccc} R_{\mathbb{F}} & \subseteq & R_{\mathbb{K}} \\ \cup\mathsf{I} & & \cup\mathsf{I} \\ \mathbb{F} & \subseteq & \mathbb{K} \end{array}$$

Hence, multilinear evaluations of the underlying field vectors and linear combinations of the committed ring vectors can all be expressed as linear maps over the same larger ring $\mathbb{K}$, and the evaluation homomorphism follows almost immediately from this linearity. This embedding and evaluation homomorphism has been used by several subsequent works [5], such as [16, 26, 68], and referred to as a *tensor of rings* approach [26] or *ring switching* [68].

Is there an embedding that has optimal packing efficiency, a pay-per-bit commitment cost, and does not require a SIMD constraint system?

If our goal is to commit to d field vectors of length n (SIMD constraint system), then the Neo embedding has the optimal packing efficiency of $d \cdot n$ field elements for a ring vector of length n. Furthermore, since the embedded field vectors are directly placed in the coefficient slots, then the commitment costs scales with

[5] We provide a more detailed comparison with related work in the following section.

the number of bits (we explain this in more detail in the technical overview). Unfortunately, the Neo embedding requires a SIMD constraint system for both efficiency and the evaluation homomorphism to hold; the same constraint system must be applied to all d underlying field vectors.

Thus, in this work, we also introduce the **SuperNeo embedding**, which is a norm-preserving embedding for a **single** field vector of length $d \cdot n$ into a ring vector of length n (which is $d\times$ shorter!) that satisfies all the desired properties (optimal packing, pay-per-bit, not SIMD) and preserves the required evaluation homomorphism. Given a field vector $z \in \mathbb{F}^{d \cdot n}$, split the vector into n sub-vectors of length d each, i.e., $z = [z_1, z_2, \ldots, z_n]$ where each $z_i := [z_{i,1}, \ldots, z_{i,d}] \in \mathbb{F}^d$. We will embed each sub-vector z_i as the coefficients of a single ring element $\boldsymbol{z}_i := \sum_j z_{i,j} X^{j-1} \in \mathrm{R}_{\mathbb{F}}$. The resulting vector of ring elements $\boldsymbol{z} := [\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n] \in \mathrm{R}_{\mathbb{F}}^n$ is the SuperNeo embedding of z.

$$z = \left[\quad z_1 \quad \middle| \quad z_2 \quad \middle| \quad \cdots \quad \middle| \quad z_n \quad \right]$$

$$\mathsf{Coeff}(\boldsymbol{z}) = \left[\begin{array}{c|c|c|c} z_1 & z_2 & \cdots & z_n \\ & & & \end{array}\right]$$

The SuperNeo embedding is a norm-preserving map since the underlying field vector $z \in \mathbb{F}^{dn}$ is exactly the coefficients of the committed ring vector $\boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}^n$. Hence, checking both the norm and CCS constraints can be done purely with field constraints over the underlying vector $z \in \mathbb{F}^{dn}$. As such, the sum-check protocol can be executed purely over the field, resulting in evaluation claims of the form $y = \widetilde{Mz}(r)$ for some CCS matrix $M \in \mathbb{F}^{m \times dn}$. Now, it is unclear whether the SuperNeo embedding respects the required evaluation homomorphism. Unlike the Neo embedding evaluation claims, we only have single evaluations $y, y' \in \mathbb{K}$ (instead of d); how can we embed these evaluations into ring elements $\boldsymbol{y}, \boldsymbol{y}'$ such that taking a random linear combination of the committed ring vectors $\boldsymbol{z}'' := \boldsymbol{z} + \delta \cdot \boldsymbol{z}' \in \mathrm{R}_{\mathbb{F}}^n$ results in a ring vector whose underlying field vector $z'' \in \mathbb{F}^{dn}$ that evaluates to the embedded evaluation in $\boldsymbol{y}'' := \boldsymbol{y} + \delta \cdot \boldsymbol{y}'$? As we will see, arguing the evaluation homomorphism for the Neo embedding relies on the fact that the underlying field vectors are directly placed in the coefficient slots of the ring vector, but in the SuperNeo embedding we do not have this nice parallel structure (taking a random linear combination over the ring permutes and mixes the coefficients in a non-trivial way).

However, we show that there is indeed a corresponding choice of evaluation embedding of $y \in \mathbb{K}$ into $\boldsymbol{y} \in \mathrm{R}_{\mathbb{K}}$ such that the evaluation homomorphism holds. To do so, we adapt a technique from the lattice literature [12, 36, 64] called the (*Galois, conjugation, or inner product*) *automorphism* trick for cyclotomic rings. In these works, the automorphism trick $\sigma : \mathrm{R}_{\mathbb{F}} \to \mathrm{R}_{\mathbb{F}}$ was used to check the norm of ring vectors by simulating field vector inner products $\langle a, b \rangle$ with ring multiplication $\sigma(\boldsymbol{a}) \cdot \boldsymbol{b}$ (which help with checking random projections [12] or products [64]). We lift this automorphism trick to embed the CCS matrices $M \in \mathbb{F}^{m \times dn}$ into ring

matrices $\boldsymbol{M} \in \mathrm{R}_{\mathbb{K}}^{m \times n}$ such that the evaluation $y = \widetilde{Mz}(r) \in \mathbb{K}$ is the constant term of $\boldsymbol{y} = \widetilde{\boldsymbol{M}\boldsymbol{z}}(r) \in \mathrm{R}_{\mathbb{K}}$. Then, we show that the evaluation homomorphism follows from linearity (similar to the Neo embedding).

Contribution 2: Strong and Weak Interactive Reductions

Is there a natural meta-framework which captures common lattice-protocols Π_{property} *and* Π_{special} *which independently are not secure, but prove their composition* $\Pi := \Pi_{\mathsf{special}} \circ \Pi_{\mathsf{property}}$ *is secure?*

We introduce the notion of *interactive reductions*, which are a generalization of reductions of knowledge introduced in [51]. In particular, an interactive reduction shares the exact same API (structure) as a reduction of knowledge, but is not required to be knowledge sound. In this way, we can view a reduction of knowledge merely as a knowledge-sound interactive reduction. We introduce two types of interactive reductions, which we call *strong* Π_{strong} and *weak* Π_{weak} interactive reductions, and prove a new composition theorem that the composition $\Pi := \Pi_{\mathsf{weak}} \circ \Pi_{\mathsf{strong}}$ of a strong interactive reduction with a weak interactive reduction results in a reduction of knowledge Π. These *strong* and *weak* properties are quite natural and easy to show for many protocols in the lattice-based proof system literature [12, 15, 33, 42, 64], such as sum-check and random projections for the strong property, and special-sound protocols for the weak property. As such, this composition theorem provides a powerful tool to prove the security of composed protocols Π in a simple and modular manner. To our knowledge, this is the first work to show, quite unintuitively, that insecure protocols can be composed together to yield an overall secure protocol. As we will see soon, these properties are not restricted to lattice-based protocols, but are quite general and capture protocols in group-based proof systems [27, 55, 56] and may capture recent work in the hash-based proof systems [6, 21, 22, 83].

On the topic of security proofs, we also extend the analysis of lattice-based folding schemes to more general cyclotomic polynomials (rather than just those of the form $\phi(X) = X^d + 1$ for d a power of two); this enables a much wider choice of fields, as we will explain later. Namely, we are able to support the use of the Goldilocks field [74], a field of order $p = 2^{64} - 2^{32} + 1$ and very efficient arithmetic. Because Goldilocks has high 2-adicity ($2^{32} \mid (p-1)$), it is particularly SNARK-friendly; a SNARK instantiated over this field can succinctly prove knowledge of lattice-based accumulators and thus enable efficient verification for SuperNeo. This mirrors MicroNova's use of HyperKZG to compress its accumulators [84].

1.3 Related works

Lattice-based folding schemes are a recent area of research. We compare with folding schemes [15, 16, 38, 59], a lattice-based SNARK [26], and a lattice-based polynomial commitment scheme [68].

Lattice-based folding schemes. Lova [38] is the only lattice-based folding scheme based solely on the unstructured Short-Integer-Solution (SIS) problem rather

than Module-SIS, making it arguably the safest in terms of assumptions, but at the cost of efficiency (several orders of magnitude slower) and generality (it targets subset-sum constraints rather than R1CS/CCS). LatticeFold+ [16, 17], the followup to LatticeFold [15], introduces an algebraic range-proof technique that encodes lookup tables into ring elements and checks norm constraints via index proofs and sum-checks, enabling significantly higher norm bounds than Hadamard-product-based approaches [19]. Despite claiming small-field support, their protocols and parameterization are ring protocols over a large prime field; the authors acknowledge [17] that their technique for field-native sum-check and the evaluation homomorphism for folding are adapted from Neo. LatticeFold+ re-interprets our technique as a tensor-of-rings approach; we provide a simpler interpretation by identifying the various fields and rings as subrings of $\mathbb{K}[X]/(\phi(X))$. SALSAA [59] is the first lattice-based folding scheme that natively relies on ℓ_2-norm constraints rather than ℓ_∞-norm constraints. SALSAA improves and extends the lattice-based proof system framework from RoK, paper, SISsors [48] and RoK and Roll [49], achieving linear prover runtime (down from quasilinear) via ring sum-check techniques. However, unlike other lattice-based folding schemes, SALSAA relies on a relatively new assumption called the *vanishing Short-Integer-Solution* (vSIS) assumption [32, 45].

Lattice-based SNARKs. Symphony [26] is a lattice-based SNARK that uses high-arity folding to prove repetitive NP claims without heuristically instantiating the random oracle (as is typical of IVC/PCD-based recursion). Symphony also relies on the tensor-of-rings approach, which as noted in LatticeFold+ [17] is a re-interpretation of Neo's techniques. Additionally, Symphony relies on space-efficient sum-check, which imposes asymptotic and concrete overheads compared to the sum-check in Neo and SuperNeo.

Lattice-based polynomial commitments. Hachi [68] is the first lattice-based polynomial commitment scheme supporting extension-field evaluation of field polynomials, adapting Greyhound [69] by replacing random projections with a sum-check over the extension field for norm constraints and providing a technique to reduce extension-field evaluation proofs to ring statements via Greyhound/Labrador [12, 69]. Neo (prior to Hachi) is the first work to check norms of committed ring vectors via a sum-check purely over the extension field; despite the use of circulant matrices in the evaluation-homomorphism proof, the sum-check reduction itself requires no ring operations. SuperNeo (concurrent with Hachi) uses a related but distinct application of the Galois automorphism trick: in Hachi it proves multilinear evaluations for a polynomial commitment, while in SuperNeo it folds CCS matrix evaluations for a folding scheme. Both achieve the same packing efficiency: committing to a vector of length $d \cdot n$ requires only a ring vector of length n.

2 Technical overview

We recall HyperNova and the challenges of moving to lattice-based commitments (§2.1), then present the Neo (§2.2) and SuperNeo (§2.3) embeddings, and finally our interactive reductions framework (§2.4).

2.1 Breaking down HyperNova

HyperNova [55] *reduces* CCS claims into random evaluation claims. These claims are instances in a *CCS relation*, CCS, and a *CCS evaluation relation*, CE.

$$\mathsf{CCS} := \left\{ (\mathsf{s};\ (c,\ x);\ w)\ :\ \begin{array}{l} \text{For } z := [x, w],\ c = \mathsf{Commit}(z) \\ \wedge\ f\left(\widetilde{M_1 z}, \ldots, \widetilde{M_t z}\right)\ \textit{vanishes on } \{0,1\}^{\log m} \end{array} \right\}$$

$$\mathsf{CE} := \left\{ \left(\mathsf{s};\ \left(c,\ x,\ r,\ \{y_j\}_{j\in[t]}\right);\ w\right)\ :\ \begin{array}{l} \text{For } z := [x, w],\ c = \mathsf{Commit}(z) \\ \wedge\ \forall j \in [t],\ y_j = \widetilde{M_j z}(r) \end{array} \right\}$$

To be considered a folding scheme, we must be able to describe HyperNova as an interactive reduction $\Pi : \mathsf{CCS} \times \mathsf{CE} \to \mathsf{CE}$, which takes as input a CCS relation claim and a CCS evaluation relation claim and outputs a single CCS evaluation relation claim.[6] Unlike the original HyperNova paper (for reasons that will become clear later), we decide to break up HyperNova into two interactive reductions which will be composed to form the overall reduction:

$$\text{(Informal)} \quad \Pi_{\mathsf{CCS}} : \mathsf{CCS} \times \mathsf{CE} \to \mathsf{CE}^2 \quad \text{and} \quad \Pi_{\mathsf{RLC}} : \mathsf{CE}^2 \to \mathsf{CE}.$$

The first reduction Π_{CCS} utilizes the sum-check protocol [62, 78] to reduce a CCS claim and a CCS evaluation claim over a point r' into a pair of CCS evaluation claims over the same point r.

$$\left(\mathsf{s};\ (c,\ x);\ w\right) \in \mathsf{CCS}\ \wedge\ \left(\mathsf{s};\ (c',\ x',\ r';\ \{y'_j\}_{j\in[t]});\ w'\right) \in \mathsf{CE}$$

$$\downarrow \Pi_{\mathsf{CCS}}$$

$$\left(\mathsf{s};\ \left(c,\ x,\ \boldsymbol{r},\ \{y_j\}_{j\in[t]}\right);\ w\right) \in \mathsf{CE}\ \wedge\ \left(\mathsf{s};\ \left(c',\ x',\ \boldsymbol{r},\ \{y'_j\}_{j\in[t]}\right);\ w'\right) \in \mathsf{CE}$$

The second reduction Π_{RLC} combines two CCS evaluation claims over the same point r into a single CCS evaluation claim over point r. In particular, the reduction Π_{RLC} is trivial if $\mathsf{Commit} : \mathbb{F}^n \to \mathbb{G}$ is itself a linear map from field vectors to a group $\mathbb{G}$ (i.e. linearly homomorphic). To combine both of these claims, the verifier just samples a random challenge $\delta \in \mathbb{F}$ and checks the following combined claim: $\left(\mathsf{s};\ \left(c^*,\ x^*,\ r,\ \{y^*_j\}_{j\in[t]}\right);\ w^*\right) \in \mathsf{CE}$, where

$$c^* := c + \delta c', \quad x^* := x + \delta x', \quad \forall j \in [t],\ y^*_j := y_j + \delta y'_j, \quad w^* := w + \delta w'.$$

Because Commit is a linear map over field vectors, we have that the new commitment $c^* = \mathsf{Commit}(z^*)$ for $z^* := [x^*, w^*]$. Similarly, for each $j \in [t]$, we have that

[6] In the technical overview, we consider only folding single instances of each relation for simplicity, but our actual protocol works for folding multiple instances at once.

$z \mapsto \widetilde{M_j z}(r)$ is also a linear map over field vectors, so $y_j^* = \widetilde{M_j z^*}(r)$. Thus, the combined claim is indeed a valid CCS evaluation claim as desired.

In order to construct the Neo and SuperNeo folding scheme, we will need to construct a commitment scheme for field vectors (using Ajtai commitments) that has a similar *evaluation homomorphism* property, which allows us to combine CCS evaluation claims by taking random linear combinations over the ring rather than the field. In doing so, we have the essential components to adapt the HyperNova folding scheme to the lattice setting. In addition, we will have to add norm-checks into the sum-check protocol and add a decomposition reduction [12, 15, 71] to reduce norm growth. Later, we also discuss the technical difficulties when trying to use prior extraction techniques to prove the security of our reductions in the lattice setting, which is not apparent in the original HyperNova setting.

2.2 The Neo embedding

The Neo embedding is incredibly simple to describe, and what's surprising is that it also preserves both norm and the required form of evaluation homomorphism. To embed field vectors $z^{(1)}, \ldots, z^{(d)} \in \mathbb{F}^n$ into a ring vector $\boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}$, the Neo embedding simply embeds the field vectors along the d coefficient slots.

$$\boldsymbol{z} = \sum_{j=1}^{d} z^{(j)} \cdot X^{j-1} \quad \iff \quad \forall i \in [n],\ \boldsymbol{z}_i = \sum_{j=1}^{d} z_i^{(j)} \cdot X^{j-1}$$

In the expression above, $z^{(j)} \cdot X^{j-1}$ denotes scaling the field vector $z^{(j)}$ by the monomial X^{j-1} (i.e. every element in the scaled vector has the form $c \cdot X^{j-1}$ for some constant $c \in \mathbb{F}$).

Preserving evaluation homomorphism The first observation is any collection of elements in the field $\mathbb{F}$ can be naturally embedded into the ring $\mathrm{R}_{\mathbb{F}} \subseteq \mathrm{R}_{\mathbb{K}}$ by interpreting them as constant polynomials. The second observation is that $\mathbb{K}$ also contains an (isomorphic) copy of the base field $\mathbb{F}$. Thus, a polynomial with coefficients in the base field $\mathbb{F}$ can also be interpreted as a polynomial with coefficients in the larger field $\mathbb{K}$. Thus, CCS matrices $M_j \in \mathbb{F}^{m \times n}$ and the evaluation point $r \in \mathbb{K}^{\log m}$ can both be trivially embedded into the ring as matrices $M_j \in \mathrm{R}_{\mathbb{K}}^{m \times n}$ and a point $r \in \mathrm{R}_{\mathbb{K}}^{\log m}$. What happens when we take the multilinear extension of $\boldsymbol{z}$ and evaluate it at this point r over the ring? The first observation is multilinear evaluation is equivalent to taking an inner product $\widetilde{\boldsymbol{z}}(r) = \langle \boldsymbol{z}, \hat{r} \rangle$, where $\hat{r} \in \mathbb{K}^n$ is a field vector derived from r. The second observation is that multiplying a ring element $a(X) = \sum_{i=0}^{d-1} a_i X^i \in \mathrm{R}_{\mathbb{F}}$ by a constant polynomial $c \in \mathbb{K}$ simply scales each coefficient by c (i.e. $a(X) \cdot c = \sum_{i=0}^{d-1} (a_i \cdot c) X^i \in \mathrm{R}_{\mathbb{K}}$). Combining these two observations, we can see that evaluating the multilinear extension of $\boldsymbol{z}$ at the point r is equivalent to evaluating each of the underlying field vectors $z^{(1)}, \ldots, z^{(d)}$ at the point r and stacking the results as coefficients in a ring element.

$$\widetilde{\boldsymbol{z}}(r) = \langle \boldsymbol{z}, \hat{r} \rangle = \langle \sum_{j=1}^{d} z^{(j)} \cdot X^{j-1}, \hat{r} \rangle = \sum_{j=1}^{d} \langle z^{(j)}, \hat{r} \rangle \cdot X^{j-1} = \sum_{j=1}^{d} \widetilde{z^{(j)}}(r) \cdot X^{j-1}$$

Moreover, this property extends to matrix-vector multiplications as well. For $r \in \mathbb{F}^{\log m}$, the evaluation $\widetilde{M_j \boldsymbol{z}}(r)$ has coefficients that are exactly the evaluations $\widetilde{M_j z^{(1)}}(r), \ldots, \widetilde{M_j z^{(d)}}(r)$.

We can now see that the evaluation homomorphism trivially holds for the Neo embedding. Consider two ring vectors $\boldsymbol{z}, \boldsymbol{z'} \in \mathrm{R}_{\mathbb{F}}^n$ that are Neo embeddings of field vectors $z^{(1)}, \ldots, z^{(d)} \in \mathbb{F}^n$ and $z'^{(1)}, \ldots, z'^{(d)} \in \mathbb{F}^n$ respectively. Also, consider their evaluations $y = \widetilde{M_j \boldsymbol{z}}(r) \in \mathrm{R}_{\mathbb{K}}$ and $y' = \widetilde{M_j \boldsymbol{z'}}(r) \in \mathrm{R}_{\mathbb{K}}$ at some point $r \in \mathbb{K}^{\log m}$. For an arbitrary ring scalar $\delta \in \mathrm{R}_{\mathbb{F}}$, define $\boldsymbol{z}^* = \boldsymbol{z} + \delta \cdot \boldsymbol{z'} \in \mathrm{R}_{\mathbb{F}}$ ($c^* = c + \delta \cdot c'$) and $y^* = y + \delta \cdot y'$. We must have that the underlying field vectors of $\boldsymbol{z}^*$ evaluate to the coefficients of y^* at the point r. This is because multilinear evaluation $\boldsymbol{z} \mapsto \widetilde{M_j \boldsymbol{z}}(r)$ is linear over the ring so $y^* = \widetilde{M_j \boldsymbol{z}^*}(r)$, and we just showed that the coefficients of $\widetilde{M_j \boldsymbol{z}^*}(r)$ are exactly the evaluations of the underlying field vectors of $\boldsymbol{z}^*$ at the point r. Also, the vector $\boldsymbol{z}^*$ still belongs to the smaller ring $\mathrm{R}_{\mathbb{F}}$ and the underlying field vectors of $\boldsymbol{z}^*$ still belong to the base field $\mathbb{F}$. Thus, as long as the norm of $\boldsymbol{z}^*$ is small enough (which happens if we sample δ from a special-set), c^* is a valid commitment to $\boldsymbol{z}^*$ directly using the Ajtai commitment scheme.

2.3 The SuperNeo embedding

The SuperNeo embedding Given a field vector $z \in \mathbb{F}^{d \cdot n}$, split the vector into n sub-vectors of length d each, i.e., $z = [z_1, z_2, \ldots, z_n]$ where each $z_i := [z_{i,1}, \ldots, z_{i,d}] \in \mathbb{F}^d$. We will embed each sub-vector z_i as the coefficients of a single ring element $\boldsymbol{z}_i := \sum_{j=1}^{d} z_{i,j} X^{j-1} \in \mathrm{R}_{\mathbb{F}}$. The resulting vector $\boldsymbol{z} := [\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n] \in \mathrm{R}_{\mathbb{F}}^n$ is the SuperNeo embedding of z.

Evaluation Homomorphism For cyclotomic rings, there exists a linear transform $\bar{\cdot} : \mathbb{F}^d \to \mathbb{F}^d$ such that for all $a, b \in \mathbb{F}^d$, we have the constant term $\mathsf{ct}(\bar{\boldsymbol{a}} \cdot \boldsymbol{b}) = \langle a, b \rangle$ where we embed $\bar{a} \in \mathbb{F}^d$ into a ring element $\bar{\boldsymbol{a}} \in \mathrm{R}_{\mathbb{F}}$ via coefficients. In layman's terms, a product over the ring simulates an inner product over the field. We can extend this transformation to vectors $m \in \mathbb{F}^{dn}$ by applying $\bar{\cdot}$ to each sub-vector of length d. Now, the constant term of the ring inner product $\mathsf{ct}(\langle \bar{\boldsymbol{m}}, \boldsymbol{z} \rangle) = \langle m, z \rangle$. Finally, we can extend this transformation to matrices $M_j \in \mathbb{F}^{m \times dn}$ by applying $\bar{\cdot}$ to each row of length dn. Thus, the constant term coefficient vector $\mathsf{ct}(\bar{\boldsymbol{M}}_j \boldsymbol{z}) = M_j z$, where we extract the constant terms of each of the m ring elements in $\bar{\boldsymbol{M}}_j \boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}^m$. Given a point $r \in \mathbb{K}^{\log m}$, we can evaluate the ring vector $\boldsymbol{y} = \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r)$. As argued before, evaluating at the point r evaluates each of the underlying coefficient vectors of $\bar{\boldsymbol{M}}_j \boldsymbol{z}$ in parallel. Hence, the constant coefficient of the ring evaluation $\boldsymbol{y}$ is exactly the desired field evaluation $y = \widetilde{M_j z}(r)$.

Moreover, since $z \mapsto \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r)$ is a linear map over $\mathrm{R}_{\mathbb{K}}$ (similar to the case in Neo), the evaluation homomorphism property will identically be preserved. In particular, given field vectors $z, z' \in \mathbb{F}^{dn}$ with embeddings $\boldsymbol{z}, \boldsymbol{z'} \in \mathrm{R}_{\mathbb{F}}^n$ and evaluations $y = \widetilde{M_j z}(r)$ and $y' = \widetilde{M_j z'}(r)$. For an arbitrary scalar $\delta \in \mathrm{R}_{\mathbb{F}}$, let $\boldsymbol{z}^* = \boldsymbol{z} + \delta \boldsymbol{z'}$ and $\boldsymbol{y}^* = \boldsymbol{y} + \delta \boldsymbol{y'}$. Then, we will have that the constant coefficient

of $\boldsymbol{y}^*$ is exactly the field evaluation $\widetilde{M_j z^*}(r)$ for z^* being the underlying field vector of $\boldsymbol{z}^*$.

Pay-per-bit costs Neo and SuperNeo are both norm preserving embeddings. Computing the Ajtai commitment $c \leftarrow A\boldsymbol{z}$ requires a ring matrix-vector product. When both the degree $d \leq 64$ and field size $|\mathbb{F}| \leq 64$ are small, a ring operation $a \cdot b \in \mathrm{R}_{\mathbb{F}}$ is most efficiently implemented (using AVX-512) with a rotation matrix product $\mathsf{cf}(a \cdot b) = \mathsf{rot}(a) \cdot \mathsf{cf}(b) = \sum_{i=1}^{d} b_i \cdot \mathsf{rot}(a)_i$ over the field $\mathbb{F}$. The dominating costs are scaling $b_i \cdot \mathsf{rot}(a)_i$, which scales linearly with the norm of b_i. When the b_i's are small (such as bits), the cost to compute the ring operation is essentially adding the rotations $\mathsf{rot}(a)_i$ for which b_i is non-zero. For security reasons [15], the cyclotomic ring cannot be perfectly splitting (ie. $\mathrm{R} \not\cong \mathbb{F}^d$), so using the NTT transform is a more inefficient method to compute ring operations than simply adding scaled rotations. In particular, the NTT transform (in the non-full splitting setting) is more memory intensive, is less cache friendly, and computing high degree extension field multiplications (ex. 16 $\mathbb{F}_{q^4}$ muls) is concretely more expensive than computing the corresponding base field scaling and additions.

2.4 Proving the security with interactive reductions

To understand why interactive reductions are necessary, we first need to understand why the original proof of security for the HyperNova folding scheme [58] does not carry over to SuperNeo. In particular, when we move to using lattice-based commitments (such as Ajtai commitments), we will have to extract candidate openings z that not only satisfy the linear relationship $c = Az$, but are also sufficiently low norm (as Ajtai commitments are only binding for low norm openings). However, in the process of solving for these candidate openings, the extractor will end up obtaining vectors that potentially may have arbitrary norm. This is often the case with lattice-based proof systems, which often must combine some information-theoretic sound method to check the norm of committed vectors (such as random projections or sum-check) with a method to batch prove knowledge of linear openings (special-sound protocols). Recall (unlike in the original work) that we broke the HyperNova folding scheme down into two reductions: $\Pi_{\mathsf{CCS}} : \mathsf{CCS} \times \mathsf{CE} \to \mathsf{CE}^2$ and $\Pi_{\mathsf{RLC}} : \mathsf{CE}^2 \to \mathsf{CE}$. The first reduction Π_{CCS} reduced the CCS and CCS evaluation claim into two CCS evaluations claims by using the sum-check protocol and the second reduction Π_{RLC} combined the two CCS evaluation claims into a single CCS evaluation claim by taking a random linear combination. The first issue arises when we try to prove Π_{RLC} is knowledge sound in the lattice setting. If we follow the same extraction strategy in HyperNova, we would produce two candidate commitment openings z_1 and z_1' by solving the linear system $z_1^* = z_1 + \delta_1 z_1'$ and $z_2^* = z_1 + \delta_2 z_1'$ for two different random challenges δ_1 and δ_2. In particular, this will require scaling by the inverse of the difference of the two challenges $(\delta_1 - \delta_2)^{-1}$. However, in the ring setting, even if δ_1 and δ_2 are low norm, the inverse $\Delta := (\delta_1 - \delta_2)^{-1}$ may have arbitrarily high norm, which would lead to candidate openings z_1 and z_1' that also have arbitrarily high norm. Thus, we cannot guarantee that the extracted openings are valid openings for the commitments, but that they do indeed satisfy the linear relations $c = Az$ and

$z \mapsto \widetilde{Mz}(r)$. This means that Π_{RLC} is not knowledge sound for the input relation, but instead a relaxed relation where we drop the low norm requirement on the openings. However, there is something special about these candidate openings. In particular, in the process of extracting these openings, we produced what are referred to as **relaxed** openings $\Delta \cdot c = Az$ for the original commitments where z is low norm and Δ belongs to the difference set $\mathcal{C} - \mathcal{C}$. Under the hardness of MSIS, producing multiple relaxed openings for the same commitment is hard. Thus, even if we run the extractor multiple times, it will ultimately only be able to output a single pair of candidate openings (z_1, z_1'); otherwise, it would be able to produce multiple relaxed openings for the same commitment. In summary, while Π_{RLC} by itself is not knowledge sound for the original relation, we can construct an extractor which extracts candidate witnesses for a relaxed relation, and is restricted to only being able to output a unique candidate witness (with high probability). Informally, we call an interactive reduction that satisfies these properties as a **weak interactive reduction**.

The second issue arises when we try to prove that Π_{CCS} is knowledge sound. In a folklore security argument (for sum-check protocols), the extractor for Π_{CCS} essentially rewinds the potentially malicious prover multiple times to obtain multiple candidate openings (the output witnesses) for the same commitment. These candidate openings must also satisfy the evaluations outputted by the sum-check protocol (assuming the prover is successful in both executions). However, since the commitment is binding, all these candidate openings must be the same. Hence, we are able to argue that the candidate opening (witness) from the first execution must have evaluated to the correct evaluations required by the sum-check protocol in the second execution. Observe, however, that the second execution used verifier challenges that were independent of the first execution. Hence, with high probability, the extracted candidate opening must satisfy the required CCS claims. This argument crucially relies on that the fact that the commitment scheme is binding to the candidate openings outputted by the prover. This is trivial in the original HyperNova setting since the commitments are binding to the whole message space. However, in the lattice setting, the commitments are only binding to low norm openings.

To see where this goes wrong, consider that we need to prove that the overall composition $\Pi := \Pi_{\mathsf{RLC}} \circ \Pi_{\mathsf{CCS}}$ is knowledge sound. Π_{RLC} is only a weak interactive reduction. Hence, it can only extract candidate openings for a relaxed relation, where the low norm requirement is dropped, but all linear relations are still satisfied. Thus, these candidate openings may not be bound by the original input commitments. However, in the case of *weak* interactive reductions, we know that the extractor can only output a unique candidate opening (with high probability). Hence, we do not need to rely on the binding property of the commitment scheme to argue that all candidate openings extracted from multiple executions of the protocol must be the same. Therefore, by the same logic above, the candidate openings must satisfy the required CCS claims with high probability. Additionally, if we include sum-check constraints which ensure that the extracted openings are low norm, then we can ensure that the extracted openings are indeed valid

openings for the original commitments. Generally, what do we require from the first interactive reduction Π_{CCS} to ensure that the overall composition is knowledge sound? Essentially, if the malicious prover must always output the same output *relaxed* witness (with high probability), then there exist an extractor which can extract a valid witness for the original relation. We call an interactive reduction that satisfies this property a **strong interactive reduction**.

In our work, we use this framework to prove the security of our SuperNeo folding scheme $\Pi_{\mathsf{SuperNeo}} := \Pi_{\mathsf{DEC}} \circ \Pi_{\mathsf{RLC}} \circ \Pi_{\mathsf{CCS}}$ by decomposing it into a strong interactive reduction Π_{CCS}, a weak interactive reduction Π_{RLC}, and a final reduction of knowledge Π_{DEC} [12, 15, 71].

Lemma 1. *The sequential composition* $\Pi_{\mathsf{RLC}} \circ \Pi_{\mathsf{CCS}}$ *is a* ***reduction of knowledge*** *(Definition 5) from* $\mathsf{CCS}(b, \mathcal{L})^K \times \mathsf{CE}(b, \mathcal{L})^k$ *to* $\mathsf{CE}(B, \mathcal{L})$.

Proof. Follows directly from Π_{CCS} being a strong interactive reduction (Lemma 3), Π_{RLC} being a weak interactive reduction (Lemma 4), and the strong-weak composition theorem (Theorem 6). □

Theorem 1. *The sequential composition* $\Pi_{\mathsf{DEC}} \circ \Pi_{\mathsf{RLC}} \circ \Pi_{\mathsf{CCS}}$ *is a* ***reduction of knowledge*** *(Definition 5) from* $\mathsf{CCS}(b, \mathcal{L})^K \times \mathsf{CE}(b, \mathcal{L})^k$ *to* $\mathsf{CE}(b, \mathcal{L})^k$.

Proof. Follows directly from $\Pi_{\mathsf{RLC}} \circ \Pi_{\mathsf{CCS}}$ being a reduction of knowledge (Lemma 1), Π_{DEC} being a reduction of knowledge (Theorem 7), and sequential composition of reductions of knowledge being reductions of knowledge (Lemma 2). □

3 Overview of the following sections

Since the SuperNeo embedding lends itself to a more efficient and natural description of our folding scheme (since it does not require a SIMD constraint system), the rest of the paper will be focused on the adaption of Hypernova-like interactive reductions with the SuperNeo embedding (of course, with the appropriate lifting and analysis in the lattice setting). We defer the original Neo work to the current eprint [70].

The rest of the paper is organized as follows. In Section 4, we provide the necessary preliminaries for our work, including the syntax and security definitions for interactive reductions. In Section 5, we define the evaluation homomorphism embedding, which is a key technical tool used in our SuperNeo folding scheme. In Section 6, we give the formal definitions of strong and weak interactive reductions, and provide the exact composition theorem needed. In Section 7.1, we define the CCS relation and the CCS evaluation relation, which are the main relations used in our folding scheme. In Section 7.3, we describe the strong interactive reduction Π_{CCS} that reduces the CCS and CCS evaluation claims into CCS evaluation claims. In Section 7.4, we describe the weak interactive reduction Π_{RLC} that combines multiple CCS evaluation claims into a single CCS evaluation claim. In Section 7.5, we describe the final reduction of knowledge Π_{DEC} that reduces the norm of the evaluation claims from $B = b^k$ to b.

4 Preliminaries

For brevity, we defer some additional background to Section C.

Notation We let λ denote the security parameter and $\mathsf{negl}(\lambda)$ denote a negligible function in λ. Throughout the paper, the depicted asymptotics depend on λ, but we elide this for brevity. We let PPT denote probabilistic polynomial time and EPT denote expected probabilistic polynomial time. We let $[n]$ denote the set $\{1, \ldots, n\}$, and $\{u_i\}_{i\in[n]}$ denote the set $\{u_1, \ldots, u_n\}$.

Polynomials We write $\mathbb{F}^d[X_1, \ldots, X_n]$ to denote multivariate polynomials over field $\mathbb{F}$ in the variables $(X_1, \ldots, X_n)$ with degree bound $\leq d$ for each variable. We omit the superscript if there is no degree bound. We define ZS_ℓ as the set of all multivariate polynomials $F \in \mathbb{F}[X_1, \ldots, X_\ell]$ such that for all $x \in \{0,1\}^\ell$, $F(x) = 0$ (i.e. vanish over the Boolean hypercube). We denote the polynomial $\mathsf{eq}(x, y) = \prod_{i=1}^{\ell}(x_i \cdot y_i + (1-x_i)\cdot(1-y_i))$, which outputs 1 if $x = y$ and 0 otherwise for $x, y \in \{0,1\}^\ell$. For vector $v \in \mathbb{F}^n$ we let $\widetilde{v} \in \mathbb{F}^1[X_1, \ldots, X_{\log n}]$ denote the multilinear polynomial extension of v: $\widetilde{v} = \sum_{j\in\{0,1\}^{\log n}} \mathsf{eq}(X_1, \ldots, X_{\log n}, j) \cdot v_j$

Definition 1 (Fields, Rings, and Dimensions).
Fields: *Let $\mathbb{F}$ be a finite field of prime order q and $\mathbb{K}$ to be the lowest degree extension of $\mathbb{F}$ such that $1/|\mathbb{K}| = \mathsf{negl}(\lambda)$. We identify $\mathbb{F}$ as a subfield of $\mathbb{K}$.*

Rings: *Let $\Phi(X) := X^d + \Phi_{d-1}X^{d-1} + \cdots + \Phi_1 X + \Phi_0 \in \mathbb{F}[X]$ be the η-th cyclotomic polynomial with degree d. We define the ring $\mathrm{R}_\mathbb{F} := \mathbb{F}[X]/(\Phi(X))$ and the ring $\mathrm{R}_\mathbb{K} := \mathbb{K}[X]/(\Phi(X))$. We identify $\mathbb{F}$ as a sub-ring of $\mathrm{R}_\mathbb{F}$ and $\mathrm{R}_\mathbb{F}$ as a sub-ring of $\mathrm{R}_\mathbb{K}$.*

Dimensions: *Let m, n_R, $n_\mathbb{F} = d \cdot n_\mathrm{R}$, $n_{\mathbb{F},\mathsf{in}} = d \cdot n_{\mathrm{R},\mathsf{in}} \in \mathbb{N}_{\geq 1}$. We use m to denote the number of constraints, $n_\mathbb{F}$ to denote the length of a field vector, n_R to denote the length of a ring vector, and $n_{\mathrm{R},\mathsf{in}}$ and $n_{\mathbb{F},\mathsf{in}}$ to denote input lengths. Let u, $t \in \mathbb{N}_{\geq 1}$, where u denotes a degree and t denotes a number of matrices. Let k, $K \in \mathbb{N}_{\geq 1}$, where k and K indicate the number of instances.*

Norm bounds: *Let b, $B = b^k < q/2 \in \mathbb{N}_{\geq 2}$ be norm bounds.*

Definition 2 (Coefficient maps). *We denote the **coefficient vector** of an element $a \in \mathrm{R}_\mathbb{F}$ as $\mathsf{cf}(a) \in \mathbb{F}^d$. Given a vector $z \in \mathrm{R}_\mathbb{F}^m$, we denote $\mathsf{cf}(z)$ to be the **coefficient matrix** $[\mathsf{cf}(z_1), \mathsf{cf}(z_2), \ldots, \mathsf{cf}(z_m)] \in \mathbb{F}^{d\times m}$. We define $\mathsf{cf}(z)_\ell$ to be the ℓ-th row of the coefficient matrix $\mathsf{cf}(z)$ (i.e. the ℓ-th coefficient vector of z).*
*We denote the **constant term** of an element $a \in \mathrm{R}_\mathbb{F}$ as $\mathsf{ct}(a) \in \mathbb{F}$. Given a vector $z \in \mathrm{R}_\mathbb{F}^m$, we denote $\mathsf{ct}(z)$ to be the vector $(\mathsf{ct}(z_1), \mathsf{ct}(z_2), \ldots, \mathsf{ct}(z_m)) \in \mathbb{F}^m$.*
We analogously define these maps for elements and vectors in $\mathrm{R}_\mathbb{K}$.

Definition 3 (Norm). *For an element $a \in \mathbb{F}$, we define $\|a\|_\infty$ as follows: Let $a' \in [0,\ q-1]$ denote the integer representation of $a \mod q$. If $a' \leq (q-1)/2$, then $\|a\|_\infty = a'$. Otherwise, if $a' > (q-1)/2$, then $\|a\|_\infty = |a' - q|$. For a vector $z \in \mathbb{F}^n$, we define the ℓ_∞-norm $\|z\|_\infty$ to be the max infinity norm of its elements. For an element $a \in \mathrm{R}_\mathbb{F}$, we define $\|a\|_\infty$ to be the ℓ_∞-norm of the vector $\mathsf{cf}(a)$. For a vector $z \in \mathrm{R}_\mathbb{F}^m$, we define $\|z\|_\infty$ to be the maximum ℓ_∞-norm of its elements.*

Decomposition We define $\mathsf{split}_b : \mathbb{F}^m \to (\mathbb{F}^m)^*$ to be the b-ary decomposition map, which performs the b-ary decomposition of a vector $z \in \mathbb{F}^m$ into vectors $z_1, z_2, \ldots, z_*$. For example, if $z \in \mathbb{F}^m$ such that and $\|z\|_\infty < b^k$, then we have

$$\mathsf{split}_b(z) := (z_1, z_2, \ldots z_k) \quad \text{such that} \quad z = \sum_{i=1}^{k} b^{i-1} \cdot z_i \text{ and } \|z_i\|_\infty < b$$

Definition 4 (Ring Commitment Scheme). *A ring commitment scheme* $\mathsf{com} := (\mathsf{Setup}, \mathsf{Commit})$ *consists of two PPT algorithms:*

- $\mathsf{Setup}(1^\lambda, m) \to \mathsf{pp}$*: Takes as input a security parameter* 1^λ *and length* $m \in \mathbb{N}_{\geq 1}$*, outputs public parameters* pp*.*
- $\mathsf{Commit}(\mathsf{pp}, z) \to c$*: Takes as input public parameters* pp *and a vector* $z \in \mathrm{R}_\mathbb{F}^m$*, outputs a commitment* $c \in \mathbb{C}$*.*

A ring commitment scheme can satisfy the following properties:

B-binding: *For every length* $m = \mathsf{poly}(\lambda)$ *and every EPT adversary* $\mathcal{A}$*, a ring commitment scheme is B-binding (for $B \in \mathbb{N}$) if the following probability holds:*

$$\Pr\left[\begin{array}{l|l} \mathsf{Commit}(\mathsf{pp}, z_1) = \mathsf{Commit}(\mathsf{pp}, z_2) & \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, m) \\ \wedge\ \|z_1\|_\infty,\ \|z_2\|_\infty < B, & z_1, z_2 \in \mathrm{R}_\mathbb{F}^m \leftarrow \mathcal{A}(\mathsf{pp}) \\ \wedge\ z_1 \neq z_2 & \end{array}\right] \leq \epsilon_{\mathsf{bind}}(B)$$

for $\epsilon_{\mathsf{bind}}(B) \leq \mathsf{negl}(\lambda)$*. We refer to a pair of vectors* (z_1, z_2) *which satisfies the conditions in the probability as a* ***B-binding collision****.*

$(B, \mathcal{C})$-relaxed binding: *For every length* $m = \mathsf{poly}(\lambda)$ *and every EPT adversary* $\mathcal{A}$*, a ring commitment scheme is $(B, \mathcal{C})$-relaxed binding (for $B \in \mathbb{N}$ and set $\mathcal{C} \subseteq \mathrm{R}_\mathbb{F}$) if the following probability holds:*

$$\Pr\left[\begin{array}{l|l} \Delta_1 \cdot c = \mathsf{Commit}(\mathsf{pp}, z_1) & \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, m) \\ \wedge\ \Delta_2 \cdot c = \mathsf{Commit}(\mathsf{pp}, z_2) & \left(\begin{array}{c} c \in \mathbb{C}, \\ \Delta_1, \Delta_2 \in (\mathcal{C} - \mathcal{C}), \\ z_1, z_2 \in \mathrm{R}_\mathbb{F}^m \end{array}\right) \leftarrow \mathcal{A}(\mathsf{pp}) \\ \wedge\ \|z_1\|_\infty,\ \|z_2\|_\infty < B, & \\ \wedge\ \Delta_1 z_2 \neq \Delta_2 z_1 & \end{array}\right] \leq \epsilon_{\mathsf{rlx}}(B, \mathcal{C})$$

for $\epsilon_{\mathsf{rlx}}(B, \mathcal{C}) \leq \mathsf{negl}(\lambda)$*. We refer to a tuple of elements* $(c, \Delta_1, \Delta_2, z_1, z_2)$ *which satisfies the conditions in the probability as a* $(B, \mathcal{C})$***-relaxed binding collision****.*

Homomorphic: *For every* $m \in \mathbb{N}$ *and* $\mathsf{pp} \in \mathsf{Setup}(1^\lambda, m)$*, the commitment algorithm,* $\mathsf{Commit}(\mathsf{pp}, \cdot) : \mathrm{R}_\mathbb{F}^m \to \mathbb{C}$*, is an* $\mathrm{R}_\mathbb{F}$*-module homomorphism.*

Theorem 2 (Properties [2, 7, 9, 14]). *The Ajtai commitment scheme (Definition 18) is a ring commitment scheme (Definition 4) that is* ***homomorphic****,* ***B-binding*** *(assuming the* $\mathsf{MSIS}^{\infty,\kappa,q}_{m,2B}$ *problem (Definition 16) is hard), and* ***$(B, \mathcal{C})$-relaxed binding*** *(assuming the* $\mathsf{MSIS}^{\infty,\kappa,q}_{m,4TB}$ *problem is hard and* $\mathcal{C}$ *is a strong sampling set (Definition 17) with expansion factor* T *(Theorem 9)).*

Definition 5 (Interactive Reductions [50, 52]). *Consider relations $\mathcal{R}_1$ and $\mathcal{R}_2$ over parameters, structure, instance, and witness tuples. An* ***interactive reduction*** *from $\mathcal{R}_1$ to $\mathcal{R}_2$ is defined by PPT algorithms $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ called the generator, encoder, prover, and verifier respectively with the following interface.*

- $\mathcal{G}(1^\lambda, \mathsf{sz}) \to \mathsf{pp}$*: Takes as input a security parameter 1^λ and size parameters sz. Outputs public parameters pp.*
- $\mathcal{K}(\mathsf{pp}, \mathsf{s}) \to (\mathsf{pk}, \mathsf{vk})$*: Takes as input public parameters pp and a structure s. Deterministically, outputs a prover key pk and a verifier key vk.*
- $\mathcal{P}(\mathsf{pk}, u_1, w_1) \to (u_2, w_2)$*: Takes as input a proving key pk and an instance-witness pair (u_1, w_1). Interactively reduces the task of checking $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1$ to the task of checking $(\mathsf{pp}, \mathsf{s}, u_2, w_2) \in \mathcal{R}_2$.*
- $\mathcal{V}(\mathsf{vk}, u_1) \to u_2$*: Takes as input a verifier key vk and an instance u_1 in $\mathcal{R}_1$. Interactively reduces the task of checking the instance u_1 to the task of checking a new instance u_2 in $\mathcal{R}_2$.*

Let $\langle \mathcal{P}, \mathcal{V} \rangle$ denote the interaction between $\mathcal{P}$ and $\mathcal{V}$. We treat $\langle \mathcal{P}, \mathcal{V} \rangle$ as a function that takes as input $((\mathsf{pk}, \mathsf{vk}), u_1, w_1)$ and runs the interaction on the prover's input (pk, u_1, w_1) and the verifier's input (vk, u_1). At the end of the interaction, $\langle \mathcal{P}, \mathcal{V} \rangle$ outputs the verifier's instance u_2 and the prover's witness w_2.

A ***reduction of knowledge*** *[51] is an interactive reduction, $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$, that satisfies the following properties:*

(i) ***Completeness****: For any EPT adversary $\mathcal{A}$, given $\mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz})$, $(\mathsf{s}, u_1, w_1) \leftarrow \mathcal{A}(\mathsf{pp})$ such that $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1$, we have that the prover's output instance is equal to the verifier's output instance u_2, and that*

$$(\mathsf{pp}, \mathsf{s}, \langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, w_1)) \in \mathcal{R}_2.$$

(ii) ***Knowledge soundness****: For any EPT adversary $(\mathcal{A}, \mathcal{P}^*)$, there exists an EPT extractor $\mathcal{E}$ such that if the success probability of the adversary*

$$\epsilon(\mathcal{A}, \mathcal{P}^*) := \Pr\left[(\mathsf{pp}, \mathsf{s}, \langle \mathcal{P}^*, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st})) \in \mathcal{R}_2 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \end{array} \right]$$

$\geq 1/\mathsf{poly}(\lambda)$, then we have that

$$\Pr\left[(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \end{array} \right] \geq \epsilon(\mathcal{A}, \mathcal{P}^*) - \mathsf{negl}(\lambda).$$

(iii) ***Public Coin****: All of the verifier's messages are uniformly random strings of some prescribed length. Furthermore, the verifier's messages contain all of the random coins (randomness) used by the verifier.*[7]

[7] If a reduction of knowledge is public-coin, then it trivially satisfies the property of **public reducibility** described in [52] as the execution of the verifier $\mathcal{V}$ can be emulated using the randomness from the transcript.

Lemma 2 (Sequential composition [50, 52]). *For reductions of knowledge* $\Pi_1 = (\mathcal{G}, \mathcal{K}, \mathcal{P}_1, \mathcal{V}_1) : \mathcal{R}_1 \to \mathcal{R}_2$ *and* $\Pi_2 = (\mathcal{G}, \mathcal{K}, \mathcal{P}_2, \mathcal{V}_2) : \mathcal{R}_2 \to \mathcal{R}_3$*, we have that* $\Pi_2 \circ \Pi_1 = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V}) : \mathcal{R}_1 \to \mathcal{R}_3$ *is a reduction of knowledge where* $\mathcal{K}(\mathsf{pp}, \mathsf{s})$ *computes* $(\mathsf{pk}, \mathsf{vk})$ *and where*

$$\mathcal{P}(\mathsf{pk}, u_1, w_1) = \mathcal{P}_2(\mathsf{pk}, \mathcal{P}_1(\mathsf{pk}, u_1, w_1))$$
$$\mathcal{V}(\mathsf{vk}, u_1) = \mathcal{V}_2(\mathsf{vk}, \mathcal{V}_1(\mathsf{vk}, u_1, w_1))$$

Definition 6 (The sum-check protocol [62]). *The sum-check protocol* $\mathsf{SumCheck}(\mathrm{T};\ Q)$ *is a classic interactive proof protocol between two* PPT *algorithms* $(\mathcal{P}, \mathcal{V})$ *that checks that the sum of evaluations of a* ℓ*-variate polynomial* $Q \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\ell]$ *on the Boolean hypercube results in some claimed value* T*. The output of the sum-check protocol is a claim that* $v \stackrel{?}{=} Q(r)$ *for some random point* $r \in \mathbb{F}^\ell$ *and claimed evaluations* v*, which the verifier* $\mathcal{V}$ *can query* Q *to check. The protocol is public-coin, has a completeness error of* 0*, and has a soundness error of* $\leq \ell d / |\mathbb{F}|$*. More generally, the field can be chosen to be an extension field* $\mathbb{K}$*. In this case, the soundness error is* $\leq \ell d / |\mathbb{K}|$*. A self-contained description of the sum-check protocol can be found in this note [81].*

5 Embedding products with evaluation homomorphism

Here, we define a bijective embedding from the field $\mathbb{F}$ into the ring $\mathrm{R}_\mathbb{F}$.

Definition 7 (Coefficient Embedding).
Element embedding: *Consider a vector* $v \in \mathbb{F}^d$*. We define* $\boldsymbol{v} \in \mathrm{R}_\mathbb{F}$ *(in bold font) to be the ring element whose coefficient vector is* v*, i.e.* $\mathsf{cf}(\boldsymbol{v}) = v$*.*

Vector embedding: *Recall that we define* $n_\mathbb{F} = d \cdot n_\mathrm{R}$*. Hence, for a field vector* $z \in \mathbb{F}^{n_\mathbb{F}}$*, we have a natural partition into* d*-sized sub-vectors* $z = [z_1, \ldots, z_{n_\mathrm{R}}]$*. We define the ring vector* $\boldsymbol{z} := (\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{n_\mathrm{R}}) \in \mathrm{R}_\mathbb{F}^{n_\mathrm{R}}$ *to be the vector of ring elements, which are the embeddings of the* $n_\mathrm{R} = n_\mathbb{F}/d$ *field sub-vectors.*

Matrix embedding: *For a matrix* $M \in \mathbb{F}^{m \times n_\mathbb{F}}$ *with rows* $M_1, \ldots, M_m \in \mathbb{F}^{n_\mathbb{F}}$*, we define* $\boldsymbol{M} := [\![\boldsymbol{M}_1, \ldots, \boldsymbol{M}_m]\!] \in \mathrm{R}_\mathbb{F}^{m \times n_\mathrm{R}}$*, which is the vertical concatenation of all the embedded rows.*

Inverse embedding: *Similarly, given a ring vector* $\boldsymbol{v} \in \mathrm{R}_\mathbb{F}^{n_\mathrm{R}}$ *or ring matrix* $\boldsymbol{M} \in \mathrm{R}_\mathbb{F}^{m \times n_\mathrm{R}}$*, we define the field vector* $v \in \mathbb{F}^{n_\mathbb{F}}$ *or field matrix* $M \in \mathbb{F}^{m \times n_\mathbb{F}}$ *as the inverse of previously defined coefficient embeddings.*

Theorem 3 (Inner Product Transform [36, 64]). *There exists a linear transform* $\bar{\cdot} : \mathbb{F}^d \to \mathbb{F}^d$ *such that for all* $a, b \in \mathbb{F}^d$*, we have the constant term*

$$\mathsf{ct}(\bar{\boldsymbol{a}} \cdot \boldsymbol{b}) = \langle a,\, b \rangle$$

where $\bar{\boldsymbol{a}}$ *denotes applying the transform to* a *and embedding* $\bar{a}$ *into the ring.*

Here, we define an extension of the inner product transform $\bar{\cdot} : \mathbb{F}^d \to \mathbb{F}^d$ (Theorem 3) to vectors and matrices.

Definition 8 (Lifting the Transform).
Vector Transform: *Consider a vector $v \in \mathbb{F}^{n_{\mathbb{F}}}$, we can partition v into d-sized sub-vectors $[v_1, \ldots, v_{n_{\mathrm{R}}}]$. We define $\bar{\cdot} : \mathbb{F}^{n_{\mathbb{F}}} \to \mathbb{F}^{n_{\mathbb{F}}}$ to be $\bar{v} := [\bar{v}_1, \ldots, \bar{v}_{n_{\mathrm{R}}}] \in \mathbb{F}^{n_{\mathbb{F}}}$.*

Matrix Transform: *Consider a matrix $M \in \mathbb{F}^{m \times n_{\mathbb{F}}}$ with rows $M_1, \ldots, M_m \in \mathbb{F}^{n_{\mathbb{F}}}$. We define $\bar{\cdot} : \mathbb{F}^{m \times n_{\mathbb{F}}} \to \mathbb{F}^{m \times n_{\mathbb{F}}}$ to be $\bar{M} := \left[\!\left[\bar{M}_1, \ldots, \bar{M}_m\right]\!\right] \in \mathbb{F}^{m \times n_{\mathbb{F}}}$.*

Remark 1 (Efficiency and Sparsity Preservation). When the cyclotomic polynomial $\phi(X)$ is a power-of-two cyclotomic or a trinomial cyclotomic, the transform $\bar{\cdot} : \mathbb{F}^{n_{\mathbb{F}}} \to \mathbb{F}^{n_{\mathbb{F}}}$ essentially only involves permuting and adding entries of the input vector, and hence can be computed in $O(n_{\mathbb{F}})$ time. Since the $\bar{\cdot}$ is linear, if the original matrix M is sparse, then the transformed matrix $\bar{M}$ is also sparse.

Theorem 4 (Matrix-Vector Product Transform). *Consider an arbitrary matrix $M \in \mathbb{F}^{m \times n_{\mathbb{F}}}$ and vector $z \in \mathbb{F}^{n_{\mathbb{F}}}$. The matrix-vector product $Mz \in \mathbb{F}^m$ is equal to the constant terms of the matrix-vector product $\bar{\boldsymbol{M}}\boldsymbol{z} \in \mathrm{R}_{\mathbb{F}}^m$, when viewing each ring element as a polynomial. More succinctly, $Mz = \mathsf{ct}(\bar{\boldsymbol{M}}\boldsymbol{z})$.*

Proof. For brevity, we defer the proof to Section D.1. □

Remark 2 (Matrix-vector Product Evaluation). Consider an arbitrary vector $z \in \mathbb{F}^{n_{\mathbb{F}}}$, matrix $M \in \mathbb{F}^{m \times n_{\mathbb{F}}}$, and multilinear evaluation point $r \in \mathbb{K}^{\log m}$. Define the evaluation $y := \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}_i}(r) = \langle \bar{\boldsymbol{M}}\boldsymbol{z}_i, \hat{r} \rangle \in \mathrm{R}_{\mathbb{K}}$. Observe that multiplying a ring element in $\mathrm{R}_{\mathbb{F}}$ with an extension field element in $\mathbb{K}$ scales each coefficient of the ring element by the extension field element. Hence, by Definition 2, we must have that for all $\ell \in [d]$, $\mathsf{cf}(y)_\ell = \widetilde{\mathsf{cf}\left(\bar{\boldsymbol{M}}\boldsymbol{z}_i\right)_\ell}(r) \in \mathbb{K}$ (i.e. the ℓ-th coefficient of y is equal to the multilinear evaluation of the ℓ-th coefficient vector of $\bar{\boldsymbol{M}}\boldsymbol{z}_i$ at point r). Since $\mathsf{ct}(y) = \mathsf{cf}(y)_1$ and $\mathsf{ct}\left(\bar{\boldsymbol{M}}\boldsymbol{z}_i\right) = \mathsf{cf}\left(\bar{\boldsymbol{M}}\boldsymbol{z}_i\right)_1$, by Theorem 4, we can observe that the constant term $\mathsf{ct}(y) = \widetilde{Mz}(r) \in \mathbb{K}$ is exactly the multilinear evaluation of the field vector Mz at point r.

Theorem 5 (Evaluation Homomorphism). *Consider an arbitrary matrix $M \in \mathbb{F}^{m \times n_{\mathbb{F}}}$, vectors $z_1, \ldots, z_\ell \in \mathbb{F}^{n_{\mathbb{F}}}$, scalars $\rho_1, \ldots, \rho_\ell \in \mathrm{R}_{\mathbb{F}}$, and evaluation point $r \in \mathbb{K}^{\log m}$. Let $\mathcal{L} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathbb{C}$, $\mathcal{L}_{\mathsf{in}} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R,in}}}$ be arbitrary $\mathrm{R}_{\mathbb{F}}$-module homomorphisms. For all $i \in [\ell]$, define*

$$c_i := \mathcal{L}(\boldsymbol{z}_i) \in \mathbb{C} \qquad \boldsymbol{x}_i := \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}_i) \in \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R,in}}} \qquad y_i := \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}_i}(r) \in \mathrm{R}_{\mathbb{K}}$$

Additionally, define

$$c := \sum_{i \in [\ell]} \rho_i c_i \in \mathbb{C}, \qquad \boldsymbol{x} := \sum_{i \in [\ell]} \rho_i \boldsymbol{x}_i \in \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R,in}}},$$
$$\boldsymbol{z} := \sum_{i \in [\ell]} \rho_i \boldsymbol{z}_i \in \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}}, \quad y := \sum_{i \in [\ell]} \rho_i y_i \in \mathrm{R}_{\mathbb{K}}$$

We must have that $c = \mathcal{L}(\boldsymbol{z})$ and $y = \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}}(r)$. Additionally, for all $i \in [\ell]$, $\mathsf{ct}(y_i) = \widetilde{Mz_i}(r)$ and $\mathsf{ct}(y) = \widetilde{Mz}(r)$.

Proof. For brevity, we defer the proof to Section D.2. □

6 Strong and weak interactive reductions

Definition 9 (Weak Interactive Reductions). *Consider relations $\mathcal{R}_1$, $\mathcal{R}'_1$, and $\mathcal{R}_2$ over public parameters, structure, instance, and witness tuples such that $\mathcal{R}_1 \subseteq \mathcal{R}'_1$. Let $\mathcal{U}_1$ be the ambient instance space of $\mathcal{R}_1$.*

*An interactive reduction $\Pi : \mathcal{R}_1 \to \mathcal{R}_2$, defined by PPT algorithms $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ (Definition 5), is **weak** if it is complete, public coin, and there exists a function $\phi : \mathcal{U}_1 \to \mathbb{C}$ (for an arbitrary space $\mathbb{C}$) such that for any EPT adversary $(\mathcal{A}, \mathcal{P}^*)$, there exists an EPT extractor $\mathcal{E}$ such that if the success probability of the adversary $\epsilon(\mathcal{A}, \mathcal{P}^*) \geq 1/\mathsf{poly}(\lambda)$, then*

$$\Pr\left[(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}'_1 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \end{array}\right] \geq \epsilon(\mathcal{A}, \mathcal{P}^*) - \mathsf{negl}(\lambda).$$

and if $\mathcal{A} := (\mathcal{B}, \mathcal{B}')$ such that

$$\Pr\left[\begin{array}{c} u_1, u'_1 \neq \perp \\ \Downarrow \\ \phi(u_1) = \phi(u'_1) \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, \mathsf{st}^*) \leftarrow \mathcal{B}(\mathsf{pp}) \\ (u_1, \mathsf{st}) \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ (u'_1, \mathsf{st}') \leftarrow \mathcal{B}'(\mathsf{st}^*) \end{array}\right] = 1,$$

then

$$\Pr\left[\begin{array}{c} \\ w_1, w'_1 \neq \perp \\ \wedge\ w_1 \neq w'_1 \\ \\ \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, \mathsf{st}^*) \leftarrow \mathcal{B}(\mathsf{pp}) \\ (u_1, \mathsf{st}) \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \\ (u'_1, \mathsf{st}') \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ w'_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u'_1, \mathsf{st}') \end{array}\right] \leq \mathsf{negl}(\lambda)$$

Definition 10 (Strong Interactive Reductions). *Consider relations $\mathcal{R}_1$, $\mathcal{R}_2$, and $\mathcal{R}'_2$ over public parameters, structure, instance, and witness tuples such that $\mathcal{R}_2 \subseteq \mathcal{R}'_2$. Let $\mathcal{U}_2$ be the ambient instance space of $\mathcal{R}_2$.*

*An interactive reduction $\Pi : \mathcal{R}_1 \to \mathcal{R}_2$, defined by PPT algorithms $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ (Definition 5), is **strong** if it is complete, public coin, and there exists a function $\phi : \mathcal{U}_2 \to \mathbb{C}$ (for an arbitrary space $\mathbb{C}$) such that*

(i) For any EPT adversary $(\mathcal{A}, \mathcal{P}^*)$,

$$\Pr\left[\begin{array}{c} u_2, u'_2 \neq \perp \\ \Downarrow \\ \phi(u_2) = \phi(u'_2) \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda) \\ (\mathsf{s}, u_1, \mathsf{st}_1) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ (u_2, w_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle ((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \\ (u'_2, w'_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle ((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \end{array}\right] = 1$$

(ii) For any EPT adversary $(\mathcal{A}, \mathcal{P}^*)$, there exists an EPT extractor $\mathcal{E}$ such that if

$$\epsilon'(\mathcal{A}, \mathcal{P}^*) := \Pr\left[(\mathsf{pp}, \mathsf{s}, \langle \mathcal{P}^*, \mathcal{V} \rangle ((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st})) \in \mathcal{R}'_2 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \end{array}\right]$$

$\geq 1/\mathsf{poly}(\lambda)$, *and*

$$\Pr\left[\begin{array}{c} w_2, w_2' \neq \bot \\ \wedge \\ w_2 \neq w_2' \end{array} \middle| \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ (u_2, w_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \\ (u_2', w_2') \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \end{array}\right] \leq \mathsf{negl}(\lambda)$$

then we have that

$$\Pr\left[(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1 \middle| \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \end{array}\right] \geq \epsilon'(\mathcal{A}, \mathcal{P}^*) - \mathsf{negl}(\lambda).$$

Theorem 6 (Strong-Weak Composition). *Consider relations $\mathcal{R}_1$, $\mathcal{R}_2$, $\mathcal{R}_2'$ and $\mathcal{R}_3$ over public parameters, structure, instance, and witness tuples such that $\mathcal{R}_2 \subseteq \mathcal{R}_2'$. Let $\mathcal{U}_2$ be the ambient instance space of $\mathcal{R}_2$. Consider interactive reductions (Definition 5) $\Pi_1 : \mathcal{R}_1 \to \mathcal{R}_2$ $(\mathcal{R}_2')$, $\Pi_2 : \mathcal{R}_2$ $(\mathcal{R}_2') \to \mathcal{R}_3$ such that*

*(i) Π_1 is **strong** (Definition 10) with respect to a function $\phi : \mathcal{U}_2 \to \mathbb{C}$ and*
*(ii) Π_2 is **weak** (Definition 9) with respect to the **same** function ϕ,*

*then the sequential composition $\Pi_2 \circ \Pi_1 : \mathcal{R}_1 \to \mathcal{R}_3$ is a **reduction of knowledge**.*

Proof. For brevity, we defer the proof to Section D.3. □

7 Neo's folding scheme for CCS

7.1 Relations

Definition 11 (Structure). *We define a **structure** as a collection of elements*

$$\mathsf{s} := \left\{ \left\{ M_j \in \mathbb{F}^{m \times n_{\mathbb{F}}} \right\}_{j \in [t]},\ f \in \mathbb{F}^{<u}[X_1, \ldots, X_t] \right\},$$

which consists of matrices and a degree-u polynomial.

Definition 12 (Norm-bounded CCS). *Let $\mathcal{L} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathbb{C}$ be an arbitrary $\mathrm{R}_{\mathbb{F}}$-module homomorphism. Let s be a structured as defined in Definition 11. We define the **norm-bounded CCS relation**, $\mathsf{CCS}(b, \mathcal{L})$, as follows:*

$$\left\{ \left(\mathsf{s};\ (c \in \mathbb{C},\ x \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}});\ w \in \mathbb{F}^{n_{\mathbb{F}} - n_{\mathbb{F},\mathsf{in}}} \right) \ :\ \begin{array}{l} \text{For } z := [x, w], \\ c = \mathcal{L}(\boldsymbol{z}) \ \wedge\ \|z\|_\infty < b \ \wedge \\ f\left(\widetilde{M_1 z}, \ldots, \widetilde{M_t z}\right) \in \mathsf{ZS}_{\log m} \end{array} \right\}$$

Definition 13 (Norm-bounded CCS Evaluation Relation). *Let s be a structure as defined in Definition 11. Let $\mathcal{L} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathbb{C}$ be an arbitrary $\mathrm{R}_{\mathbb{F}}$-module homomorphism. Define $\mathcal{L}_{\mathsf{in}} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R},\mathsf{in}}}$ to be the trivial $\mathrm{R}_{\mathbb{F}}$-module*

homomorphism that projects the first $n_{\mathrm{R,in}}$ indices. We define the ***norm-bounded CCS evaluation relation****,* $\mathsf{CE}(b, \mathcal{L})$*, as follows:*

$$\left\{ \left(\mathsf{s};\ \begin{pmatrix} c \in \mathbb{C}, \\ x \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}}, \\ r \in \mathbb{K}^{\log m}, \\ \{y_j \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]} \end{pmatrix};\ z \in \mathbb{F}^n \right) \ :\ \begin{array}{l} c = \mathcal{L}(\boldsymbol{z}) \ \wedge\ \boldsymbol{x} = \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}) \\ \wedge\ \|z\|_\infty < b \ \wedge \\ \forall\, j \in [t],\ y_j = \widetilde{\bar{M}_j \boldsymbol{z}}(r) \end{array} \right\}$$

7.2 A folding scheme for CCS via interactive reductions

Definition 14 (Global Reduction Parameters).
Here, we define the global parameters used in our reductions:

- *Define* $\mathbb{F}, \mathbb{K}, d, \mathrm{R}_{\mathbb{F}}, \mathrm{R}_{\mathbb{K}}, m, n_{\mathbb{F}}, n_{\mathrm{R}}, n_{\mathbb{F},\mathsf{in}}, n_{\mathrm{R},\mathsf{in}}, u, t, k, K, b, B$ *as in Definition 1.*
- *Let* $\mathcal{C} \subseteq \mathrm{R}_{\mathbb{F}}$ *be a strong sampling set (Definition 17) with expansion factor* T *such that* $(K + k)T(b - 1) < B$ *and* $1/|\mathcal{C}| = \mathsf{negl}(\lambda)$*.*
- *Let* $\mathsf{com} := (\mathsf{Setup}, \mathsf{Commit})$ *be a ring commitment scheme (Definition 4), which is homomorphic and* $(2B, \mathcal{C})$*-relaxed binding. For* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, m)$*, define* $\mathcal{L} := \mathsf{Commit}(\mathsf{pp}, \cdot) : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathbb{C}$*, which is a* $\mathrm{R}_{\mathbb{F}}$*-module homomorphism.*
- *Let* $\mathcal{L}_{\mathsf{in}} : \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R}}} \to \mathrm{R}_{\mathbb{F}}^{n_{\mathrm{R},\mathsf{in}}}$ *be the trivial* $\mathrm{R}_{\mathbb{F}}$*-module homomorphism that projects the first* $n_{\mathrm{R},\mathsf{in}}$ *columns.*
- *Let* s *denote a structure as defined in Definition 11.*

In Section B, we instantiate these parameters with concrete values.

7.3 Interactive reduction for CCS – Π_{CCS}

Overview. The reduction of knowledge Π_{CCS} checks that the K incoming CCS instances (Definition 12) indeed satisfy the required CCS constraints, the k *evaluation* claims (Definition 13), from the prior folding step, hold for point r, and checks that the norms of all of the witness vectors (all $K + k$ of them) involved are less than b. To do so, Π_{CCS} relies on the classic sum-check protocol (Definition 6). The approach is inspired by similar reductions from [14, 55]. Π_{CCS} defines helper polynomials that, when used in the sum-check protocol, will perform the previously specified checks. $\mathsf{F}(\vec{X})$ encodes the CCS constraints (all K of them). $\mathsf{NC}(\vec{X})$ encodes the norm constraints (all $K + k$ of them). $\mathsf{Eval}(\vec{X})$ encodes the evaluation claims (all k of them) from the prior step. Finally, $Q(\vec{X})$ is defined such that if its sum over the boolean hypercube $\{0, 1\}^{\log(m)}$ equals to the constructed sum T, then all the respective checks hold.

CCS reduction Π_{CCS}

Parameters: Refer to Definition 14. Without loss of generality, assume that $m = n_{\mathbb{F}}$ and $n_{\mathbb{F}}$ is a power of two and that $M_1 = \mathrm{I}_{n_{\mathbb{F}}}$ is the identity matrix.

Input $\in \mathsf{CCS}(b, \mathcal{L})^K \times \mathsf{CE}(b, \mathcal{L})^k$

$\big(\mathsf{s};\quad (c_i \in \mathbb{C},\, x_i \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}});\quad w_i \in \mathbb{F}^{n_{\mathbb{F}}-n_{\mathbb{F},\mathsf{in}}}\big)_{i=1}^{K}$,

$\Big(\mathsf{s};\quad c_i \in \mathbb{C},\, x_i \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\, r \in \mathbb{K}^{\log m},\, \{y_{i,j} \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\quad z_i \in \mathbb{F}^{n_{\mathbb{F}}}\Big)_{i=K+1}^{K+k}$

Output $\in \mathsf{CE}(b,\mathcal{L})^{K+k}$

$\big(\mathsf{s};\quad c_i \in \mathbb{C},\, x_i \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\, r' \in \mathbb{K}^{\log m},\, \{y'_{i,j} \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\quad z_i \in \mathbb{F}^{n_{\mathbb{F}}}\big)_{i\in[K+k]}$

Setup $\mathcal{G}(1^\lambda, n_{\mathrm{R}}) \to \mathsf{pp}$: Output $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n_{\mathrm{R}})$.

Encoder $\mathcal{K}(\mathsf{pp},\mathsf{s}) \to (\mathsf{pk},\mathsf{vk})$: Output $\big((\mathsf{pp},\mathsf{s}),\perp\big)$.

Reduction $\langle\mathcal{P},\mathcal{V}\rangle((\mathsf{pk},\mathsf{vk}),u_1,w_1) \to (u_2;w_2)$:

1. $\mathcal{V}$: Send challenges $\alpha \xleftarrow{\$} \mathbb{K}^{\log m}$ and $\gamma \xleftarrow{\$} \mathbb{K}$ to $\mathcal{P}$.
2. $\mathcal{V} \leftrightarrow \mathcal{P}$: For all $i \in [K]$, define $z_i := [x_i,\, w_i]$. Define $\vec{X} := (X_1, \ldots, X_{\log m})$,

$$\mathsf{F}(\vec{X}) := \sum\nolimits_{i=1}^{K} \gamma^{i-1} \cdot f\Big(\widetilde{M_1 z_i},\, \ldots,\, \widetilde{M_t z_i}\Big) \in \mathbb{K}[\vec{X}]$$

$$\mathsf{NC}(\vec{X}) := \sum\nolimits_{i=1}^{K+k} \gamma^{i-1} \cdot \prod\nolimits_{j=-b-1}^{b-1} \Big(\widetilde{z_i} - j\Big) \in \mathbb{K}[\vec{X}]$$

$$\mathsf{Eval}(\vec{X}) := \mathsf{eq}\big(\vec{X},\, r\big) \cdot \sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} \gamma^{\mathrm{I}(i,j,\ell)} \cdot \widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell} \in \mathbb{K}[\vec{X}]$$

   where $\mathrm{I}(i,j,\ell) = \big(i\text{-}(K+1)\big) + k(j\text{-}1) + kt(\ell\text{-}1)$ and $\widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell}$ is the multilinear extension of the ℓ-th coefficient vector of $\bar{\boldsymbol{M}}_j \boldsymbol{z}_i$ (Definition 2).
   Define

$$Q(\vec{X}) := \mathsf{eq}(\vec{X},\, \alpha) \cdot \Big(\mathsf{F}(\vec{X}) + \gamma^K \cdot \mathsf{NC}(\vec{X})\Big) + \gamma^{2K+k} \cdot \mathsf{Eval}(\vec{X}) \in \mathbb{K}[\vec{X}]$$

   Define claimed sum of Q over $\{0,1\}^{\log m}$ as

$$\mathrm{T} := \sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} \gamma^{\mathrm{I}(i,j,\ell)} \cdot \mathsf{cf}(y_{i,j})_\ell \in \mathbb{K}$$

   Perform $\mathsf{SumCheck}\,(\mathrm{T};\; Q)$ (Definition 6) which reduces the claim that

$$\mathrm{T} = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x})$$

   to a new evaluation claim $v \stackrel{?}{=} Q(r')$ for new evaluation point $r' \in \mathbb{K}^{\log m}$.
3. $\mathcal{P}$: Send $\forall\, i \in [K+k],\, \forall\, j \in [t],\; y'_{i,j} \leftarrow \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r') \in \mathrm{R}_{\mathbb{K}}$.
4. $\mathcal{V}$: Derive the claimed intermediate evaluations (Remark 2),

$$\mathrm{F} := \sum\nolimits_{i=1}^{K} \gamma^{i-1} \cdot f\big(\mathsf{ct}(y'_{i,1}),\, \ldots,\, \mathsf{ct}(y'_{i,t})\big) \in \mathbb{K}$$

$$\mathrm{N} := \sum\nolimits_{i=1}^{K+k} \gamma^{i-1} \cdot \prod\nolimits_{j=-b-1}^{b-1} \big(\mathsf{ct}(y'_{i,1}) - j\big) \in \mathbb{K}$$

$$\mathrm{E} := \mathsf{eq}(r',\, r) \sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} \gamma^{\mathrm{I}(i,j,\ell)} \cdot \mathsf{cf}\big(y'_{i,j}\big)_{\ell} \in \mathbb{K}$$

Check the evaluation claim $v \stackrel{?}{=} Q\big(r'\big)$ as follows,

$$v \stackrel{?}{=} \mathsf{eq}(r',\, \alpha) \cdot \Big(\mathrm{F} + \gamma^{K} \cdot \mathrm{N}\Big) + \gamma^{2K+k} \cdot \mathrm{E}$$

5. Output $\big(\mathsf{s};\ c_i,\ x_i,\ r',\ \{y'_{i,j}\}_{j\in[t]};\ z_i\big)_{i\in[K+k]}$

Remark 3. By choosing $M_1 = \mathrm{I}_{n_{\mathbb{F}}}$, we simplify our notation, because folding $\widetilde{M_1 z} = \widetilde{\mathrm{I}_{n_{\mathbb{F}}} z}$ evaluations is equivalent to folding $\widetilde{z}$ evaluations.

Lemma 3 (Π_{CCS} is strong). *The interactive reduction $\Pi_{\mathsf{CCS}} : \mathsf{CCS}(b,\mathcal{L})^K \times \mathsf{CE}(b,\mathcal{L})^k \to \mathsf{CE}(b,\mathcal{L})^{K+k}$ ($\mathsf{CE}(q/2,\mathcal{L})^{K+k}$) is **strong** (Definition 10) for the function ϕ, which projects commitments $(c_i)_{i\in[K+k]}$ from the instance.*

Proof. For brevity, we defer the proof to Section D.4. □

7.4 Random linear combination reduction – Π_{RLC}

The interactive reduction Π_{RLC} does exactly as the name suggests. Given $K+k$ input CCS evaluation claims of norm b, it outputs a single CCS evaluation claim of larger norm B, which is a random linear combination of the input claims using challenges from a strong sampling set $\mathcal{C}$ (Definition 17).

Random linear combination reduction Π_{RLC}

Parameters: Refer to Definition 14.

Input $\in \mathsf{CE}(b,\mathcal{L})^{K+k}$

$\big(\mathsf{s};\ c_i \in \mathbb{C},\, x_i \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\, r \in \mathbb{K}^{\log m},\, \{y_{i,j} \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\ z_i \in \mathbb{F}^{n_{\mathbb{F}}}\big)_{i\in[K+k]}$

Output $\in \mathsf{CE}(B,\mathcal{L})$

$\big(\mathsf{s};\ c \in \mathbb{C},\, x \in \mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\, r \in \mathbb{K}^{\log m},\, \{y_j \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\ z \in \mathbb{F}^{n_{\mathbb{F}}}\big)$

Setup $\mathcal{G}(1^\lambda, n_{\mathrm{R}}) \to \mathsf{pp}$: Output $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n_{\mathrm{R}})$.

Encoder $\mathcal{K}(\mathsf{pp},\mathsf{s}) \to (\mathsf{pk},\mathsf{vk})$: Output $\big((\mathsf{pp},\mathsf{s}),\perp\big)$.

Reduction $\langle \mathcal{P},\mathcal{V}\rangle((\mathsf{pk},\mathsf{vk}),u_1,w_1) \to (u_2;w_2)$:

1. $\mathcal{V}$: Sample $\rho_1,\ldots,\rho_{K+k} \xleftarrow{\$} \mathcal{C}$ and compute:

$$c \leftarrow \sum_{i\in[K+k]} \rho_i c_i, \quad \boldsymbol{x} \leftarrow \sum_{i\in[K+k]} \rho_i \boldsymbol{x}_i, \quad \forall\, j \in [t],\ y_j \leftarrow \sum_{i\in[K+k]} \rho_i y_{i,j}$$

   Send $\rho_1,\ldots,\rho_\ell$ to $\mathcal{P}$.

2. $\mathcal{P}$: Compute $\boldsymbol{z} \leftarrow \sum_{i\in[K+k]} \rho_i \boldsymbol{z}_i$.

3. Output $\big(\mathsf{s};\ c,\ x,\ r,\ \{y_j\}_{j\in[t]};\ z\big)$.

Lemma 4 (Π_{RLC} is weak). *The interactive reduction* $\Pi_{\mathsf{RLC}} : \mathsf{CE}(b,\mathcal{L})^{K+k} \big(\mathsf{CE}(q/2,\mathcal{L})^{K+k}\big) \to \mathsf{CE}(B,\mathcal{L})$ *is* ***weak*** *(Definition 9) for the function ϕ, which projects commitments $(c_i)_{i\in[K+k]}$ from the instance.*

Proof. For brevity, we defer the proof to Section D.5. □

7.5 Decomposition reduction – Π_{DEC}

Inspired by folklore techniques [12, 15, 71], our final reduction aims to reduce the norm of claims from $B = b^k$ to b, which will allow us to continually fold CCS claims without increasing the norm of the openings $(z_i)_i$ to the commitments.

Decomposition reduction Π_{DEC}

Parameters: Refer to Definition 14.

Input $\in \mathsf{CE}(B,\mathcal{L})$

$\big(\mathsf{s};\quad c\in\mathbb{C},\ x\in\mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\ r\in\mathbb{K}^{\log m},\ \{y_j\in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\quad z\in\mathbb{F}^{n_{\mathbb{F}}}\big)$

Output $\in \mathsf{CE}(b,\mathcal{L})^k$

$\big(\mathsf{s};\quad c_i\in\mathbb{C},\ x_i\in\mathbb{F}^{n_{\mathbb{F},\mathsf{in}}},\ r\in\mathbb{K}^{\log m},\ \{y_{i,j}\in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]};\quad z_i\in\mathbb{F}^{n_{\mathbb{F}}}\big)_{i\in[k]}$

Setup $\mathcal{G}(1^\lambda, n_{\mathrm{R}}) \to \mathsf{pp}$: Output $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n_{\mathrm{R}})$.

Encoder $\mathcal{K}(\mathsf{pp},\mathsf{s}) \to (\mathsf{pk},\mathsf{vk})$: Output $\big((\mathsf{pp},\mathsf{s}),\perp\big)$.

Reduction $\langle\mathcal{P},\mathcal{V}\rangle((\mathsf{pk},\mathsf{vk}),u_1,w_1) \to (u_2;w_2)$:

1. $\mathcal{P}$: Compute $\big(c_i,\ \{y_{i,j}\}_{j\in[t]};\ z_i\big)_{i\in[k]}$ as follows,

$$(z_1,\ldots,z_k) \leftarrow \mathsf{split}_b(z),\quad c_i \leftarrow \mathcal{L}(\boldsymbol{z}_i),\quad \forall\, j\in[t],\ y_{i,j} \leftarrow \widetilde{\boldsymbol{M}_j \boldsymbol{z}_i}(r)$$

   Send $\big(c_i,\ \{y_{i,j}\}_{j\in[t]}\big)_{i\in[k]}$ to $\mathcal{V}$.

2. $\mathcal{V}$: Compute $(x_1,\ldots,x_k) \leftarrow \mathsf{split}_b(x)$. Check the following equations,

$$c \stackrel{?}{=} \sum\nolimits_{i\in[k]} b^{\,i-1}\cdot c_i \quad\text{and}\quad \forall\, j\in[t],\ y_j \stackrel{?}{=} \sum\nolimits_{i\in[k]} b^{\,i-1}\cdot y_{i,j}$$

   where the norm-bound b is treated as a field element.

3. Output $\big(\mathsf{s};\ c_i,\ x_i,\ r,\ \{y_{i,j}\}_{j\in[t]};\ z_i\big)_{i\in[k]}$

Theorem 7. $\Pi_{\mathsf{DEC}} : \mathsf{CE}(B,\mathcal{L}) \to \mathsf{CE}(b,\mathcal{L})^k$ *is a* ***reduction of knowledge*** *(Definition 5).*

Proof. For brevity, we defer the proof to Section D.6. □

References

[1] Aardal, M.A., Aranha, D.F., Boudgoust, K., Kolby, S., Takahashi, A.: Aggregating falcon signatures with LaBRADOR. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part I. Lecture Notes in Computer Science, vol. 14920, pp. 71–106. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68376-3_3

[2] Ajtai, M.: Generating hard instances of lattice problems (extended abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press, Philadephia, PA, USA (May 22–24, 1996). https://doi.org/10.1145/237814.237838

[3] Albrecht, M.R., Lai, R.W.F.: Subtractive sets over cyclotomic rings - limits of Schnorr-like arguments over lattices. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology – CRYPTO 2021, Part II. Lecture Notes in Computer Science, vol. 12826, pp. 519–548. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_18

[4] Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. Journal of Mathematical Cryptology **9**(3), 169–203 (2015)

[5] Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - a new hope. Cryptology ePrint Archive, Report 2015/1092 (2015), https://eprint.iacr.org/2015/1092

[6] Arnon, G., Chiesa, A., Fenzi, G., Yogev, E.: WHIR: Reed–solomon proximity testing with super-fast verification. Cryptology ePrint Archive, Report 2024/1586 (2024), https://eprint.iacr.org/2024/1586

[7] Attema, T., Cramer, R., Kohl, L.: A compressed Σ-protocol theory for lattices. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology – CRYPTO 2021, Part II. Lecture Notes in Computer Science, vol. 12826, pp. 549–579. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_19

[8] Attema, T., Klooß, M., Lai, R.W.F., Yatsyna, P.: Adaptive special soundness: Improved knowledge extraction by adaptive useful challenge sampling. Cryptology ePrint Archive, Report 2024/2038 (2024), https://eprint.iacr.org/2024/2038

[9] Attema, T., Lyubashevsky, V., Seiler, G.: Practical product proofs for lattice commitments. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology – CRYPTO 2020, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 470–499. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). https://doi.org/10.1007/978-3-030-56880-1_17

[10] Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable zero knowledge with no trusted setup. In: CRYPTO (2019)

[11] Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. In: CRYPTO (2014)

[12] Beullens, W., Seiler, G.: LaBRADOR: Compact proofs for R1CS from module-SIS. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryp-

tology – CRYPTO 2023, Part V. Lecture Notes in Computer Science, vol. 14085, pp. 518–548. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38554-4_17

[13] Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: Recursive composition and bootstrapping for SNARKs and proof-carrying data. In: STOC (2013)

[14] Boneh, D., Chen, B.: LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. Cryptology ePrint Archive, Paper 2024/257 (2024)

[15] Boneh, D., Chen, B.: LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. In: Hanaoka, G., Yang, B.Y. (eds.) Advances in Cryptology – ASIACRYPT 2025, Part III. Lecture Notes in Computer Science, vol. 16247, pp. 330–362. Springer, Singapore, Singapore, Melbourne, VIC, Australia (Dec 8–12, 2025). https://doi.org/10.1007/978-981-95-5099-9_11

[16] Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology – CRYPTO 2025, Part VII. Lecture Notes in Computer Science, vol. 16006, pp. 327–361. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). https://doi.org/10.1007/978-3-032-01907-3_11

[17] Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. Cryptology ePrint Archive, Report 2025/247 (2025), https://eprint.iacr.org/2025/247

[18] Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. In: Advances in Cryptology—ASIACRYPT 2001 (2001)

[19] Bootle, J., Lyubashevsky, V., Seiler, G.: Algebraic techniques for short(er) exact lattice-based zero-knowledge proofs. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology – CRYPTO 2019, Part I. Lecture Notes in Computer Science, vol. 11692, pp. 176–202. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). https://doi.org/10.1007/978-3-030-26948-7_7

[20] Bünz, B., Chen, B.: Protostar: Generic efficient accumulation/folding for special sound protocols. Cryptology ePrint Archive, Paper 2023/620 (2023)

[21] Bünz, B., Chiesa, A., Fenzi, G., Wang, W.: Linear-time accumulation schemes. In: Applebaum, B., Lin, H.R. (eds.) TCC 2025: 23rd Theory of Cryptography Conference, Part I. Lecture Notes in Computer Science, vol. 16268, pp. 369–399. Springer, Cham, Switzerland, Aarhus, Denmark (Dec 1–5, 2025). https://doi.org/10.1007/978-3-032-12287-2_13

[22] Bunz, B., Fenzi, G., Rothblum, R., Wang, W.: Tensorswitch: Nearly optimal polynomial commitments from tensor codes. Cryptology ePrint Archive, Paper 2025/2065 (2025), https://eprint.iacr.org/2025/2065, https://eprint.iacr.org/2025/2065

[23] Bünz, B., Fisch, B., Szepieniec, A.: Transparent SNARKs from DARK compilers. In: EUROCRYPT (2020)

[24] Bunz, B., Mishra, P., Nguyen, W., Wang, W.: Arc: Accumulation for reed–solomon codes. Cryptology ePrint Archive, Paper 2024/1731 (2024)

[25] Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Accumulation without homomorphism. Cryptology ePrint Archive, Paper 2024/474 (2024)
[26] Chen, B.: Symphony: Scalable SNARKs in the random oracle model from lattice-based high-arity folding. Cryptology ePrint Archive, Report 2025/1905 (2025), https://eprint.iacr.org/2025/1905
[27] Chen, B., Bünz, B., Boneh, D., Zhang, Z.: Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In: EUROCRYPT (2023)
[28] Chen, S., Cheon, J.H., Kim, D., Park, D.: Verifiable computing for approximate computation. Cryptology ePrint Archive, Report 2019/762 (2019), https://eprint.iacr.org/2019/762
[29] Chen, W., Chiesa, A., Dauterman, E., Ward, N.P.: Reducing participation costs via incremental verification for ledger systems. Cryptology ePrint Archive, Report 2020/1522 (2020)
[30] Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: EUROCRYPT (2020)
[31] Chiesa, A., Ojha, D., Spooner, N.: Fractal: Post-quantum and transparent recursive proofs from holography. In: EUROCRYPT (2020)
[32] Cini, V., Lai, R.W.F., Malavolta, G.: Lattice-based succinct arguments from vanishing polynomials - (extended abstract). In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part II. Lecture Notes in Computer Science, vol. 14082, pp. 72–105. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38545-2_3
[33] Cini, V., Malavolta, G., Nguyen, N.K., Wee, H.: Polynomial commitments from lattices: Post-quantum security, fast verification and transparent setup. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part X. Lecture Notes in Computer Science, vol. 14929, pp. 207–242. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68403-6_7
[34] Coratger, T., Setty, S.: Post Quantum Signature Aggregation: a Folding Approach. https://ethresear.ch/t/post-quantum-signature-aggregation-a-folding-approach/23639 (2025)
[35] Eagen, L., Gabizon, A.: Protogalaxy: Efficient protostar-style folding of multiple instances. Cryptology ePrint Archive, Paper 2023/1106 (2023)
[36] Esgin, M.F., Nguyen, N.K., Seiler, G.: Practical exact proofs from lattices: New techniques to exploit fully-splitting rings. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2020, Part II. Lecture Notes in Computer Science, vol. 12492, pp. 259–288. Springer, Cham, Switzerland, Daejeon, South Korea (Dec 7–11, 2020). https://doi.org/10.1007/978-3-030-64834-3_9
[37] Fenzi, G., Knabenhans, C., Nguyen, N.K., Pham, D.T.: Lova: Lattice-based folding scheme from unstructured lattices. Cryptology ePrint Archive, Paper 2024/1964 (2024)
[38] Fenzi, G., Knabenhans, C., Nguyen, N.K., Pham, D.T.: Lova: Lattice-based folding scheme from unstructured lattices. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024, Part IV. Lecture Notes in

Computer Science, vol. 15487, pp. 303–326. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). https://doi.org/10.1007/978-981-96-0894-2_10

[39] Fenzi, G., Moghaddas, H., Nguyen, N.K.: Lattice-based polynomial commitments: Towards asymptotic and concrete efficiency. Journal of Cryptology **37**(3), 31 (Jul 2024). https://doi.org/10.1007/s00145-024-09511-8

[40] Flynn, M.J.: Very high-speed computing systems. Proceedings of the IEEE **54**(12), 1901–1909 (2005)

[41] Flynn, M.J.: Some computer organizations and their effectiveness. IEEE transactions on computers **100**(9), 948–960 (2009)

[42] Gentry, C., Halevi, S., Lyubashevsky, V.: Practical non-interactive publicly verifiable secret sharing with thousands of parties. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022, Part I. Lecture Notes in Computer Science, vol. 13275, pp. 458–487. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). https://doi.org/10.1007/978-3-031-06944-4_16

[43] Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. Cryptology ePrint Archive, Paper 2019/458 (2019)

[44] Hülsing, A., Butin, D., Gazdag, S.L., Rijneveld, J., Mohaisen, A.: XMSS: eXtended Merkle signature scheme. RFC 8391 (2018)

[45] Jyrkinen, K., Lai, R.W.F.: Vanishing short integer solution, revisited - reductions, trapdoors, homomorphic signatures for low-degree polynomials. In: Jager, T., Pan, J. (eds.) PKC 2025: 28th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 15675, pp. 273–300. Springer, Cham, Switzerland, Røros, Norway (May 12–15, 2025). https://doi.org/10.1007/978-3-031-91823-0_9

[46] Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: ASIACRYPT. pp. 177–194 (2010)

[47] Kilian, J.: A note on efficient zero-knowledge proofs and arguments (extended abstract). In: STOC (1992)

[48] Klooß, M., Lai, R.W.F., Nguyen, N.K., Osadnik, M.: RoK, paper, SISsors toolkit for lattice-based succinct arguments - (extended abstract). In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024, Part V. Lecture Notes in Computer Science, vol. 15488, pp. 203–235. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). https://doi.org/10.1007/978-981-96-0935-2_7

[49] Klooß, M., Lai, R.W.F., Nguyen, N.K., Osadnik, M.: RoK and roll - verifier-efficient random projection for $\tilde{O}(\lambda)$-size lattice arguments - (extended abstract). In: Hanaoka, G., Yang, B.Y. (eds.) Advances in Cryptology – ASIACRYPT 2025, Part III. Lecture Notes in Computer Science, vol. 16247, pp. 297–329. Springer, Singapore, Singapore, Melbourne, VIC, Australia (Dec 8–12, 2025). https://doi.org/10.1007/978-981-95-5099-9_10

[50] Kothapalli, A.: A Theory of Composition for Proofs of Knowledge. Ph.D. thesis, Carnegie Mellon University (2024)

[51] Kothapalli, A., Parno, B.: Algebraic reductions of knowledge. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part IV. Lecture Notes in Computer Science, vol. 14084, pp. 669–701. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38551-3_21

[52] Kothapalli, A., Parno, B.: Algebraic reductions of knowledge. In: CRYPTO (2023)

[53] Kothapalli, A., Setty, S.: SuperNova: Proving universal machine executions without universal circuits. Cryptology ePrint Archive (2022)

[54] Kothapalli, A., Setty, S.: Cyclefold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. Cryptology ePrint Archive, Paper 2023/1192 (2023), https://eprint.iacr.org/2023/1192, https://eprint.iacr.org/2023/1192

[55] Kothapalli, A., Setty, S.: HyperNova: Recursive arguments for customizable constraint systems. In: CRYPTO (2024)

[56] Kothapalli, A., Setty, S.: NeutronNova: Folding everything that reduces to zero-check. Cryptology ePrint Archive (2024)

[57] Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In: CRYPTO (2022)

[58] Kothapalli, A., Setty, S.T.V.: HyperNova: Recursive arguments for customizable constraint systems. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part X. Lecture Notes in Computer Science, vol. 14929, pp. 345–379. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68403-6_11

[59] Kuriyama, S., Lai, R., Osadnik, M., Tucci, L.: Salsaa - sumcheck-aided lattice-based succinct arguments and applications. Cryptology ePrint Archive, Paper 2025/2124 (2025), https://eprint.iacr.org/2025/2124, https://eprint.iacr.org/2025/2124

[60] Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. Designs, Codes and Cryptography **75**(3), 565–599 (2015). https://doi.org/10.1007/s10623-014-9938-4

[61] Longa, P., Naehrig, M.: Speeding up the number theoretic transform for faster ideal lattice-based cryptography. In: Foresti, S., Persiano, G. (eds.) CANS 16: 15th International Conference on Cryptology and Network Security. Lecture Notes in Computer Science, vol. 10052, pp. 124–139. Springer, Cham, Switzerland, Milan, Italy (Nov 14–16, 2016). https://doi.org/10.1007/978-3-319-48965-0_8

[62] Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. In: FOCS (Oct 1990)

[63] Lyubashevsky, V., Micciancio, D.: Generalized compact Knapsacks are collision resistant. In: Bugliesi, M., Preneel, B., Sassone, V., Wegener, I. (eds.) ICALP 2006: 33rd International Colloquium on Automata, Languages and Programming, Part II. Lecture Notes in Computer Science, vol. 4052, pp. 144–155. Springer Berlin Heidelberg, Germany, Venice, Italy (Jul 10–14, 2006). https://doi.org/10.1007/11787006_13

[64] Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 71–101. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_3

[65] Lyubashevsky, V., Seiler, G.: Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part I. Lecture Notes in Computer Science, vol. 10820, pp. 204–224. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). https://doi.org/10.1007/978-3-319-78381-9_8

[66] Micali, S.: CS proofs. In: FOCS (1994)

[67] Nethermind Research: Lattice-based operations performance report. https://nethermind.notion.site/Latticefold-and-lattice-based-operations-performance-report-153360fc38d080ac930cdeeffed69559 (2025)

[68] Nguyen, N.K., O'Rourk, G., Zhang, J.: Hachi: Efficient lattice-based multilinear polynomial commitments over extension fields. Cryptology ePrint Archive, Paper 2026/156 (2026), https://eprint.iacr.org/2026/156, https://eprint.iacr.org/2026/156

[69] Nguyen, N.K., Seiler, G.: Greyhound: Fast polynomial commitments from lattices. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part X. Lecture Notes in Computer Science, vol. 14929, pp. 243–275. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68403-6_8

[70] Nguyen, W., Setty, S.: Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Report 2025/294 (2025), https://eprint.iacr.org/2025/294

[71] Papamanthou, C., Shi, E., Tamassia, R., Yi, K.: Streaming authenticated data structures. In: Johansson, T., Nguyen, P.Q. (eds.) Advances in Cryptology – EUROCRYPT 2013. Lecture Notes in Computer Science, vol. 7881, pp. 353–370. Springer Berlin Heidelberg, Germany, Athens, Greece (May 26–30, 2013). https://doi.org/10.1007/978-3-642-38348-9_22

[72] Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) Advances in Cryptology – CRYPTO'91. Lecture Notes in Computer Science, vol. 576, pp. 129–140. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 11–15, 1992). https://doi.org/10.1007/3-540-46766-1_9

[73] Peikert, C., Rosen, A.: Efficient collision-resistant hashing from worst-case assumptions on cyclic lattices. In: Halevi, S., Rabin, T. (eds.) TCC 2006: 3rd Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 3876, pp. 145–166. Springer Berlin Heidelberg, Germany, New York, NY, USA (Mar 4–7, 2006). https://doi.org/10.1007/11681878_8

[74] Polygon Zero Team: Plonky2: Fast recursive arguments with PLONK and FRI (2022), https://docs.rs/crate/plonky2/latest/source/plonky2.pdf, https://docs.rs/crate/plonky2/latest/source/plonky2.pdf
[75] Regev, O.: Lattice-based cryptography. In: Annual International Cryptology Conference. pp. 131–141. Springer (2006)
[76] Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. J. ACM **27**(4) (1980)
[77] Seiler, G.: Faster avx2 optimized ntt multiplication for ring-lwe lattice cryptography. Cryptology ePrint Archive (2018)
[78] Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: CRYPTO (2020)
[79] Setty, S., Thaler, J., Wahby, R.: Customizable constraint systems for succinct arguments. Cryptology ePrint Archive (2023)
[80] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. vol. 26, pp. 1484–1509 (1997)
[81] Thaler, J.: The sum-check protocol. https://people.cs.georgetown.edu/jthaler/sumcheck.pdf (Sep 2017)
[82] Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: TCC. pp. 552–576 (2008)
[83] Zeilberger, H., Chen, B., Fisch, B.: BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024, Part X. Lecture Notes in Computer Science, vol. 14929, pp. 138–169. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68403-6_5
[84] Zhao, J., Setty, S.T.V., Cui, W., Zaverucha, G.: MicroNova: Folding-based arguments with efficient (on-chain) verification. In: Blanton, M., Enck, W., Nita-Rotaru, C. (eds.) 2025 IEEE Symposium on Security and Privacy. pp. 1964–1982. IEEE Computer Society Press, San Francisco, CA, USA (May 12–15, 2025). https://doi.org/10.1109/SP61157.2025.00168
[85] Zhou, Z., Zhang, Z., Dong, J.: Proof-carrying data from multi-folding schemes. Cryptology ePrint Archive, Paper 2023/1282 (2023)

Supplementary Material

A AI Disclaimer

Portions of this manuscript were edited with the assistance of an AI writing tool (Github copilot), which was used to improve grammar, wording, and formatting consistency. All technical content-including definitions, theorems, and proofs-was produced and verified by the authors, who take full responsibility for the correctness and originality of the work.

B Concrete parameters

This section provides three efficient parameterizations over ≤ 64-bit fields. Additionally, Section D.7 and Section D.8 provide the corresponding sage scripts that we used to determine valid parameterizations. In Definition 14, we require the commitment scheme to be $(d, m, 2B, \mathcal{C})$-relaxed binding (Definition 4). Thus, we need the commitment scheme to be $(d, m, 4TB)$-binding (Definition 4). Finally, Ajtai's commitment scheme is $(d, m, 4TB)$-binding if $\mathsf{MSIS}^{\infty,\kappa,q}_{m,8TB}$ is hard. We estimate the hardness of Module-SIS using the lattice estimator library provided by [4] using our script (Section D.8).

B.1 Almost Goldilocks: $(2^{64} - 2^{32} + 1) - 32$

We provide a new field, which we refer to as *Almost Goldilocks*. This field's order is $q = (2^{64} - 2^{32} + 1) - 32$, which is close to the order of the Goldilocks field $2^{64} - 2^{32} + 1$. Because of this, the field admits an efficient implementation with a small change to the Solinas prime reduction algorithm (which is typically used for the Goldilocks field).

$\eta = 128$, $\Phi = X^{64} + 1$, $d = 64$, $\mathrm{R}_{\mathbb{F}} := \mathbb{F}[X]/(\Phi)$, $\kappa = 15$, $n_{\mathbb{F}} = 2^{33}$, $b = 2$, $k = 13$, $K \in [50]$, $B = 2^{13}$. Define $\mathcal{C}$ to be the set polynomials in $\mathrm{R}_{\mathbb{F}}$ whose coefficients belong to $[-1, 0, 1, 2]$. By Theorem 9, $T = 128$. By Theorem 8, $\mathsf{b}_{\mathsf{inv}} \approx 4$. $\mathbb{K} = \mathbb{F}_{q^2}$.

$|\mathcal{C}| = 2^{128}$, $|\mathbb{K}| \approx 2^{128}$, $\mathsf{MSIS}^{\infty,\kappa,q}_{m,8TB} \approx 129$ bits of security.

B.2 Goldilocks: $(2^{64} - 2^{32} + 1)$

This is a popular choice of field for SNARKs as the field admits an efficient implementation: field operations can be implemented with essentially only bit-shifts and the field has high 2-adicity ($2^{32} \mid (p-1)$), which is useful for compressing Neo's IVC proofs with SNARKs.

$\eta = 81$, $\Phi = X^{54} + X^{27} + 1$, $d = 54$, $\mathrm{R}_{\mathbb{F}} := \mathbb{F}[X]/(\Phi)$, $\kappa = 18$, $n_{\mathbb{F}} = 2^{30}$, $b = 2$, $k = 14$, $K \in [61]$, $B = 2^{14}$. Define $\mathcal{C}$ to be the set polynomials in $\mathrm{R}_{\mathbb{F}}$ whose coefficients belong to $[-2, -1, 0, 1, 2]$. By Theorem 9, $T = 216$. By Theorem 8, $\mathsf{b}_{\mathsf{inv}} \approx 2.5 \cdot 10^9$. $\mathbb{K} = \mathbb{F}_{q^2}$.

$|\mathcal{C}| \approx 2^{125}$, $|\mathbb{K}| \approx 2^{128}$, $\mathsf{MSIS}^{\infty,\kappa,q}_{m,8TB} \approx 129$ bits of security.

Remark 4 (Incompatibility with Latticefold [14]). In LatticeFold [14], the constructions and analysis are limited to power-of-two cyclotomic polynomials, namely of the form $X^d + 1$ with d being a power-of-two. Since the Goldilocks field has high 2-adicity, the cyclotomic polynomial completely factors into linear terms. This means that the ring $\mathrm{R}_{\mathbb{F}}$ is isomorphic to $\mathbb{F}_q^d$ (the NTT representation). The security of LatticeFold's construction depends on the size of the field in the NTT representation [14, Sec 3.3], which here is only 64 bits.

B.3 Mersenne 61: $2^{61} - 1$

This field admits an incredibly efficient implementation as it is only one off from a power-of-two. Specifically, modular arithmetic over this field can be implemented with simple bit-shifts with an algorithm more efficient than Goldilocks.

$\eta = 81$, $\Phi = X^{54} + X^{27} + 1$, $d = 54$, $\mathrm{R}_{\mathbb{F}} := \mathbb{F}[X]/(\Phi)$, $\kappa = 18$, $n_{\mathbb{F}} = 2^{28}$, $b = 2$, $k = 14$, $K \in [61]$, $B = 2^{14}$. Define $\mathcal{C}$ to be the set polynomials in $\mathrm{R}_{\mathbb{F}}$ whose coefficients belong to $[-2, -1, 0, 1, 2]$. By Theorem 9, $T = 216$. By Theorem 8, $\mathsf{b}_{\mathsf{inv}} \approx 383$. $\mathbb{K} = \mathbb{F}_{q^2}$.

$|\mathcal{C}| \approx 2^{125}$, $|\mathbb{K}| \approx 2^{122}$, $\mathsf{MSIS}^{\infty,\kappa,q}_{m,8TB} \approx 129$ bits of security.

Remark 5 (Incompatibility with Latticefold [14]). As stated earlier, LatticeFold's constructions and analysis are limited to power-of-two cyclotomic polynomials, namely of the form X^d+1 for d being a power-of-two. For Mersenne 61, there is no choice of power-of-two cyclotomic polynomials, which satisfies the requirements of Theorem 8. Hence, it cannot be determined whether a choice of parameters with $\Phi = X^d + 1$ leads to a secure construction.

C Additional Background

Relation Products For relations $\mathcal{R}_1$ and $\mathcal{R}_2$ over public parameter, structure, instance, and witness pairs we define the relation $\mathcal{R}_1 \times \mathcal{R}_2$ such that $(\mathsf{pp}, \mathsf{s}, (u_1, u_2), (w_1, w_2)) \in \mathcal{R}_1 \times \mathcal{R}_2$ if and only if $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1$, and $(\mathsf{pp}, \mathsf{s}, u_2, w_2) \in \mathcal{R}_2$. We let $\mathcal{R}^n$ denote $\mathcal{R} \times \ldots \times \mathcal{R}$ for n times.

Lemma 5 (Schwartz-Zippel [76]). *let $g : \mathbb{F}^\ell \to \mathbb{F}$ be an ℓ-variate polynomial of total degree at most d. Then, on any finite set $S \subseteq \mathbb{F}$,*

$$\Pr_{x \leftarrow S^\ell}[g(x) = 0] \leq d/|S|.$$

Lemma 6. *Let $Q \in \mathbb{F}[X_1, \ldots, X_\ell]$ be an arbitrary multivariate polynomial. Define multivariate polynomial $Q'(\vec{X}, \vec{Z}) := \mathsf{eq}(\vec{X}, \vec{Z}) \cdot Q(\vec{X})$.*

$$0 = \sum_{\vec{x} \in \{0,1\}^{\log \ell}} Q'(\vec{x}, \vec{Z}) \quad \textit{if and only if} \quad Q(\vec{X}) \in \mathsf{ZS}_\ell$$

Definition 15 (Module Homomorphism). *Modules are a generalization of vector spaces for which the field of scalars is replaced by a ring* R*. Suppose* R *is a commutative ring with identity* 1 *and* G *is an abelian (commutative) group. The group* G *is an* R*-module if there is an operation* $\cdot : \mathrm{R} \times \mathrm{G} \to \mathrm{G}$ *such that for all* $r, s \in \mathrm{R}$ *and* $x, y \in \mathrm{G}$, $r \cdot (x+y) = r \cdot x + r \cdot y,\ (r+s) \cdot x = r \cdot x + s \cdot x,\ (rs) \cdot x = r \cdot (s \cdot x),\ 1 \cdot x = x$*. Suppose* G_1 *and* G_2 *are* R*-modules. Similarly, an* R*-module homomorphism is a map* $\mathcal{L} : \mathrm{G}_1 \to \mathrm{G}_2$ *that is a generalization of a linear map of vector spaces.* $\mathcal{L}$ *is an* R*-module homomorphism if for all* $x, y \in \mathrm{G}_1$ *and* $r \in \mathrm{R}$*,* $\mathcal{L}(x+y) = \mathcal{L}(x) + \mathcal{L}(y)$ *and* $\mathcal{L}(r \cdot x) = r \cdot \mathcal{L}(x)$*.*

Definition 16 (Module short integer solution [60, 63, 73]). *Define the ring* $\mathrm{R}_{\mathbb{Z}} := \mathbb{Z}[X]/(\Phi(X))$*. The* $\mathsf{MSIS}^{\infty,\kappa,q}_{m,B}$ *problem is defined as follows: Given a matrix* $M \xleftarrow{\$} \mathrm{R}_{\mathbb{F}}^{\kappa \times m}$ *sampled uniformly at random, find a non-zero vector* $z \in \mathrm{R}_{\mathbb{Z}}$ *such that* $Mz = 0 \mod q$ *and* $\|z\|_\infty < B$*.*

Theorem 8 (Low norm invertibility [65, Theorem 1.1, Conjecture 2.6]). *Let* $z \in \mathbb{N}$ *such that* $z \mid \eta$*,* $q \equiv 1 \pmod z$*, and* $\mathrm{ord}_\eta(q) = \eta/z$*. Define* $\mathsf{b_{inv}} := 1/\sqrt{\tau(z)} \cdot q^{1/\phi(z)}$ *where* $\tau(z) := z$ *if* z *is odd, otherwise* $\tau(z) = z/2$*. For an arbitrary* $a \in \mathrm{R}_{\mathbb{F}}$*, if* $0 < \|a\|_\infty < \mathsf{b_{inv}}$*, then* a *is invertible in* $\mathrm{R}_{\mathbb{F}}$*.*

Definition 17 (Strong sampling sets [3, 28]). *Define* $\mathcal{C} \subseteq \mathrm{R}_{\mathbb{F}}$ *to be any set of ring elements such that for any distinct elements* $a, b \in \mathcal{C}$*,* $\|a - b\|_\infty < \mathsf{b_{inv}}$ *(Theorem 8). Furthermore, we define the*

$$\textbf{\textit{expansion factor}} \text{ of } \mathcal{C} \ := \ \max_{\substack{v \in \mathrm{R}_{\mathbb{F}} \\ \rho \in \mathcal{C}}} \frac{\|\rho v\|_\infty}{\|v\|_\infty}$$

Theorem 9 (Expansion factors [3]). *Let* $\mathcal{C}$ *be a strong sampling set over the cyclotomic ring* $\mathrm{R}_{\mathbb{F}}$ *(Definition 17), We denote the Euler totient function as* ϕ*. We must have that the expansion factor of* $\mathcal{C}$ *is* $\leq 2 \cdot \phi(\eta) \cdot \max_{\rho \in \mathcal{C}} \|\rho\|_\infty$*.*

Definition 18 (Ajtai commitment scheme [2]). *Let message length* $m \in \mathbb{N}$*. The Ajtai commitment scheme* $\mathsf{com} := (\mathsf{Setup}, \mathsf{Commit})$ *consists of the following PPT algorithms:*

- $\mathsf{Setup}(\kappa, m) \to \mathsf{pp}$*: Sample a random matrix* $M \xleftarrow{\$} \mathrm{R}_{\mathbb{F}}^{\kappa \times m}$*. Output* $\mathsf{pp} \leftarrow M$*.*
- $\mathsf{Commit}(\mathsf{pp}, z) \to c$*: Given parameters* pp *and vector* $z \in \mathrm{R}_{\mathbb{F}}^m$*, output* Mz*.*

In this work, we are primarily interested in building folding schemes, a particular type of reduction of knowledge that reduces the task of checking instances in some relation $\mathcal{R}_2$ into a running instance in a relation $\mathcal{R}_1$.

Definition 19 (Folding scheme). *A folding scheme for a relation* $\mathcal{R}$ *is a reduction of knowledge of type* $\mathcal{R} \times \mathcal{R}_{\mathsf{ACC}} \to \mathcal{R}_{\mathsf{ACC}}$ *for some relation* $\mathcal{R}_{\mathsf{ACC}}$*.*

Definition 20 (Special sets [39]). *Let* $\mathcal{C}$ *be a set and* $\ell \in \mathbb{N}$*. Consider two vectors* $x, y \in \mathcal{C}^\ell$*. We define the relation* $\equiv_i$ *for* $i \in [\ell]$ *as follows:*

$$x \equiv_i y \iff x_i \neq y_i \ \wedge \ x_j = y_j \text{ for all } j \in [\ell] \setminus \{i\}.$$

A special set $\mathsf{SS}(\mathcal{C}, \ell)$ *is as follows:*

$$\mathsf{SS}(\mathcal{C}, \ell) = \left\{ (\vec{c},\, \vec{c}_1,\, \ldots,\, \vec{c}_\ell) \in (\mathcal{C}^\ell)^{\ell+1} \;\; : \;\; \begin{array}{c} \forall\, i \in [\ell], \\ \vec{c} \equiv_i \vec{c}_i \end{array} \right\},$$

Theorem 10 (Coordinate-wise extraction [39, Lemma 7.1]). *Let* $\mathcal{C}$ *be a finite set,* $\ell \in \mathbb{N}$*, and* $\vec{\mathcal{C}} := \mathcal{C}^\ell$ *be a challenge space. Let* $A : \vec{\mathcal{C}} \to \{0,1\}^*$ *be an arbitrary (probabilistic) expected polynomial-time algorithm (adversary), and* $V : \vec{\mathcal{C}} \times \{0,1\}^* \to \{0,1\}$ *be an arbitrary (probabilistic) polynomial-time function (verification). Define the success probability of adversary* A *as*

$$\epsilon^V(A) := \Pr_{\vec{c} \xleftarrow{\$} \vec{\mathcal{C}}}[V(\vec{c},\, A(\vec{c})) = 1]$$

Then, there exists an expected polynomial-time oracle algorithm E_A *(extractor) that makes at most* $\ell + 1$ *queries to* A *in expectation and with probability at least* $\epsilon^V(A) - \frac{\ell}{|\mathcal{C}|}$ *outputs* $\ell + 1$ *pairs* $(\vec{c}, w), (\vec{c}_1, w_1), \ldots (\vec{c}_\ell, w_\ell)$ *such that*

- $V(\vec{c}, w) = 1$,
- *for all* $i \in [\ell]$, $V(\vec{c}_i, w_i) = 1$,
- *and* $(\vec{c}, \vec{c}_1, \ldots, \vec{c}_\ell) \in \mathsf{SS}(\mathcal{C}, \ell)$.

D Deferred theorems and proofs

D.1 Proof of Matrix-Vector Product Transformation (Theorem 4)

Proof. Let $M_1, \ldots, M_m \in \mathbb{F}^{n_\mathbb{F}}$ be the rows of M. Define $z_1, \ldots, z_{n_\mathrm{R}} \in \mathbb{F}^d$ and $M_{i,1}, \ldots, M_{i,n_\mathrm{R}} \in \mathbb{F}^d$ (for all $i \in [m]$) to be the partition of vector z and row M_i into d-sized sub-vectors, respectively. We must have that for all $i \in [m]$,

$$\mathsf{ct}\big(\langle \bar{\boldsymbol{M}}_i,\, \boldsymbol{z} \rangle\big) = \sum_{j \in [n_\mathrm{R}]} \mathsf{ct}\big(\bar{\boldsymbol{M}}_{i,j} \cdot \boldsymbol{z}_j\big) = \sum_{j \in [n_\mathrm{R}]} \langle M_{i,j},\, z_j \rangle = \langle M_i,\, z \rangle$$

The first equality is true because, by definition of inner product, $\langle \bar{\boldsymbol{M}}_i,\, \boldsymbol{z} \rangle = \sum_{j \in [n_\mathrm{R}]} \bar{\boldsymbol{M}}_{i,j} \cdot \boldsymbol{z}_j$ and the constant term of a sum of polynomials is equal to the sum of the constant terms of the polynomials. The second equality follows from Theorem 3. The third inequality follows from $(M_{i,j})_j$ and $(z_j)_j$ being partitions of M_i and z, respectively. Since, for all $i \in [m]$ (i.e. for each row), we have $\mathsf{ct}\big(\langle \bar{\boldsymbol{M}}_i,\, \boldsymbol{z} \rangle\big) = \langle M_i,\, z \rangle$, we must have that $Mz = \mathsf{ct}(\bar{\boldsymbol{M}}\boldsymbol{z})$. □

D.2 Proof of Evaluation Homomorphism (Theorem 5)

Proof. First, we will prove that $c = \mathcal{L}(\boldsymbol{z})$. Since $\mathcal{L}$ is a $\mathrm{R}_\mathbb{F}$-module homomorphism, the following holds

$$c = \sum_{i \in [\ell]} \rho_i c_i = \sum_{i \in [\ell]} \rho_i \mathcal{L}(\boldsymbol{z}_i) = \mathcal{L}\left(\textstyle\sum_{i \in [k]} \rho_i \boldsymbol{z}_i\right) = \mathcal{L}(\boldsymbol{z}).$$

Now, we will prove that $y = \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}}(r)$. Since multilinear evaluation $\boldsymbol{z} \mapsto \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}}(r) = \langle \bar{\boldsymbol{M}}\boldsymbol{z}, \hat{r} \rangle$ is a linear map over $\mathrm{R}_{\mathbb{K}}$ (i.e. is a $\mathrm{R}_{\mathbb{K}}$-module homomorphism), the following holds

$$y = \sum_{i \in [\ell]} \rho_i y_i = \sum_{i \in [\ell]} \rho_i \cdot \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}_i}(r) = \widetilde{\bar{\boldsymbol{M}} \cdot \left(\sum_{i \in [\ell]} \rho_i \cdot \boldsymbol{z}_i \right)}(r) = \widetilde{\bar{\boldsymbol{M}}\boldsymbol{z}}(r)$$

Finally, by Remark 2, we have that for all $i \in [\ell]$, $\mathsf{ct}(y_i) = \widetilde{Mz_i}(r)$ and $\mathsf{ct}(y) = \widetilde{Mz}(r)$. This concludes our proof. □

D.3 Proof of Composition Theorem (Theorem 6)

Proof. Consider an arbitrary expected polynomial-time adversary $(\mathcal{A}, \mathcal{P}^*)$ for the composition $\Pi := \Pi_2 \circ \Pi_1$ with success probability $\epsilon(\mathcal{A}, \mathcal{P}^*) \geq 1/\mathsf{poly}(\lambda)$. Without loss of generality, the adversary $\mathcal{P}^*$ can be split into two adversaries $(\mathcal{P}_1^*, \mathcal{P}_2^*)$ such that given $\mathsf{pp} \leftarrow \mathcal{G}(1^\lambda)$, $(\mathsf{s}, u_1, \mathsf{st}_1) \leftarrow \mathcal{A}(\mathsf{pp})$, and $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s})$,

- $\langle \mathcal{P}_1^*, \mathcal{V}_1 \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}_1) \to (u_2, \mathsf{st}_2)$
- $\langle \mathcal{P}_2^*, \mathcal{V}_2 \rangle((\mathsf{pk}, \mathsf{vk}), u_2, \mathsf{st}_2) \to (u_3, w_3)$

Furthermore, we assume that $\mathcal{A}$ outputs st_1 which contains $(\mathsf{s}, \mathsf{pp})$; otherwise, we could trivially construct an adversary $\mathcal{A}'$ with an identical distribution of prior outputs that does so. First, we construct an adversary $\mathcal{A}_2 := (\mathcal{B}_2, \mathcal{B}_2')$ for Π_2:

$\mathcal{B}_2(\mathsf{pp}) \to (\mathsf{s}, \mathsf{st}_1)$:
1. $(\mathsf{s}, u_1, \mathsf{st}_1) \leftarrow A(\mathsf{pp})$.
2. Output $(\mathsf{s}, \mathsf{st}_1)$.

$\mathcal{B}_2'(\mathsf{st}_1) \to (u_2, \mathsf{st}_2)$:
1. Parse st_1 to obtain $(\mathsf{s}, \mathsf{pp})$.
2. $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s})$.
3. Simulate $(u_2, \mathsf{st}_2) \leftarrow \langle \mathcal{P}_1^*, \mathcal{V}_1 \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}_1)$.
4. Output (u_2, st_2).

$\mathcal{A}_2(\mathsf{pp}) \to (\mathsf{s}, u_2, \mathsf{st}_2)$:
1. $(\mathsf{s}, \mathsf{st}_1) \leftarrow \mathcal{B}_2(\mathsf{pp})$.
2. $(u_2, \mathsf{st}_2) \leftarrow \mathcal{B}_2'(\mathsf{st}_1)$.
3. Output $(\mathsf{s}, u_2, \mathsf{st}_2)$.

Observe that, by construction, the success probability $\epsilon(\mathcal{A}_2, \mathcal{P}_2^*)$ of adversary $(\mathcal{A}_2, \mathcal{P}_2^*)$ for Π_2 is equal to the success probabilty $\epsilon(\mathcal{A}, \mathcal{P}^*)$ of adversary $(\mathcal{A}, \mathcal{P}^*)$ for Π. Since Π_1 is ϕ-restricted, we must have

$$\Pr\left[\begin{array}{c} u_2, u_2' \neq \perp \\ \Downarrow \\ \phi(u_2) = \phi(u_2') \end{array} \middle| \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, \mathsf{st}_1) \leftarrow \mathcal{B}_2(\mathsf{pp}) \\ (u_2, \mathsf{st}_2) \leftarrow \mathcal{B}_2'(\mathsf{st}_1) \\ (u_2', \mathsf{st}_2') \leftarrow \mathcal{B}_2'(\mathsf{st}_1) \end{array}\right] = 1, \tag{1}$$

Thus, we have by (1) and the ϕ-relaxed knowledge soundness of Π_2 that there exists an expected polynomial-time extractor $\mathcal{E}_2$ such that

$$\Pr\left[(\mathsf{pp},\mathsf{s},u_2,w_2)\in\mathcal{R}_2' \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\ (\mathsf{s},u_2,\mathsf{st}_2)\leftarrow\mathcal{A}_2(\mathsf{pp})\\ (\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\ w_2\leftarrow\mathcal{E}_2(\mathsf{pp},\mathsf{s},u_2,\mathsf{st}_2)\end{array}\right]\geq\epsilon(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda) \quad (2)$$

$$\text{and}\qquad \Pr\left[\begin{array}{c}w_2,w_2'\neq\bot\\ \wedge\ w_2\neq w_2'\end{array} \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\ (\mathsf{s},\mathsf{st}_1)\leftarrow\mathcal{B}_2(\mathsf{pp})\\ (u_2,\mathsf{st}_2)\leftarrow\mathcal{B}_2'(\mathsf{st}_1)\\ w_2\leftarrow\mathcal{E}_2(\mathsf{pp},\mathsf{s},u_2,\mathsf{st}_2)\\ (u_2',\mathsf{st}_2')\leftarrow\mathcal{B}_2'(\mathsf{st}_1)\\ w_2'\leftarrow\mathcal{E}_2(\mathsf{pp},\mathsf{s},u_2',\mathsf{st}_2')\end{array}\right]\leq\mathsf{negl}(\lambda) \quad (3)$$

Next, we will construct an adversary $\mathcal{P}_1^{**}$ for Π_1:

$\mathcal{P}_1^{**}(\mathsf{pk},u_1,\mathsf{st}_1)\to w_2$:
1. Parse st_1 to obtain (s,pp).
2. $(\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})$.
3. Simulate $(u_2,\mathsf{st}_2)\leftarrow\langle\mathcal{P}_1^*,\mathcal{V}_1\rangle((\mathsf{pk},\mathsf{vk}),u_1,\mathsf{st}_1)$.
4. $w_2\leftarrow\mathcal{E}_2(\mathsf{pp},\mathsf{s},u_2,\mathsf{st}_2)$.
5. Output w_2.

Observe that, by construction, the relaxed success probability $\epsilon'(\mathcal{A},\mathcal{P}_1^{**})$ of adversary $(\mathcal{A},\mathcal{P}_1^{**})$ for Π_1 is equal to $\epsilon(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda)\geq 1/\mathsf{poly}(\lambda)$ which is the success probability of the relaxed extractor $\mathcal{E}_2$ from equation (2). Furthermore, by equation (3) and construction of $(\mathcal{B}_2,\mathcal{B}_2')$, we must have that

$$\Pr\left[\begin{array}{c}w_2,w_2'\neq\bot\\ \wedge\\ w_2\neq w_2'\end{array} \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathsf{Gen}(1^\lambda)\\ (\mathsf{s},u_1,\mathsf{st}_1)\leftarrow\mathcal{A}(\mathsf{pp})\\ (\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\ (u_2,w_2)\leftarrow\langle\mathcal{P}_1^{**},\mathcal{V}\rangle((\mathsf{pk},\mathsf{vk}),u_1,\mathsf{st}_1)\\ (u_2',w_2')\leftarrow\langle\mathcal{P}_1^{**},\mathcal{V}\rangle((\mathsf{pk},\mathsf{vk}),u_1,\mathsf{st}_1)\end{array}\right]\leq\mathsf{negl}(\lambda) \quad (4)$$

Thus, we have by (4) and the restricted knowledge soundness of Π_1 that there exists an expected polynomial-time extractor $\mathcal{E}_1$ such that

$$\Pr\left[(\mathsf{pp},\mathsf{s},u_1,w_1)\in\mathcal{R}_1 \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\ (\mathsf{s},u_1,\mathsf{st}_1)\leftarrow\mathcal{A}(\mathsf{pp})\\ (\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\ w_1\leftarrow\mathcal{E}_1(\mathsf{pp},\mathsf{s},u_1,\mathsf{st}_1)\end{array}\right]\geq\epsilon(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda) \quad (5)$$

In conclusion, we have constructed an extractor $\mathcal{E}:=\mathcal{E}_1$ with respect to adversary $(\mathcal{A},\mathcal{P}^*)$ such that

$$\Pr\left[(\mathsf{pp},\mathsf{s},u_1,w_1)\in\mathcal{R}_1 \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\ (\mathsf{s},u_1,\mathsf{st}_1)\leftarrow\mathcal{A}(\mathsf{pp})\\ (\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\ w_1\leftarrow\mathcal{E}(\mathsf{pp},\mathsf{s},u_1,\mathsf{st}_1)\end{array}\right]\geq\epsilon(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda).$$

Thus, $\Pi:=\Pi_2\circ\Pi_1$ is knowledge sound. □

D.4 Proofs for Π_{CCS}

We first provide a lemma that will be helpful for both the security and completeness of the interactive reduction.

Lemma 7. *Consider the following arbitrary items:*

$$\textit{structure } \mathsf{s}, \quad \textit{vectors } z_1, \ldots, z_{K+k} \in \mathbb{F}^{n_{\mathbb{F}}},$$
$$\textit{point } r \in \mathbb{K}^{\log m}, \quad \textit{evaluations } \left(\{y_{i,j} \in \mathrm{R}_{\mathbb{K}}\}_{j\in[t]} \right)_{i=K+1}^{K+k}.$$

Similarly to Π_{CCS} *(Section 7.3), define polynomials*

$$\mathsf{F}(\vec{X},\, C) := \sum\nolimits_{i=1}^{K} C^{i-1} \cdot f\Big(\widetilde{M_1 z_i}(\vec{X}),\, \ldots,\, \widetilde{M_t z_i}(\vec{X})\Big)$$

$$\mathsf{NC}(\vec{X},\, C) := \sum\nolimits_{i=1}^{K+k} C^{i-1} \cdot \prod\nolimits_{j=-b-1}^{b-1} \Big(\widetilde{z_i}(\vec{X}) - j\Big)$$

$$\mathsf{Eval}(\vec{X},\, C) := \mathsf{eq}\big(\vec{X},\, r\big) \cdot \sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} C^{\mathrm{I}(i,j,\ell)} \cdot \widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell}(\vec{X})$$

$$Q(\vec{X},\, \vec{A},\, C) := \mathsf{eq}(\vec{X},\, \vec{A}) \cdot \Big(\mathsf{F}(\vec{X},\, C) + C^K \cdot \mathsf{NC}(\vec{X}\, C)\Big) + C^{2K+k} \cdot \mathsf{Eval}(\vec{X},\, C)$$

$$\mathrm{T}(C) := \sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} C^{\mathrm{I}(i,j,\ell)} \cdot \mathsf{cf}(y_{i,j})_\ell$$

where challenges $\alpha \in \mathbb{K}^{\log m}$ *and* $\gamma \in \mathbb{K}$ *are replaced by indeterminate variables* $\vec{A} := (A_1, \ldots, A_{\log m})$ *and* C*, respectively.*
We must have $T(C) = \sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C)$ *if and only if*

1. $f\Big(\widetilde{M_1 z_i}, \ldots, \widetilde{M_t z_i}\Big) \in \mathsf{ZS}_{\log m}$ *for all* $i \in [K]$,
2. *and for all* $i \in [K+k]$, $\|z_i\|_\infty < b$,
3. *and for all* $i \in [K+1, K+k]$ *and* $j \in [t]$, $y_{i,j} = \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r)$.

Proof. By definition of $T(C)$,

$$T(C) = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C)$$

if and only if

$$\sum_{i=K+1}^{K+k} \sum_{j=1}^{t} \sum_{\ell=1}^{d} C^{\mathrm{I}(i,j,\ell)} \cdot \mathsf{cf}(y_{i,j})_\ell = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C) \tag{6}$$

Since powers of C are linearly independent, Equation (6) occurs if and only if

$$\forall\, i \in [K], \qquad 0 = \sum_{\vec{x}\in\{0,1\}^{\log m}} \mathsf{eq}(\vec{x}, \vec{A}) \cdot f\Big(\widetilde{M_1 z_i}(\vec{x}),\, \ldots,\, \widetilde{M_t z_i}(\vec{x})\Big), \tag{7}$$

$$\forall\, i \in [K+k], \qquad 0 = \sum_{\vec{x}\in\{0,1\}^{\log m}} \mathsf{eq}(\vec{x}, \vec{A}) \cdot \prod_{j=-b-1}^{b-1} \Big(\widetilde{z_i}(\vec{x}) - j\Big), \tag{8}$$

$$\begin{aligned} &\forall\, i \in [K+1, K+k], \\ &\forall\, j \in [t], \\ &\forall\, \ell \in [d], \end{aligned} \qquad \mathsf{cf}(y_{i,j})_\ell = \sum_{\vec{x}\in\{0,1\}^{\log m}} \mathsf{eq}(\vec{x},\, r) \cdot \widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell}(\vec{x}) \tag{9}$$

By Lemma 6, Equation (7) and Equation (8) occur if and only if

1. $f\Big(\widetilde{M_1 z_i}, \ldots, \widetilde{M_t z_i}\Big) \in \mathsf{ZS}_{\log m}$ for all $i \in [K]$ (Item 1),
2. $\prod_{j=-b-1}^{b-1} \Big(\widetilde{z_i}(\vec{x}) - j\Big) \in \mathsf{ZS}_{\log m}$ for all $i \in [K+k]$

Over a field, we must have $\prod_{j=-b-1}^{b-1} \Big(\widetilde{z_i}(\vec{x}) - j\Big) \in \mathsf{ZS}_{\log m}$ if and only if for all $i \in [K+k]$, $\|z_i\|_\infty < b$ (Item 2). By definition of multilinear extension and Remark 2, we must have that

$$\begin{aligned} &\forall\, i \in [K+1, K+k], \forall\, j \in [t], \\ &\quad \forall\, \ell \in [d],\ \mathsf{cf}(y_{i,j})_\ell = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} \mathsf{eq}(\vec{x},\, r) \cdot \widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell}(\vec{x}) \quad \text{(Equation (9))} \end{aligned}$$

if and only if

$$\begin{aligned} &\forall\, i \in [K+1, K+k], \forall\, j \in [t], \\ &\qquad y_{i,j} = \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r) \quad \text{(Item 3)} \end{aligned}$$

In conclusion, we have shown

$$T(C) = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C)$$

if and only if (Item 1), (Item 2), and (Item 3). □

Lemma 8. *The interactive reduction* $\Pi_{\mathsf{CCS}} : \mathsf{CCS}(b, \mathcal{L})^K \times \mathsf{CE}(b, \mathcal{L})^k \to \mathsf{CE}(b, \mathcal{L})^{K+k}$ *is* ***complete*** *and* ***public coin****.*

Proof. **Completeness.** Assume the original input tuples belong to relations $\mathsf{CCS}(b, \mathcal{L})$ (Definition 12) and $\mathsf{CE}(b, \mathcal{L})$ (Definition 13). We will first argue that the sum-check verifier in step 2 passes. Then, we will argue that the evaluation claim check in step 4 passes. Finally, we will argue that output tuples belong to $\mathsf{CE}(b, \mathcal{L})^{K+k}$.

By the definition of relations $\mathsf{CCS}(b, \mathcal{L})$ (Definition 12) and $\mathsf{CE}(b, \mathcal{L})$ (Definition 13), we must have that (Item 1), (Item 2), and (Item 3) from Lemma 7 hold. Therefore, we must have that

$$T(C) = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C).$$

Thus, for any choice of challenges $\alpha \in \mathbb{K}^{\log m}$ and $\gamma \in \mathbb{K}$ chosen in step 1,

$$T(\gamma) = \textstyle\sum_{\vec{x}\in\{0,1\}^{\log m}} Q(\vec{x}, \alpha, \gamma).$$

Thus, by the completeness of the sum-check protocol (Definition 6), we must have that the sum-check verifier (step 2) always passes.

By step 3 and Remark 2, we must have that

$$\mathsf{ct}(y'_{i,j}) = \widetilde{M_j z_i}(r')$$

for all $i \in [K+k]$ and $j \in [t]$. Since $M_1 = I_n$, we must have that

$$\mathsf{ct}(y'_{i,1}) = \widetilde{z}_i(r')$$

for all $i \in [K+k]$. Finally, by Remark 2, we must have that

$$\mathsf{cf}\big(y'_{i,j}\big)_\ell = \widetilde{\mathsf{cf}\big(\bar{\boldsymbol{M}}_j \boldsymbol{z}_i\big)_\ell}(r')$$

for all $i \in [K+1, K+k], j \in [t], \ell \in [d]$. By definition of $Q(\vec{X})$ in step 2, we must have that

$$Q(r') = \mathsf{eq}(r',\, \alpha) \cdot \Big(\mathrm{F} + \gamma^K \cdot \mathrm{N}\Big) + \gamma^{2K+k} \cdot \mathrm{E}$$

for values $\mathrm{F}, \mathrm{N}, \mathrm{E}$ derived in step 4. Thus, the verifier check in step 4 passes.

Observe that Π_{CCS} outputs exactly the original structure s, commitments $(c_i)_{i\in[K+k]}$, vectors $(z_i)_{i\in[K+k]}$, and instances $(x_i)_{i\in[K+k]}$. Thus, by the definition of $\mathsf{CCS}(b, \mathcal{L})$, we must have immediately that every condition in $\mathsf{CE}(b, \mathcal{L})$ is satisfied for all the $K+k$ tuples, except that

$$\forall i \in [K+k], j \in [t], \quad y'_{i,j} = \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r').$$

However, this is exactly what is computed by the honest prover in step 3. Therefore, the output tuples do belong to $\mathsf{CE}(b, \mathcal{L})^{K+k}$ as required.

Public coin. The sum-check protocol itself is a public-coin protocol. The remaining randomness from the verifier are the challenges $\alpha \in \mathbb{K}^{\log m}$, $\gamma \in \mathbb{K}$, which are sampled uniformly at random and sent to the prover. □

We prove conditions (i) and (ii) of weak interactive reductions (Definition 9).

Proof.

Proof of (i) By construction, the verifier trivially sets the commitments in the output instance u_2 to be the original commitments $(c_i)_{i\in[K+k]}$ from the input instance u_1. Hence, for repeated executions with respect to the same input instance u_1 with output instances u_2, u'_2, the commitments in these output instances must be the same.

Proof of (ii) Consider an arbitrary expected polynomial-time adversary $(\mathcal{A}, \mathcal{P}^*)$, such that the relaxed success probability of the adversary $\epsilon'(\mathcal{A}, \mathcal{P}^*) \geq 1/\mathsf{poly}(\lambda)$ and

$$\Pr\left[\begin{array}{c|l} & \mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda) \\ w_2, w'_2 \neq \perp & (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ \wedge & (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ w_2 \neq w'_2 & (u_2, w_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V}\rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \\ & (u'_2, w'_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V}\rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}) \end{array}\right] \leq \mathsf{negl}(\lambda) \qquad (10)$$

then we will show that there exists an expected polynomial-time extractor $\mathcal{E}$ such that

$$\Pr\left[(\mathsf{pp},\mathsf{s},u_1,w_1)\in\mathcal{R}_1 \;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\(\mathsf{s},u_1,\mathsf{st})\leftarrow\mathcal{A}(\mathsf{pp})\\(\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\w_1\leftarrow\mathcal{E}(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})\end{array}\right]\geq\epsilon'(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda).$$

Namely, the following extractor $\mathcal{E}$,

$\mathcal{E}(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})\to w_1$:

1. $(\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})$.
2. Assign $\mathsf{result}:=\bot$.
3. While $\mathsf{result}=\bot$:
   - Simulate $\mathsf{result}\leftarrow\langle\mathcal{P}_1^*,\mathcal{V}_1\rangle((\mathsf{pk},\mathsf{vk}),u_1,\mathsf{st}^*)$
   - If $\mathsf{result}\neq\bot$
     - Parse $(u_2,w_2)\leftarrow\mathsf{result}$.
     - If $(\mathsf{pp},\mathsf{s},u_2,w_2)\notin\mathcal{R}_2'$, then set $\mathsf{result}=\bot$.
4. Simulate $\mathsf{result}'\leftarrow\langle\mathcal{P}_1^*,\mathcal{V}_1\rangle((\mathsf{pk},\mathsf{vk}),u_1,\mathsf{st}^*)$
5. If $\mathsf{result}'\neq\bot$:
   - Parse $(u_2',w_2')\leftarrow\mathsf{result}'$.
   - If $(\mathsf{pp},\mathsf{s},u_2',w_2')\notin\mathcal{R}_2'$, then set $\mathsf{result}'=\bot$.
6. If $\mathsf{result}'=\bot$, then output $\bot$.
7. Parse $(u_2,w_2)\leftarrow\mathsf{result}$ and $(u_2',w_2')\leftarrow\mathsf{result}'$.
8. If $w_2\neq w_2'$, then output $\bot$.
9. Parse $(z_1,\ldots,z_K,z_{K+1},\ldots,z_{K+k})\leftarrow w_2$.
10. For all $i\in[K]$, assign $w_i^{\mathsf{CCS}}\leftarrow z_i[n_{\mathsf{F},\mathsf{in}}:]$.
11. Output $w_1:=(w_1^{\mathsf{CCS}},\ldots,w_K^{\mathsf{CCS}},z_{K+1},\ldots,z_{K+k})$.

Extractor runtime. We will show that the extractor $\mathcal{E}$ makes at most $1+1/\epsilon'(\mathcal{A},\mathcal{P}^*)$ calls to $\mathcal{P}^*$ in expectation. Since $\epsilon'(\mathcal{A},\mathcal{P}^*)\geq 1/\mathsf{poly}(\lambda)$, we have that the extractor makes at most a polynomial number of calls to $\mathcal{P}^*$ in expectation. Hence, since $\mathcal{K}$ and $\mathcal{V}_1$ run in $\mathsf{poly}(\lambda)$ time, we have that overall the extractor runs in expected polynomial-time.

By construction, the while loop (Item 3) terminates when the adversary $(\mathcal{A},\mathcal{P}^*)$ succeeds. Since the relaxed success probability is $\epsilon'(\mathcal{A},\mathcal{P}^*)$, the while loop executes $1/\epsilon'(\mathcal{A},\mathcal{P}^*)$ times in expectation. This implies the while loop performs $1/\epsilon'(\mathcal{A},\mathcal{P}^*)$ calls to $\mathcal{P}^*$ in expectation. Finally, Item 4 performs one call to $\mathcal{P}^*$.

Extractor success probability. First, we will show

$$\Pr\left[\begin{array}{c}\mathsf{result}'\neq\bot\\\wedge\\w_2=w_2'\end{array}\;\middle|\; \begin{array}{l}\mathsf{pp}\leftarrow\mathcal{G}(1^\lambda,\mathsf{sz})\\(\mathsf{s},u_1,\mathsf{st})\leftarrow\mathcal{A}(\mathsf{pp})\\(\mathsf{pk},\mathsf{vk})\leftarrow\mathcal{K}(\mathsf{pp},\mathsf{s})\\w_1\leftarrow\mathcal{E}(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})\end{array}\right]\geq\epsilon'(\mathcal{A},\mathcal{P}^*)-\mathsf{negl}(\lambda).\qquad(11)$$

Item 5 exactly checks that the simulated adversary in Item 4 succeeds. Thus, the event that $\mathsf{result}' \neq \bot$ occurs with probability $\epsilon'(\mathcal{A}, \mathcal{P}^*)$. Assume that the event $\mathsf{result}' \neq \bot$ occurs. By (10), $w_2 \neq w_2'$ with at most $\mathsf{negl}(\lambda)$ probability. Thus, all together, we have (11) holds.

Assume that the event $\mathsf{result}' \neq \bot \wedge w_2 = w_2'$ occurs, which implies the extractor outputs a witness $w_1 := (w_1^{\mathsf{CCS}}, \ldots, w_K^{\mathsf{CCS}}, z_{K+1}, \ldots, z_{K+k}) \neq \bot$ (as the extractor passes the checks in Item 6 and Item 8). We will show that $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \notin \mathcal{R}_1$ with probabilty at most $\mathsf{negl}(\lambda)$. Hence,

$$\Pr\left[(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}) \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}) \\ w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \end{array}\right]$$

$\geq \big(\epsilon'(\mathcal{A}, \mathcal{P}^*) - \mathsf{negl}(\lambda)\big) - \mathsf{negl}(\lambda) = \epsilon'(\mathcal{A}, \mathcal{P}^*) - \mathsf{negl}(\lambda)$.

Since $\mathsf{result}' \neq \bot$, by construction (Item 5), we must have $(\mathsf{pp}, \mathsf{s}, u_2', w_2') \in \mathcal{R}_2'$ and during the simulation $\langle \mathcal{P}_1^*, \mathcal{V}_1 \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st}^*)$ in Item 4, the verifier $\mathcal{V}_1$ did not abort. Namely, that the sum-check verifier in (step 2) did not abort and the evaluation checks (step 4) were satisfied. By definition of $\mathcal{R}_2' = \mathsf{CE}(q/2, \mathcal{L})^{K+k}$, we must know that for all $i \in [K+k]$,

$$\boldsymbol{x}_i := \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}_i) \qquad \text{and} \qquad c_i := \mathcal{L}(\boldsymbol{z}_i). \tag{12}$$

By the definition of $\mathcal{L}_{\mathsf{in}}$ (Definition 14), Equation (12) also implies that for all $i \in [K+k]$, the first $n_{\mathbb{F},\mathsf{in}}$ entries of z_i are equal to x_i (from input instance u_1).

Assume that $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \notin \mathcal{R}_1$. Recall that $\mathcal{R}_1 := \mathsf{CCS}(b, \mathcal{L})^K \times \mathsf{CE}(b, \mathcal{L})^k$. By Equation (12), we know that all conditions in $\mathsf{CCS}(b, \mathcal{L})$ and $\mathsf{CE}(b, \mathcal{L})$, except for the norm-bound ($\|z\|_\infty < b$), evaluation ($y_j = \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r)$), or CCS requirements ($f(\widetilde{M_1 z}, \ldots, \widetilde{M_t z}) \in \mathsf{ZS}_{\log m}$), are satisfied. Thus, using notation from Lemma 7, $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \notin \mathcal{R}_1$ implies that either:

1. there exists an $i \in [K]$, $f\left(\widetilde{M_1 z_i}, \ldots, \widetilde{M_t z_i}\right) \notin \mathsf{ZS}_{\log m}$,
2. OR there exists an $i \in [K+k]$, $\|z_i\|_\infty \geq b$,
3. OR there exists an $i \in [K+1, K+k]$, $j \in [t]$, $y_{i,j} \neq \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r)$.

By Lemma 7, we must have

$$T(C) \neq \textstyle\sum_{\vec{x} \in \{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C).$$

Let's focus on the second simulation of Π_{CCS} in step 4. Note, that the randomness used in the second simulation of the Π_{CCS} (extractor step 4) is fresh and independent of the first simulation of the Π_{CCS} (extractor step 3). Since $(\mathsf{pp}, \mathsf{s}, u_2', w_2') \in \mathcal{R}_2'$, we must have the the prover's claimed evaluations in protocol step 3 are true. Thus, by the construction of the verifier's checks in protocol step 4, we must have that the sum-check evaluation claim $v = Q(r')$ is true.

Recall that the witness from the first simulation, w_2, agrees with the witness from the second simulation, w_2'. Thus, in order for the sum-check verifier to have passed in the second simulation, either the adversary $\mathcal{P}^*$

- succeeded in the sum-check protocol, despite $T(C) \neq \sum_{\vec{x} \in \{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C)$ (in other words, violated the soundness of the sum-check protocol)
- OR the non-zero polynomial

$$P(C, \vec{A}) := T(C) - \sum_{\vec{x} \in \{0,1\}^{\log m}} Q(\vec{x}, \vec{A}, C)$$

evaluated to zero on random point $(\gamma, \alpha) \in \mathbb{K}^{1+\log m}$.

By the soundness error of the sum-check protocol (Definition 6), the first event occurs with probability at most $\epsilon_{\mathsf{SC}} := \max(u, 2b+1, 2) \cdot \log m / |\mathbb{K}|$, where u is the degree of f, b is the norm bound, 2 comes from $\mathsf{Eval}(\vec{X})$. By the Schwartz–Zippel lemma, the second event occurs with probability at most $\epsilon_{\mathsf{SZ}} := (2K+k)\max(\log m, ktd)/|\mathbb{K}|$. Thus, by union bound, $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \notin \mathcal{R}_1$ with probability at most $\mathsf{negl}(\lambda) := \epsilon_{\mathsf{SC}} + \epsilon_{\mathsf{SZ}}$. □

D.5 Proofs for Π_{RLC}

Lemma 9. *The interactive reduction* $\Pi_{\mathsf{RLC}} : \mathsf{CE}(b, \mathcal{L})^{K+k} \to \mathsf{CE}(B, \mathcal{L})$ *is* ***complete*** *and* ***public coin****.*

Proof. **Completeness.** By definition of $\mathsf{CE}(b, \mathcal{L})$ (Definition 13), we must have the input tuples exactly satisfy the conditions in Theorem 5. Thus, we have that the output tuple satisfies all of the requirements of $\mathsf{CE}(B, \mathcal{L})$, except for the requirement that $\|z\|_\infty < B = b^k$.

However, we show that this bound follows from the expansion factor T of $\mathcal{C}$ chosen in Definition 14:

$$\begin{aligned}\|\boldsymbol{z}\|_\infty &= \left\|\sum_{i=1}^{k+K} \rho_i \boldsymbol{z}_i\right\|_\infty \leq \sum_{i=1}^{k+K} \|\rho_i \boldsymbol{z}_i\|_\infty \\ &\leq \sum_{i=1}^{k+K} T \|\boldsymbol{z}_i\|_\infty \leq (k+K)T(b-1) < B\end{aligned}$$

where the second inequality is from the expansion factor of $\mathcal{C}$ being T, the third inequality is from the definition of $\mathsf{CE}(b, \mathcal{L})$, which enforces a norm bound of b, and the last inequality is by assumption (Definition 14). Hence, the output tuple must belong to $\mathsf{CE}(B, \mathcal{L})$.

Public coin. The verifier's randomness consists of challenges $\rho_1, \ldots, \rho_{k+K}$, which are sampled uniformly at random from $\mathcal{C}$ and sent to the prover. □

We prove the conditions of strong interactive reductions (Definition 10).

Proof. Consider an arbitrary expected-polynomial time adversary $(\mathcal{A}, \mathcal{P}^*)$ for Π_{RLC} with success probability, $\epsilon(\mathcal{A}, \mathcal{P}^*) \geq 1/\mathsf{poly}(\lambda)$. First, we can construct an adversary and verification function for Theorem 10,

$A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}(\vec{c})$:

1. Execute encoder $(\mathsf{pk},\mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp},\mathsf{s})$.
2. Simulate $(u_2, w_2) \leftarrow \langle \mathcal{P}^*(\mathsf{pk}, u_1, \mathsf{st}),\ \mathcal{V}(\mathsf{vk}, u_1)\rangle$ with verifier randomness $\vec{c}$.
3. Output w_2

$V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}(\vec{c}, w_2) \to \{0,1\}$:

1. Execute encoder $(\mathsf{pk},\mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp},\mathsf{s})$.
2. Simulate $(u_2, \_) \leftarrow \langle \mathcal{P}^*(\mathsf{pk}, u_1, \mathsf{st}),\ \mathcal{V}(\mathsf{vk}, u_1)\rangle$ with verifier randomness $\vec{c}$.
3. Output accept if and only if $(u_2, w_2) \in \mathsf{CE}(B, \mathcal{L})$.

Let $E_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ be the corresponding extractor from Theorem 10. We define $E(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})$ as the trivial algorithm that executes $E_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ by simulating calls to $A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$. We construct an extractor for adversary $(\mathcal{A}, \mathcal{P}^*)$ as follows:

$\mathcal{E}(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})$:

1. $\mathsf{result} \leftarrow E(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})$.
2. If $u_1 = \bot$ or $\mathsf{result} = \bot$, output $\bot$.
3. Parse $(\vec{c}, w'), (\vec{c}_1, w'_1), \ldots (\vec{c}_{K+k}, w'_{K+k}) \leftarrow \mathsf{result}$.
4. Parse $z \leftarrow w'$ and $\rho_1, \ldots, \rho_{K+k} \leftarrow \vec{c}$.
5. For $i \in [K+k]$,
   (a) Parse $z^{(i)} \leftarrow w'_i$ and $\rho_1^{(i)}, \ldots, \rho_{K+k}^{(i)} \leftarrow \vec{c}_i$.
   (b) Assign $\boldsymbol{z}_i \leftarrow \left(\rho_i - \rho_i^{(i)}\right)^{-1} \cdot (\boldsymbol{z} - \boldsymbol{z}^{(i)})$.
6. Parse $(c_i,\ x_i,\ r,\ \{y_{i,j}\}_{j\in[t]})_{i\in[K+k]} \leftarrow u_1$.
7. Output $w_1 := (z_i)_{i\in[K+k]}$ if and only if

$$\left(\mathsf{s};\ c_i, x_i,\ r, \{y_{i,j}\}_{j\in[t]};\ z_i\right)_{i\in[K+k]} \in \mathsf{CE}(q/2, \mathcal{L})^{K+k}$$

Extractor runtime. By Theorem 10, we are guaranteed $E_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ makes in expectation at most $(K+k)+1$ calls to $A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$. Hence, our overall extractor $\mathcal{E}$ runs in expected polynomial time.

Extractor success probability. By Theorem 10, we are guaranteed that $E(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})$ outputs $(K+k)+1$ pairs $(\vec{c}, w'), (\vec{c}_1, w'_1), \ldots (\vec{c}_{K+k}, w'_{K+k})$ such that

- $V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}(\vec{c}, w') = 1$,
- for all $i \in [K+k]$, $V(\vec{c}_i, w'_i) = 1$, and
- $(\vec{c}, \vec{c}_1, \ldots, \vec{c}_{K+k}) \in \mathsf{SS}(\mathcal{C}, K+k)$

with probability $\epsilon^{V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}}(A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}) - \frac{(K+k)+1}{|\mathcal{C}|}$. Since $A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ and $V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ simulate the interaction between $\mathcal{P}^*$ and $\mathcal{V}$ and checks if the output pair (u_2, w_2) belongs to $\mathsf{CE}(B, \mathcal{L})$, we must have that

$$\epsilon^{V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}}(A_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}) - \frac{(K+k)+1}{|\mathcal{C}|} = \epsilon(\mathcal{A}, \mathcal{P}^*) - \frac{(K+k)+1}{|\mathcal{C}|}. \tag{13}$$

Assume that this event above occurs. Since $V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}(\vec{c},w')=1$ and $V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ executes Π_{RLC}'s $\mathcal{V}$ and outputs accept if and only if $(u_2,w_2)\in\mathsf{CE}(B,\mathcal{L})$, we must have for $\boldsymbol{x}:=\sum_{i=1}^{K+k}\rho_i\boldsymbol{x}_i$ that

$$\left(\begin{array}{c} c:=\sum_{i=1}^{K+k}\rho_i c_i, \\ \mathsf{s}; \quad x,\ r, \\ \left\{y_j:=\sum_{i=1}^{K+k}\rho_i y_{i,j}\right\}_{j\in[t]} \end{array} ;\ z\right)\in\mathsf{CE}(B,\mathcal{L}) \tag{14}$$

where $(c_i,x_i,r,\{y_{i,j}\}_j)_i$ are the instance elements in u_1 (parsed in step 6) and $z\leftarrow w'$ and $(\rho_i)_i\leftarrow\vec{c}$ are the elements parsed in step 4.

For all $i\in[K+k]$, we will make a similar argument to the one directly above. Namely, since $V(\vec{c}_i,w_i')=1$, we must have for $\boldsymbol{x}^{(i)}:=\sum_{i=1}^{K+k}\rho_i^{(i)}\boldsymbol{x}_i$ that

$$\left(\begin{array}{c} c^{(i)}:=\sum_{i=1}^{K+k}\rho_i^{(i)} c_i, \\ \mathsf{s}; \quad x^{(i)},\ r, \\ \left\{y_j^{(i)}:=\sum_{i=1}^{K+k}\rho_i^{(i)} y_{i,j}\right\}_{j\in[t]} \end{array} ;\ z^{(i)}\right)\in\mathsf{CE}(B,\mathcal{L}) \tag{15}$$

where $(c_i,x_i,r,\{y_{i,j}\}_j)_i$ are in u_1 and $z^{(i)}\leftarrow w_i'$ and $(\rho_j^{(i)})_j\leftarrow\vec{c}_i$ are the elements parsed in step 5a. By definition of $\mathsf{CE}(B,\mathcal{L})$ (Definition 13), we must have

$$c=\mathcal{L}(\boldsymbol{z}),\quad c^{(i)}=\mathcal{L}(\boldsymbol{z}^{(i)}),\quad \boldsymbol{x}=\mathcal{L}_{\mathsf{in}}(\boldsymbol{z}),\quad \boldsymbol{x}^{(i)}=\mathcal{L}_{\mathsf{in}}(\boldsymbol{x}^{(i)}) \tag{16}$$

Since $(\vec{c},\vec{c}_1,\ldots,\vec{c}_{K+k})\in\mathsf{SS}(\mathcal{C},K+k)$, we must have for all $i\in[K+k]$ that

$$(\rho_1,\ldots,\rho_{K+k})\equiv_i(\rho_1^{(i)},\ldots,\rho_{K+k}^{(i)}) \tag{17}$$

which means the challenges differ only on index i. By definition of strong sampling set (Definition 17), we must have $(\rho_i-\rho_i^{(i)})$ is invertible for all $i\in[K+k]$.

Thus, by Equation (16) and Equation (17), we have for all $i\in[K+k]$,

$$\begin{aligned} c-c^{(i)}&=\mathcal{L}(\boldsymbol{z})-\mathcal{L}(\boldsymbol{z}^{(i)}) \\ \boldsymbol{x}-\boldsymbol{x}^{(i)}&=\mathcal{L}_{\mathsf{in}}(\boldsymbol{z})-\mathcal{L}_{\mathsf{in}}(\boldsymbol{z}^{(i)}) \end{aligned}$$

$$\begin{aligned} \textstyle\sum_{i=1}^{K+k}\rho_i c_i-\sum_{i=1}^{K+k}\rho_i^{(i)}c_i&=\mathcal{L}(\boldsymbol{z})-\mathcal{L}\big(\boldsymbol{z}^{(i)}\big), \\ \textstyle\sum_{i=1}^{K+k}\rho_i \boldsymbol{x}_i-\sum_{i=1}^{K+k}\rho_i^{(i)}\boldsymbol{x}_i&=\mathcal{L}_{\mathsf{in}}(\boldsymbol{z})-\mathcal{L}_{\mathsf{in}}\big(\boldsymbol{z}^{(i)}\big) \end{aligned} \tag{18}$$

$$\begin{aligned} \big(\rho_i-\rho_i^{(i)}\big)\cdot c_i&=\mathcal{L}(\boldsymbol{z})-\mathcal{L}\big(\boldsymbol{z}^{(i)}\big), \\ \big(\rho_i-\rho_i^{(i)}\big)\cdot \boldsymbol{x}_i&=\mathcal{L}_{\mathsf{in}}(\boldsymbol{z})-\mathcal{L}_{\mathsf{in}}\big(\boldsymbol{z}^{(i)}\big) \end{aligned} \tag{19}$$

$$\begin{aligned} c_i&=\mathcal{L}\Big(\big(\rho_i-\rho_i^{(i)}\big)^{-1}\cdot\big(\boldsymbol{z}-\boldsymbol{z}^{(i)}\big)\Big), \\ \boldsymbol{x}_i&=\mathcal{L}_{\mathsf{in}}\Big(\big(\rho_i-\rho_i^{(i)}\big)^{-1}\cdot\big(\boldsymbol{z}-\boldsymbol{z}^{(i)}\big)\Big) \end{aligned} \tag{20}$$

$$c_i = \mathcal{L}(\boldsymbol{z}_i), \qquad \boldsymbol{x}_i = \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}_i) \tag{21}$$

where equation (18) follows from (14), (15), and Equation (16). Equation (19) follows from the equivalence in Equation (17). Equation (20) follows from $\mathcal{L}, \mathcal{L}_{\mathsf{in}}$ being $\mathcal{R}$-module homomorphisms and $\mathcal{C}$ being a strong sampling set (Definition 17) which because $\rho_i \neq \rho_i^{(i)}$ (guaranteed by (17)) means $\rho_i - \rho_i^{(i)}$ is invertible. Equation (21) is by construction (step 5b).

We make a similar argument for the evaluations. In particular, by the definition of $\mathsf{CE}(B, \mathcal{L})$ (Definition 13), Equation (14), and Equation (15), we must have that

$$y_j := \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r), \quad y_j^{(i)} := \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}^{(i)}}(r) \tag{22}$$

Thus, we must have for all $i \in [K+k]$ and $j \in [t]$,

$$\begin{aligned}
y_j - y_j^{(i)} &= \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r) - \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}^{(i)}}(r) && (23)\\
\textstyle\sum_{i=1}^{K+k} \rho_i y_{i,j} - \sum_{i=1}^{K+k} \rho_i^{(i)} y_{i,j} &= \widetilde{\bar{\boldsymbol{M}}_j (\boldsymbol{z} - \boldsymbol{z}^{(i)})}(r) && (24)\\
\left(\rho_i - \rho_i^{(i)}\right) \cdot y_{i,j} &= \widetilde{\bar{\boldsymbol{M}}_j (\boldsymbol{z} - \boldsymbol{z}^{(i)})}(r) && (25)\\
y_{i,j} &= \widetilde{\bar{\boldsymbol{M}}_j \Big((\rho_i - \rho_i^{(i)})^{-1} \cdot (\boldsymbol{z} - \boldsymbol{z}^{(i)}) \Big)}(r) && (26)\\
&= \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r) && (27)
\end{aligned}$$

where Equation (23) follows from Equation (22), Equation (24) follows from Equation (14) and Equation (15) and the linearity of multilinear evaluation, Equation (25) follows from the equivalence (17), and (26) follows from $\mathcal{C}$ being a strong sampling set (Definition 17) which because $\rho_i \neq \rho_i^{(i)}$ (guaranteed by (17)) means $\rho_i - \rho_i^{(i)}$ is invertible.

Therefore, Equation (13), by Equation (20), and Equation (26), we must have with probability $\epsilon(\mathcal{A}, \mathcal{P}^*) - ((K+k)+1)/|\mathcal{C}|$, the extractor outputs witness elements $z_1, \ldots, z_{K+k}$ such that

$$\left(\mathsf{s};\ c_i, x_i,\ r, \{y_{i,j}\}_{j\in[t]};\ z_i\right)_{i\in[K+k]} \in \mathsf{CE}(q/2, \mathcal{L})^{K+k},$$

for the trivial norm bound of $q/2$, as any element in $\mathbb{F}$ satisfies this bound.

Now, assume that $\mathcal{A} := (\mathcal{B}, \mathcal{B}')$ such that

$$\Pr\left[\begin{array}{c} u_1, u_1' \neq \perp \\ \Downarrow \\ \phi(u_1) = \phi(u_1') \end{array} \middle| \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ (\mathsf{s}, \mathsf{st}^*) \leftarrow \mathcal{B}(\mathsf{pp}) \\ (u_1, \mathsf{st}) \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ (u_1', \mathsf{st}') \leftarrow \mathcal{B}'(\mathsf{st}^*) \end{array}\right] = 1 \tag{28}$$

We will show that

$$\Pr\left[\begin{array}{l|l} & \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, \mathsf{sz}) \\ & (\mathsf{s}, \mathsf{st}^*) \leftarrow \mathcal{B}(\mathsf{pp}) \\ w_1, w_1' \neq \bot & (u_1, \mathsf{st}) \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ \wedge\ w_1 \neq w_1' & w_1 \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st}) \\ & (u_1', \mathsf{st}') \leftarrow \mathcal{B}'(\mathsf{st}^*) \\ & w_1' \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1', \mathsf{st}') \end{array}\right] \leq \mathsf{negl}(\lambda) \tag{29}$$

Assume that the event $w_1, w_1' \neq \bot \ \wedge\ w_1 \neq w_1'$ occurs. Since $w_1, w_1' \neq \bot$, we have that $u_1, u_1' \neq \bot$ (otherwise, the extractor $\mathcal{E}$ would have outputted $\bot$).

1. By Equation (28), we must have that $\phi(u_1) = \phi(u_1')$, which guarantees the instances share identical commitments $(c_i)_{i\in[K+k]}$.
2. Define $(z_i)_{i\in[K+k]} = w_1$ and $(z_i')_{i\in[K+k]} = w_1'$. Then, $w_1 \neq w_1'$ implies that there exist an $i \in [K+k]$ such that $z_i \neq z_i'$.

During the execution of $\mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st})$, the call to algorithm $E(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st})$ produces elements $\rho_i, \rho_i^{(i)}, z, z^{(i)}$. Similarly, during the execution of $\mathcal{E}(\mathsf{pp}, \mathsf{s}', u_1', \mathsf{st}')$, the call to algorithm $E(\mathsf{pp}, \mathsf{s}', u_1', \mathsf{st}')$ produces elements $\rho_i', \rho_i^{(i)\prime}, z', z^{(i)\prime}$. These elements satisfy the following

$$z_i \neq z_i' \iff \left(\rho_i - \rho_i^{(i)}\right)^{-1} \cdot (\boldsymbol{z} - \boldsymbol{z}^{(i)}) \neq \left(\rho_i' - \rho_i^{(i)\prime}\right)^{-1} \cdot (\boldsymbol{z}' - \boldsymbol{z}^{(i)\prime}) \tag{30}$$

$$\|z\|_\infty < B,\ \left\|z^{(i)}\right\|_\infty < B,\ \|z'\|_\infty < B,\ \left\|z^{(i)\prime}\right\|_\infty < B \tag{31}$$

Equation (30) follows from (Item 2) and by the construction of $\mathcal{E}$. Equation (31) follows from the construction of $\mathcal{E}$, which only outputs $w_1, w_1' \neq \bot$ when the internal extractor E (from Theorem 10) succeeds. In particular, the internal extractor E succeeding guarantees that the verification function $V_{(\mathsf{pp},\mathsf{s},u_1,\mathsf{st})}$ accepts. This verification function checks that output tuples (corresponding to Equation (14) and Equation (15)) belong to $\mathsf{CE}(B, \mathcal{L})$, which exactly checks the require norm bound on vectors $z, z^{(i)}, z', z^{(i)\prime}$. By Item 1 and (20), we must have

$$c_i = \mathcal{L}\Big(\left(\rho_i - \rho_i^{(i)}\right)^{-1} \cdot \left(\boldsymbol{z} - \boldsymbol{z}^{(i)}\right)\Big) = \mathcal{L}\Big(\left(\rho_i' - \rho_i^{(i)\prime}\right)^{-1} \cdot \left(\boldsymbol{z}' - \boldsymbol{z}^{(i)\prime}\right)\Big)$$

Thus, since $\mathcal{L}$ is a R-module homomorphism, we have

$$\left(\rho_i - \rho_i^{(i)}\right) \cdot c_i = \mathcal{L}\left(\boldsymbol{z} - \boldsymbol{z}^{(i)}\right)\ \wedge\ \left(\rho_i' - \rho_i^{(i)\prime}\right) \cdot c_i = \mathcal{L}\left(\boldsymbol{z}' - \boldsymbol{z}^{(i)\prime}\right) \tag{32}$$

All together, by (30), (31), and (32), we have that $(c_i,\ \Delta_1 = \rho_i - \rho_i^{(i)},\ \Delta_2 = \rho_i' - \rho_i^{(i)\prime},\ z_1 = z - z^{(i)},\ z_2 = z' - z^{(i)\prime})$ is a $(2B, \mathcal{C})$-relaxed binding collision (Definition 4).

By assumption (Definition 14), $\mathcal{L}$ is a ring commitment scheme that satisfies $(2B, \mathcal{C})$-relaxed binding. Thus, the probability of the original event (Equation (29)) must be less than or equal to $\mathsf{negl}(\lambda)$. Otherwise, we could construct a corresponding relaxed-binding adversary which executes the extractor $\mathcal{E}$ twice to retrieve the corresponding elements for the $2B$-relaxed binding collision with probability $\epsilon_{\mathsf{rlx}}(B, \mathcal{C}) \geq \mathsf{negl}(\lambda)$. □

D.6 Π_{DEC} is a Reduction of Knowledge (Theorem 7)

Proof. **Completeness:** First, we show that the verifier's checks in step 2 pass. Then, we will show that the output tuples belongs to $\mathsf{CE}(b, \mathcal{L})^k$.

By the definition of $\mathsf{CE}(B, \mathcal{L})$, we must have that $\|z\|_\infty < B = b^k$ (Definition 14). Thus, by definition of split_b, we must have $z = \sum_{i=1}^k b^{i-1} \cdot z_i$. Therefore,

$$
\begin{aligned}
z &= \textstyle\sum_{i=1}^k b^{i-1} \cdot z_i \\
\boldsymbol{z} &= \textstyle\sum_{i=1}^k b^{i-1} \cdot \boldsymbol{z}_i && (33) \\
\mathcal{L}(\boldsymbol{z}) &= \mathcal{L}\big(\textstyle\sum_{i=1}^k b^{i-1} \cdot \boldsymbol{z}_i\big), \\
c &= \textstyle\sum_{i=1}^k b^{i-1} \cdot \mathcal{L}(\boldsymbol{z}_i), && (34) \\
c &= \textstyle\sum_{i=1}^k b^{i-1} \cdot c_i, && (35)
\end{aligned}
$$

Equation (34) follows directly from $\mathcal{L}$ being a R-module homomorphism. Equation (35) follows by construction of $c_i \leftarrow \mathcal{L}(z_i)$ in step 1. Starting from equation (33), we must have for all $j \in [t]$,

$$
\begin{aligned}
\boldsymbol{z} &= \textstyle\sum_{i=1}^k b^{i-1} \cdot \boldsymbol{z}_i \\
\bar{\boldsymbol{M}}_j \boldsymbol{z} &= \textstyle\sum_{i=1}^k b^{i-1} \cdot \bar{\boldsymbol{M}}_j \boldsymbol{z}_i \\
\widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r) &= \widetilde{\textstyle\sum_{i=1}^k b^{i-1} \cdot \bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r) \\
\widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r) &= \textstyle\sum_{i=1}^k b^{i-1} \cdot \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r) && (36) \\
y_j &= \textstyle\sum_{i=1}^k b^{i-1} \cdot y_{i,j} && (37)
\end{aligned}
$$

Equation (36) follows directly from the linearity of multilinear evaluation. Equation (37) follows by definition of $\mathsf{CE}(B, \mathcal{L})$ and by construction $y_{i,j} \leftarrow \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r)$ in step 1. Thus, by (35) and (37), we have the verifier's checks must pass.

Next, we show that the output tuple, $(\mathsf{s};\ \{c_i,\ x_i,\ r,\ \{y_{i,j}\}_{j\in[t]}\}_{i\in[k]};\ \{z_i\}_{i\in[k]})$, belongs to $\mathsf{CE}(b, \mathcal{L})^k$. By the definition of split_b, we must have that $\|z_i\|_\infty < b$ for all $i \in [k]$. Since $\mathcal{L}_{\mathsf{in}}$ is the trivial $\mathcal{R}$-module homomorphism which projects the first $n_{\mathrm{R,in}}$ columns, we must have that, by construction in step 2, that $\boldsymbol{x}_i = \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}_i)$ for all $i \in [k]$. Thus, in total, we must have, along with the construction of $\big(c_i,\ \{y_{i,j}\}_{j\in[t]}\big)_{i\in[k]}$ in step 1, that the output tuples belong to $\mathsf{CE}(b, \mathcal{L})^k$.

Public coin: The verifier uses no randomness in this protocol. Thus, the protocol is trivially public coin.

Knowledge soundness: Consider an arbitrary expected-polynomial time adversary $(\mathcal{A}, \mathcal{P}^*)$ for Π_{DEC} with success probability, $\epsilon(\mathcal{A}, \mathcal{P}^*) \geq 1/\mathsf{poly}(\lambda)$. We construct an extractor $\mathcal{E}$ for Π_{DEC} as follows,

$\mathcal{E}(\mathsf{pp},\ \mathsf{s},\ u_1 := (c,\ x,\ r,\ (y_j)_{j\in[t]}),\ \mathsf{st})$:

1. Execute encoder $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s})$.
2. Simulate $(u_2, w_2) \leftarrow \langle \mathcal{P}^*(\mathsf{pk}, u_1, \mathsf{st}),\ \mathcal{V}(\mathsf{vk}, u_1)\rangle$.
3. If $u_2 = \perp$, output $\perp$.
4. Parse $(z_1, \ldots, z_k) \leftarrow w_2$.
5. Output $w_1 := \sum_{i=1}^{k} b^{i-1} z_i$.

Extractor runtime: The extractor runs in expected polynomial time, since it simulates only one execution between the adversary $\mathcal{P}^*$ and verifier $\mathcal{V}$, which both run in expected polynomial time.

Extractor success probability: Assume that the simulated adversary $(\mathcal{A}, \mathcal{P}^*)$ succeeds in convincing the verifier $\mathcal{V}$ and the parties jointly output $(\mathsf{s}, u_2, w_2) \in \mathsf{CE}(b, \mathcal{L})^k$; note that this occurs with probability $\epsilon(\mathcal{A}, \mathcal{P}^*)$. Define

$$(c_i,\ x_i,\ r, (y_{i,j})_{j\in[t]})_{i\in[k]} := u_2 \text{ and } z_1, \ldots, z_k := w_2.$$

By the definition of $\mathsf{CE}(b, \mathcal{L})$, we have for all $i \in [k]$ and $j \in [t]$,

$$c_i := \mathcal{L}(\boldsymbol{z}_i), \quad \boldsymbol{x}_i := \mathcal{L}_{\mathsf{in}}(\boldsymbol{z}_i), \quad \|z_i\|_\infty < b \quad \text{and} \quad y_{i,j} := \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}_i}(r) \tag{38}$$

Since the adversary convinces the verifier, we must have that for all $j \in [t]$,

$$c = \textstyle\sum_{i=1}^{k} b^{i-1} \cdot c_i \qquad \text{and} \qquad y_j = \sum_{i=1}^{k} b^{i-1} \cdot y_{i,j} \tag{39}$$

By construction in step 2 (i.e. definition of split_b), we also must have $x = \sum_{i=1}^{k} b^{i-1} \cdot x_i$. By defining $z := \sum_{i=1}^{k} b^{i-1} z_i$, observe that $x = \sum_{i=1}^{k} b^{i-1} \cdot x_i$, (38), and (39) satisfy the remaining conditions stated in Theorem 5. We must have $c = \mathcal{L}(\boldsymbol{z})$, $\boldsymbol{x} = \mathcal{L}_{\mathsf{in}}(\boldsymbol{z})$, and for all $j \in [t]$, $y_j := \widetilde{\bar{\boldsymbol{M}}_j \boldsymbol{z}}(r)$. Since in (38), we have for all $i \in [k]$, $\|z_i\|_\infty < b$, we must also have $\|z\|_\infty < B = b^k$. These are exactly the conditions for $(\mathsf{s};\ u_1 := \{c,\ x,\ r,\ \{y_j\}_{j\in[t]}\};\ w_1 := z)$ to belong to $\mathsf{CE}(B, \mathcal{L})$. Therefore, since the adversary succeeds with probability $\epsilon(\mathcal{A}, \mathcal{P}^*)$, we must have by construction, that $\mathcal{E}$ outputs a satisfying witness such that $(\mathsf{s}, u_1, w_1) \in \mathsf{CE}(B, \mathcal{L})$ with probability $\epsilon(\mathcal{A}, \mathcal{P}^*)$. □

D.7 Finding choices of cyclotomic and fields

```
# [LS18, eprint 2017-523] pg 6
# m is the cyclotomic polynomial index
def tau(m):
    return m if (m % 2) != 0 else m / 2

# [LS18, eprint 2017-523] Thm 1.1, pg 4
# m is the cyclotomic polynomial index
# p is the prime
# z is any divisor of m
```

```
# This tests for the condition for thm 1.1 to hold
def thm1_1_cond(m, p, z):
    cond1 = (p % z) == 1
    cond2 = Mod(p,m).multiplicative_order() == m/z
    return cond1 and cond2

# [LS18, eprint 2017-523] Thm 1.1, pg 4
# p is the prime
# z is any divisor of m
# lInf bound for elements to be invertible
# given that m,p,z satisfy thm 1.1 cond
def thm1_1_inv_bound(p, z):
    return (1/s1(z)*p^(1/euler_phi(z))).n()

def thm1_1_num_factors(z):
    return euler_phi(z)

# Output divisors of m
def divisors(m):
    zs = list()
    for i in range(1,m+1):
        if m % i == 0:
            zs.append(i)
    return zs

# [LS18, eprint 2017-523] pg 6, pg 9
# We only consider prime power cyclotomics
# m is the cyclotomic polynomial index
def s1(m):
    return sqrt(tau(m))

# checks if cyclotomic index m is power of two
def is_pow2(m):
    return sum(m.digits(2)) == 1

# [MR09] lattice-based cryptography
# makes sure characteristic does not lead
# to trivial bound
def non_trivial(q, n, d, delta):
    return (q/2).n() >= (2^(2 * sqrt( n*d * log(q,2) * log(delta, 2)))).n()

# [AL21] eprint Prop 2. 2021/202
# for all u,v in R, |u*v| / |v| <= gamma*|u|
# outputs T = gamma * |u|
# assumes we are only testing prime powers
def expansion_factor(m, norm):
    if is_pow2(m):
        return euler_phi(m) * norm
    else:
        return 2 * euler_phi(m) * norm
```

```
# p is prime
# max_idx is max cyclotomic index
# outputs list of (m, z)
def candidates(p, min_idx=10, max_idx=200):
    # prime powers
    possible_indices = [i for i in range(min_idx, max_idx) if len(factor(i)) == 1]
    c = list()
    for m in possible_indices:
        zs = divisors(m)
        for z in zs:
            if thm1_1_cond(m, p, z):
                c.append((Integer(m), Integer(z)))
    return c

def pre_filter(q, cyclotomic_index, z, n, m, chals):
    chals_norm = max({abs(c) for c in chals})
    chals_max_diff = chals[-1] - chals[0]
    delta = 1.0045 # root hermite factor, chosen from [ESSLL19] eprint 2018/773
    phi = cyclotomic_polynomial(cyclotomic_index) # index cyclotomic polynomial
    d = phi.degree() # degree of cyclotomic

    # return non_trivial(q, n, d, delta) and chals_max_diff < thm1_1_inv_bound(q, z) and log(len(chals
    # We remove non_trivial(...) because we use the lattice estimator for hardness
    return chals_max_diff < thm1_1_inv_bound(q, z) and log(len(chals)^d,2).n() >= 120

def info(q, cyclotomic_index, z, n, m, chals):
    chals_norm = max({abs(c) for c in chals})
    chals_max_diff = chals[-1] - chals[0]
    delta = 1.0045 # root hermite factor, chosen from [ESSLL19] eprint 2018/773
    phi = cyclotomic_polynomial(cyclotomic_index) # index cyclotomic polynomial
    d = phi.degree() # degree of cyclotomic
    T = expansion_factor(cyclotomic_index, chals_norm)

    # Bounds for MSIS to be hard
    # [MR09] lattice-based cryptography pg 6
    # [CMNW24] pg 38 eprint 2024/281
    MSIS_B_l2_bound = min(q, 2^(2 * sqrt( n*d * log(q,2) * log(delta, 2))))
    MSIS_B_linf_bound = MSIS_B_l2_bound / sqrt(m*d)

    # We need MSIS infinity bound 8TB to be hard
    B = MSIS_B_linf_bound / (8*T)

    print("####")
    print("Cyclotomic idx:", cyclotomic_index)
    print("Cyclotomic Poly:", phi)
    print("z:", z)
    #print("Prime is non-trivial?", non_trivial(q, n, d, delta))
    print("Csmall norm is small enough?", chals_max_diff < thm1_1_inv_bound(q, z))
    print("Csmall large enough?", log(len(chals)^d,2).n() >= 120)
```

```
    print("Degree of Cyclotomic:", d)
    # print("log(B):", log(B, 2).n())
    print("Expansion Factor T:", T)
    print("Invertible Norm bound:", thm1_1_inv_bound(q, z))
    print("log(|C_Small|):", log(len(chals)^d,2).n())
    print("Factors of Cyclotomic:", thm1_1_num_factors(z))
    print()

def possible_settings(q, n, m, chals):
    for (cyclotomic_index, z) in candidates(q):
        if pre_filter(q, cyclotomic_index, z, n, m, chals):
            info(q, cyclotomic_index, z, n, m, chals)
        else:
            delta = 1.0045
            d = cyclotomic_polynomial(cyclotomic_index).degree()
            print("[Does not satisfy security requirements] index: {}, degree: {}, z: {}, non_trivial

# Primes:
GL = 2^64 - 2^32 + 1
AGL = GL - 32
print("################################")
print("AGL ################################")
print("################################")
# MSIS settings
n = 13 # rows, kappa in latticefold
m = 2^26 # cols
# Small Challenge set
chals = [-1, 0, 1, 2]
possible_settings(AGL, n, m, chals)
print("################################")
print("M61 ################################")
print("################################")
# MSIS settings
n = 16 # rows, kappa in latticefold
m = 2^22 # cols
# Small Challenge set
chals = [-2, -1, 0, 1, 2]
possible_settings(2^61-1, n, m, chals)
print("################################")
print("GL ################################")
print("################################")
# MSIS settings
n = 16 # rows, kappa in latticefold
m = 2^24 # cols
# Small Challenge set
chals = [-2, -1, 0, 1, 2]
possible_settings(GL, n, m, chals)
print("################################")
```

D.8 Lattice Estimator Script

```
from estimator import *
Logging.set_level(Logging.LEVEL0)

M61 = 2^61 -1
GL = 2^64 - 2^32 +1
AGL = GL - 32
b = 2

n = 15
d = 64
k = 13
K = 26
B = b^k
m = 2^33 / d

T = 128
q = AGL

n_sis = n*d
m_sis = m*d
B_l2 = sqrt(m*d)*(8*T*B)

params = SIS.Parameters(n=n_sis, q=q, m=m_sis,length_bound=B_l2, norm=2)
_ = SIS.estimate(params)
print((K+k)*T*(b-1) < B)

n = 18
d = 54
k = 14
K = 61
B = b^k
m = 2^30 / d

T=216
q = GL

n_sis = n*d
m_sis = m*d
B_l2 = sqrt(m*d)*(8*T*B)

params = SIS.Parameters(n=n_sis, q=q, m=m_sis,length_bound=B_l2, norm=2)
_ = SIS.estimate(params)
print((K+k)*T*(b-1) < B)

n = 18
d = 54
k = 14
K = 61
```

```
B = b^k
m = 2^28 / d

T = 216
q = M61

n_sis = n*d
m_sis = m*d
B_l2 = sqrt(m*d)*(8*T*B)

params = SIS.Parameters(n=n_sis, q=q, m=m_sis,length_bound=B_l2, norm=2)
_ = SIS.estimate(params)
print((K+k)*T*(b-1) < B)
```